

RML: Theory and practice of a domain specific language for runtime verification

Davide Ancona^{a,*}, Luca Franceschini^a, Angelo Ferrando^b, Viviana Mascardi^a

^a DIBRIS – via Dodecaneso 35, University of Genova, 16146 Genova, Italy

^b University of Manchester, UK

ARTICLE INFO

Article history:

Received 1 May 2020

Received in revised form 5 January 2021

Accepted 5 January 2021

Available online 12 January 2021

Keywords:

Runtime verification

Domain specific language

Operational semantics

ABSTRACT

Runtime verification (RV) is an approach to verification consisting in dynamically checking that the event traces generated by single runs of a system under scrutiny (SUS) are compliant with the formal specification of its expected correct behavior.

RML (Runtime Monitoring Language) is a simple but powerful Domain Specific Language (DSL) for RV which is able to express non context-free properties. When designing RML, particular care has been taken to favor abstraction and simplicity, to better support reusability and portability of specifications and interoperability of the monitors generated from them; this is mainly achieved by decoupling the two problems of property specification and event generation, and by minimizing the available primitive constructs.

The formalization and implementation of RML is based on a trace calculus with a fully deterministic rewriting semantics. The semantics of RML is defined by translation into such a calculus, which, in fact, is used as intermediate representation (IR) by the RML compiler. The effectiveness of RML and its methodological impact on RV are presented through interesting patterns that can be adapted to different contexts requiring verification of standard properties. A collection of tested examples is provided, together with benchmarks showing that the deterministic semantics and the performed dynamic optimizations based on the laws of the trace calculus significantly improve the performances of the generated monitors.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

RV [1] is a general approach to correctness consisting in dynamically checking that the event traces generated by single runs of a SUS are compliant with the formal specification of its expected correct behavior.

The inputs of the RV process (Fig. 1) are the SUS and the specification of the properties to be verified. The specification is usually defined in either a domain specific language (DSL) or a programming language, and it denotes sets of possibly infinite event traces; a trace is called *valid* according to the specification if and only if it belongs to such a set. From the specification a monitor is synthesized: it consumes the observed events generated by the SUS, emits verdicts and, possibly, some feedback which is used for error recovery when monitoring continues after deployment. Essentially a monitor deals with a word problem, since its aim is to check that a trace of events (the word) belongs to the set of valid traces defined by the specification from which it has been generated.

* Corresponding author.

E-mail addresses: davide.ancona@unige.it (D. Ancona), luca.franceschini@dibris.unige.it (L. Franceschini), angelo.ferrando@manchester.ac.uk (A. Ferrando), viviana.mascardi@unige.it (V. Mascardi).

<https://doi.org/10.1016/j.scico.2021.102610>

0167-6423/© 2021 Elsevier B.V. All rights reserved.

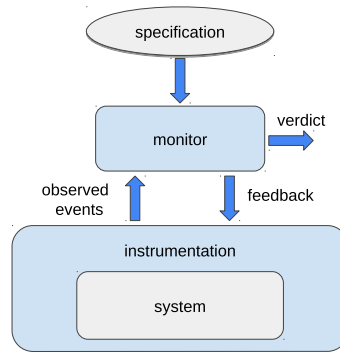


Fig. 1. Overview of the RV process.

Since at each time the monitor inspects only a finite prefix of the possibly infinite trace generated by the run of the SUS, the verdicts emitted after each observed event may be inconclusive [1] and for this reason a 4-valued logic [2] is used where the two conclusive verdicts *False* and *True* are coupled with the two inconclusive ones *MaybeFalse* and *MaybeTrue*.

The problem of event generation is usually dealt with code instrumentation of the SUS, to allow the monitor to observe the relevant events; although specification and instrumentation can be mixed together, decoupling the two problems favor abstraction and simplicity, as explained below. During the verification process the monitor analyzes the events generated by the execution of the SUS through the instrumentation; the analysis can be “online” if the monitor checks the events at runtime, or “offline” if monitoring starts after the SUS has been halted.

RV is complementary to other verification methods: as formal verification, it is based on a specification formalism, but scales well to real systems and complex properties, by forgoing exhaustiveness as software testing; however, it also exhibits several distinguishing features: as opposite to formal verification and testing, it is more suitable to check control rather than data-oriented properties [3], and offers opportunities for runtime contract enforcement, fault protection and automatic program repair when monitoring continues after deployment. Furthermore, while testing often requires the use of a programming language,¹ RV allows the adoption of DSL languages with several advantages: they are easier to learn, they favor readability, portability and reuse of specifications and interoperability of the generated monitors. Another reason for preferring a DSL over a programming language is that a DSL provides more correctness guarantees: a monitor synthesized from a specification written in a higher level DSL has less chances to be faulty when compared with a monitor implemented by hand in an ordinary programming language.

For the above reasons, in this paper we focus on the specification part of the picture in Fig. 1 and assume that instrumentation is strictly separated from the specification and the monitor generated from it, so that specifications can be written independently of the instrumentation infrastructure. Instrumented code has the only aim of sending events to the monitor in a suitable format, and to react to any feedback, if any; for simplicity, however, in this work we assume that monitors are only used to verify code before deployment, hence we deliberately neglect all aspects related to monitor feedback.

To achieve reusability of specifications and interoperability of synthesized monitors, the DSL should impose minimal assumptions on the instrumented code and the generated events, in order to be as system agnostic as possible; if this happens, then, not only the same DSL can be used for RV of different systems, but also the specification of a certain property can be written once for all, and then easily adapted to be reused in many contexts; in this way, the generated monitor can be used as a universal oracle to test different implementations of the same specification. To this aim, we require the DSL to be based on an event format general enough to guarantee system agnosticism; furthermore, if the synthesized monitor is based on a language-independent and universally recognized data format to represent events, monitors can better inter-operate with heterogeneous components. This further favors decoupling instrumentation from specification issues which is important when runtime monitoring is used for verifying or debugging distributed systems; for instance, specifications can take advantage of useful properties associated with the monitored events, such as global clock timestamps or geographic localization, when the instrumentation is able to generate these types of information.

While RV is well suited for control-oriented properties, verifying data-oriented properties often requires an expressive power higher than that supported by many RV tools which are often based on regular expressions, temporal logics or context-free (CF) grammars; for instance, all these formalisms do not allow RV of FIFO properties. Furthermore, when a more expressive formalism is used, non-determinism [3] may introduce serious performance penalties; this problem already arises in a relatively simple formalism as extended regular expressions: consider, for instance, the issue of Regular expression Denial of Service (ReDoS) [4]. For a more expressive formalism, non-determinism can be even more challenging because the problem of automatically determinizing a specification could be undecidable. Therefore an effective DSL for RV must be able to reconcile its expressive power and the efficiency of the generated monitors.

¹ Except for model-based testing.

RML is a rewriting-based RV DSL designed to meet the above mentioned requirements; it allows users to specify event types and sets of event traces and the compiler generates deterministic monitors which work independently of the SUS and of its instrumentation. The language is inspired by previous work on trace expressions and global types [5–8], which have been adopted for runtime verification in several contexts. RML is characterized by the following features:

- it is a simple but expressive language able to monitor non-CF properties;
- it is based on the data-interchange format JSON to model events and favor interoperability;
- its formal semantics is expressed by translation in a trace calculus whose rewrite semantics is proved to be deterministic; the calculus serves as IR for the RML compiler;
- it is implemented through a compiler² which generates SWI-Prolog monitors.

Overview. This paper reports on the past four years of research devoted to RML, its design, implementation and uses [9]. After introducing in Section 2 the general characteristics of RML throughout simple examples, in Section 3 we present the semantics of RML in terms of a trace calculus and its labeled transition system. Section 4 and 5 formalize the semantics of event pattern matching and the translation from RML to the trace calculus, while Section 6 provides RML adaptable patterns to verify resource management, LIFO, and FIFO properties. In Section 7 we provide some details on the implementation of RML; experiments and benchmarks on the running examples are reported in Section 8 together with a more substantial case study consisting in the verification of the safe interaction between iterators and their containers. Section 9 presents the most prominent related work on RV and their corresponding DSLs and tools, while Section 10 contains final remarks and directions for future developments. All formal proofs can be found in Appendix A.

2. Runtime monitoring language

This section is a gentle introduction to RML; the main concepts, the syntax and constructs of the language are presented through simple examples.

2.1. Events

To support portability and interoperability, RML is based on a general object model where events are represented as JavaScript object literals. Besides being a standard way to represent objects, this choice integrates well with the lightweight data-interchange format JSON to transmit events in a distributed system.

As an example,

```
{event:'func_post', name:'fs.open', args:['foo.txt'], res:42}
```

specifies the event 'function call `fs.open('foo.txt')` has returned value 42'.

Events can be structured, since properties can be associated not only with primitive data values, but also with arrays or objects; the current implementation of RML supports booleans, numbers and strings as primitive data values. For instance,

```
{event:'func_post', name:'http.request', args:[{headers:{'content-length':13}}, resId:24]}
```

denotes the event 'function `http.request` has been called with one argument, namely, the object `{headers:{'content-length':13}}`, and has returned an object identified by id 24'. While top-level objects as `{event:'func_post', ...}` always represent events, nested objects like `{'content-length':13}` usually correspond to structured data values carried by events.

Because they are kept separate, instrumentation and specification have to agree on the adopted *event domain*, that is, the shape and intended semantics of the event objects; however, as shown in Section 2.2, the notion of *event type* allows flexible adaptation to different SUS with no modification of the instrumentation and of the specification of traces. The two examples above share the same domain defined by a Node.js instrumentation to trace function and method calls for monitoring Node.js distributed components, but other kinds of domains for different systems may be considered. For instance, in the context of the JADE platform [10] for multi-agent systems,

```
{action:'send', sender:'alice', receiver:'bob', performative:'tell', content:'auction started'}
```

denotes the event 'alice has told bob that the auction has been started'.

2.2. Event types

Event types are an abstraction mechanism useful to enhance readability, portability and reuse of specifications and the expressive power of RML.

An event type is a predicate which defines a possibly infinite set of events and corresponds to what is also referred as symbolic event in the RV community [11]. In RML event types are referred in specifications by means of *event type patterns*,

² <https://rmlatdibris.github.io/downloads.html>.

terms built on top of possibly overloaded event type names with different arities, and subterms consisting of variables and data values of primitive, array, or object type. An event type pattern specifies the set of events that are expected to occur at a certain point in the event trace; since event type patterns can contain variables, upon a successful match a substitution is computed to bind the variables of the pattern with the data values carried by the matched event.

Let us consider the simple constant event type `relevant` defined below, which denotes all events that are relevant for checking a certain property:

```
// event and name required, other properties are optional
relevant matches
  {event:'func_post', name:'fs.open'} |
  {event:'func_pre', name:'fs.write' | 'fs.close'};
```

According to the definition, the type matches only events of shape

```
{event:'func_post', name:'fs.open'}
```

or

```
{event:'func_pre', name:'fs.write' | 'fs.close'}.
```

In the former case, events are objects with properties `event` and `name` with value `'func_post'` and `'fs.open'`, respectively, and possibly other properties, with the meaning 'call to method `fs.open` has returned'; in the latter case, events are objects with property `event` with value `'func_pre'`, property `name` with value either `'fs.write'` or `'fs.close'`, and possibly other properties, with the meaning 'method `fs.write` or `fs.close` has been called'.

RML supports two basic predefined event types denoted by `none` and `any`, which match no event and all events, respectively. The former has an empty definition, while the latter matches `{}`, that is, any event object.

A non-constant event type can be defined as follows:

```
open(fd) matches {event:'func_post', name:'fs.open', res:fd};
```

Event type `open` has arity 1, and its argument consists of the file descriptor (an integer value) returned by a call to `fs.open`. The definition is parametric in the variable `fd` which can be bound only when the corresponding event is matched with the information of the file descriptor associated with the property `res`; for instance, `open(42)` matches all events of shape `{event:'func_post', name:'fs.open', res:42}`, that is, all returns from call to `fs.open` with value 42.

Wildcards are useful to match events which must contain certain properties with unspecified values:

```
sensor_values(time) matches {temperature:_, humidity:_, timestamp:time};
```

The type `sensor_values(time)` matches all events corresponding to new sensor measurements of temperature and humidity with timestamp `time`; in this case, an event which does not contain property `temperature` or `humidity` cannot match the type `sensor_values(time)`.

Arrays in events may be useful for tracing arguments of function or method calls; ellipsis can be used when the remaining arguments are not relevant.

```
head_req(reqId) matches {event:'func_post', name:'http.request', args:[{method:'HEAD'}, ...], res:reqId};
```

Type `head_req(reqId)` matches any event corresponding to a return from a call to `http.request` with value `reqId` where the first argument is an object with property `method` associated with `'HEAD'`.

For conciseness, event types can also be defined in terms of other event types.

```
file_op(fd) matches write(fd) | read(fd);
```

The event type pattern `file_op(fd)` matches all events corresponding to operations on a file descriptor `fd`, and can be directly derived from the event type patterns `write(fd)` or `read(fd)`.

As shown in the last part of this section, the expressive power of RML can be increased if event types are closed by negation; to this aim, RML supports also negative definitions of event types.

```
irrelevant not matches
  {event:'func_post', name:'fs.open'} |
  {event:'func_pre', name:'fs.write' | 'fs.close'};
```

Event type `irrelevant` matches all events which do not have shape

```
{event:'func_post', name:'fs.open'}
```

or

```
{event:'func_pre', name:'fs.write' | 'fs.close'};
```

equivalently, and more directly, the type can be defined as the negation of the previously defined event type `relevant`:

```
irrelevant not matches relevant;
```

2.3. Basic layer

RML consists of three layers: a basic layer, providing a set of primitive and derived operators, and two others that increasingly enhance the expressive power of the language by allowing *parametric* and *generic* specifications, respectively.

2.3.1. Introductory examples

In RML properties are specified as set of event traces; in the basic layer specifications are obtained by combining event type patterns with primitive and derived operators; four primitive binary left-associative operators are provided, listed in decreasing order of precedence:

- concatenation (juxtaposition) enforces sequentiality;
- intersection ($\wedge$) requires simultaneous verification of multiple properties;
- union ($\vee$) expresses alternatives;
- shuffle ($()$) allows interleaving of events in traces.

Furthermore, the unary postfix operator `!`, with higher precedence on the other operators, can be used to consider as valid all prefixes of a set of traces and, thus, change the verdicts emitted by monitors (see below).

Recursion is supported, and the keyword `empty` denotes the singleton set containing the empty trace.

Thanks to recursion and the primitive operators, several useful operators can be derived, as shown in the rest of this paper. As a first simple example, let us consider the standard postfix operators `?`, `+` and `*` borrowed from regular expressions: for any expression *exp*, $(exp)?$ is equivalent to `empty  $\vee$  (exp)`, while $(exp)^*$ and $(exp)^+$ correspond to the following RML specifications `Star` and `Plus`, respectively:

```
Star = empty  $\vee$  (exp) Star // exp*
Plus = (exp) Star // exp+
```

Another useful derived operator is the constant `all` which denotes the universe of all traces, and is an abbreviation for `any*`.

Specification of iterator objects is a first simple example of use of RML.

```
hasNext(res) matches
{event:'func_post', name:'hasNext', res:res};
next matches {event:'func_pre', name:'next'};

Main = (hasNext(true) next) * hasNext(false);
```

Listing 1: Specification of a single iterator.

In this case there is only the main definition, but in general, a specification may consist of more definitions, where one must be associated with the name `Main`. To avoid ambiguities, the first letter of a specification name and of an event type must be uppercase and lowercase, respectively.

The specification above is a strong version which also enforces best practice:

1. `next` can be called only immediately after `hasNext(true)`;
2. `hasNext` can only be called once per try;
3. `hasNext(true)` requires `next` to be called;
4. the iterator must be fully consumed.

Weaker versions³ where only property 1 and a subset of the others are checked could be considered as well.

The specification combines event types `hasNext` and `next` with concatenation and the derived `*` operator. This specification looks very similar to an EBNF context-free grammar, where event type patterns and specification names play the role of terminal and non-terminal symbols, respectively. However, the examples in Section 6 show that RML is also able to express non context-free properties.

The monitor generated from the RML specification above can also check the behavior of an infinite iterator, that is, an object where method `hasNext()` always returns true; as long as methods `hasNext()` and `next()` are called in the correct order as specified by `Main`, the monitor will keep checking the system without reporting any error. While this may seem a rather artificial example, it is not difficult to think of more complex examples for realistic scenarios where a server requires a continuous monitoring.

The specification above is of limited use because it allows monitoring of a single iterator, but with RML it is possible to scale the example to more iterator instances. We first provide a simple generalization which is based on the shuffle operator and is able to monitor two distinct iterators at the same time.

³ See <https://rmlatdibris.github.io/examples/solution-iter>.

```

hasNext(target,res) matches {event:'func_post', name:'hasNext', targetId:target, res:res};
next(target) matches {event:'func_pre', name:'next', targetId:target};

Iterator1 = (hasNext(1,true) next(1)) * hasNext(1,false);
Iterator2 = (hasNext(2,true) next(2)) * hasNext(2,false);
Main = Iterator1 | Iterator2;

```

Listing 2: Example of use of the shuffle operator: specification of two iterators.

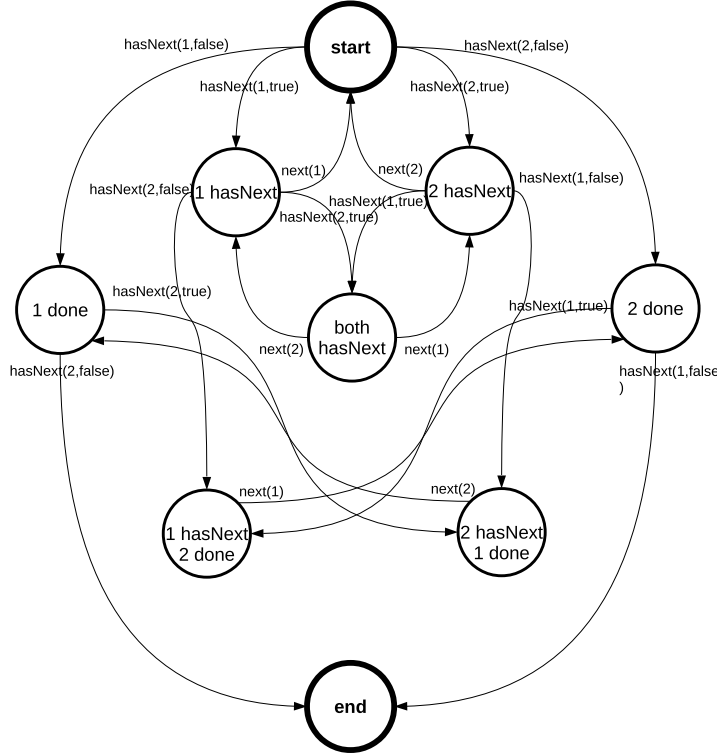


Fig. 2. A state machine specifying the correct behavior of two distinct iterators.

Event types `hasNext` and `next` have now an additional argument which identifies the involved iterator; `Iterator1` and `Iterator2` are generalizations of the specification in Listing 1 instantiated with two different iterator identifiers; they can be correctly shuffled together because their events are independent. Although not strictly necessary in this case, the shuffle operator allows a more concise specification; for instance, an equivalent specification could be provided by the state machine in Fig. 2 which can be directly translated into RML by only using the union and concatenation operators, but both the specifications (the state machine, and the corresponding RML specification) are more complex. The example in Listing 2 provides a first step towards scalability of the iterator specification, but suffers from a serious issue: it is possible to deal only with a statically fixed (and limited) number of iterators with a-priori known identifiers.

Section 2.4 below shows how this example can be further generalized to support a specification fully parametric in the number and identifiers of the monitored iterators.

2.3.2. Scalable and compositional specifications with shuffle and intersection

We have already seen in Listing 2 that shuffle allows composition of independent properties and, hence, favors scalability of specifications; intersection is also a crucial operation which is often used in combination with shuffle. While concatenation and union are familiar operators used in many contexts, including regular expressions and context-free grammars, shuffle and intersection are not so widespread, therefore some starting example is needed to explain why they are so useful in RML and, more in general, in RV and monitoring, to favor conciseness and readability of specifications. More complex examples provided in Section 6 show that shuffle and intersection actually increase the expressive power of RML.

Let us consider the specification of the alternating bit protocol [12] which has to verify the following three properties, where event types `msg(1)`, `msg(2)`, `ack(1)`, and `ack(2)` are assumed to be mutually disjoint:

1. for any event e_1 matching `msg(1)` there must be a subsequent event e_2 matching `ack(1)` and no other event matching `msg(1)` is allowed to occur between e_1 and e_2 ;
2. for any event e_1 matching `msg(2)` there must be a subsequent event e_2 matching `ack(2)` and no other event matching `msg(2)` is allowed to occur between e_1 and e_2 ;
3. for any event e_1 matching `msg(1)` there must be a subsequent event e_2 matching `msg(2)` and no other event matching `msg(1)` is allowed to occur between e_1 and e_2 .

Because of the independence of their event types, the first two properties can be conveniently combined with the shuffle operator:

```
Prop1_2 = (msg(1)ack(1))* | (msg(2)ack(2))*;
```

However, the same consideration does not apply to property 3; in this case, intersection is needed to allow a compositional specification:

```
no_msg not matches msg(_);

Prop1_2 = (msg(1)ack(1))* | (msg(2)ack(2))*;
Prop3 = (msg(1) no_msg* msg(2) no_msg*);
Main = Prop1_2 ^ Prop3;
```

Listing 3: Alternating bit protocol with the intersection and shuffle operators.

Event type `no_msg` matches all events that do not match `msg(_)`, and is needed because `Prop3` involves only events of type `msg(1)` or `msg(2)`, while `Prop1_2` checks also events of type `ack(1)` or `ack(2)`; indeed, the combination `Prop1_2 ^ (msg(1)msg(2))*` specifies only the empty trace.

As happens for the iterator example in Listing 2, also in this case the specification of the alternating bit protocol can be obtained in a non-compositional way by defining a state machine that is able to monitor the three properties above and that can be directly translated into an RML specification which is, however, much less readable.

Property 3 in Listing 3 has been specified with a standard regular expression, but the obtained specification is not so simple to parse and understand. Thanks again to the independence of the involved event types, shuffle allows a more concise and readable specification:

```
no_msg not matches msg(_);

Prop1_2 = (msg(1)ack(1))* | (msg(2)ack(2))*;
Prop3 = (msg(1) msg(2))* | no_msg*;
Main = Prop1_2 ^ Prop3;
```

Listing 4: A more concise specification of the alternating bit protocol with the intersection and shuffle operators.

The pattern shown in Listing 4 obtained through the combination of the shuffle and intersection operators occurs so often when composing specifications of different properties that it justifies the introduction of the derived *filter* operator, presented below.

2.3.3. Monitor verdicts and prefix closure

Any time a monitor receives an event, it emits a verdict in the logic [2] consisting of the four values *False*, *MaybeFalse*, *MaybeTrue*, *True*. Given a finite trace p and a possibly infinite trace t , $p \cdot t$ denotes the trace obtained by appending t at the end of p ; a possibly infinite trace t is called a *continuation* of a finite trace p if and only if there exists a trace t' s.t. $t = p \cdot t'$. Given a finite trace p and a set T of valid traces, verdict v holds w.r.t. p and T if and only if the following properties are verified:

- $v=False$ iff all continuations of p do not belong to T ;
- $v=MaybeFalse$ iff $p \notin T$, but verdict *False* does not hold w.r.t. p and T ;
- $v=MaybeTrue$ iff $p \in T$, but verdict *True* does not hold w.r.t. p and T ;
- $v=True$ iff all continuations of p belong to T .

From the definition above it follows that there always exists a unique verdict which holds w.r.t. p and T for any finite trace p and set of valid traces T . Based on such a definition, we can introduce the following monitor properties [13].

Let s be an RML specification denoting the set T_s of valid traces; a monitor M generated from s is *sound* for verdict v if and only if for any finite trace p , if v is emitted by M after inspecting p , then v holds w.r.t. p and the set of traces T_s denoted by s .

A monitor M generated from a specification s is *complete* for verdict v if and only if for any finite trace p , if v holds w.r.t. p and the set of traces T_s denoted by s , then v is emitted by M after inspecting p .

If a monitor is sound for *MaybeTrue/True* (resp. *MaybeFalse/False*), then false negatives (resp. positives) cannot occur, while if a monitor is complete for *MaybeTrue/True* (resp. *MaybeFalse/False*), then negatives (resp. positives) are detected as soon as possible.

To understand the difference between verdicts, let us consider monitors $M_i, i = 1, \dots, 4$, generated from the four specifications in Listing 5, where `eventType` $\neq$ `any`, and the inspected trace $p = [e]$ consisting of a single event e matching `eventType`. M_1 emits *True*, because p is a valid trace for `Spec1`, as well as any continuation $p \cdot t$, since by definition `all` accepts all traces t . M_2 emits *MaybeTrue* because p is a valid trace for `Spec2` and $[ee']$ is not a valid trace for `Spec2` if e' does not match `eventType`; indeed, monitoring cannot be stopped, because events not matching `eventType` could be observed. M_3 emits *MaybeFalse*, because p is not valid for `Spec3`, while $[ee]$ is valid; indeed, the monitor has to wait the next event to turn its verdict into *MaybeTrue* or *False*. Finally, M_4 emits *False* because $p \cdot t$ is not valid for `Spec4` for all traces t .

```
Spec1 = eventType all;
Spec2 = eventType*;
Spec3 = eventType eventType;
Spec4 = empty;
```

Listing 5: Specifications leading to different monitor verdicts.

`Spec1` and `Spec2` show that constants `empty` and `all` play an important role in determining the verdict of a monitor synthesized from an RML specification; there are cases when one would like to consider as valid all prefixes of a set of traces, so that the monitor emits verdict *MaybeTrue* instead of *MaybeFalse*, as long as the incoming events are matched by the specification. For instance, the specification for one iterator in Listing 1 requires that the iterator must be entirely consumed, therefore a sound and complete synthesized monitor has to emit *MaybeFalse* for all correct events that precede the final event matching `hasNext(false)`; a more relaxed version of the specification, where the iterator is not required to be fully consumed can be easily obtained in a compositional way by using the prefix closure operator `!`.

```
hasNext(res) matches
{event:'func_post', name:'hasNext', res:res};
next matches {event:'func_pre', name:'next'};

Main = ((hasNext(true) next) * hasNext(false))!;
```

Listing 6: Prefix closure: an iterator which is not required to be fully consumed.

Thanks to the prefix closure operator, all prefixes of traces valid for `(hasNext(true) next) * hasNext(false)` are now considered valid.

2.3.4. Filter operators

At the beginning of Section 2.3 we have shown how the constant `all` and the usual postfix operators of regular expressions can be derived in RML; these are not the only useful derived operators in RML.

The specification of `Prop3` in Listing 4 can be further simplified by using a filter operator:

```
msg matches msg(_);

Prop1_2 = (msg(1)ack(1))* | (msg(2)ack(2))*;
Prop3 = msg >> (msg(1) msg(2))*;
Main = Prop1_2 ^ Prop3;

|labellist:filter
```

Listing 7: Alternating bit protocol with the filter operator.

The specification `msg >> (msg(1) msg(2))*` denotes the set of all traces verifying `(msg(1) msg(2))*` when only all events matching event type `msg` are kept; event type `msg` matches all events matching `msg(_)`.

The `>>` operator can be generalized into a conditional filter which takes an additional operand: `eventType >> Spec1 : Spec2` denotes the set of all traces verifying `Spec1` when only all events matching `eventType` are kept, and `Spec2` when only all events not matching `eventType` are kept. The less general version `eventType >> Spec` presented above is equivalent to `eventType >> Spec : all`.

Section 3.3 shows that conditional filter can be derived from the shuffle, intersection and star operators, with the assumption, as it is the case of RML, that event types are closed w.r.t. negation.

2.4. Parametric layer

The parametric layer of RML is built on top of the basic one; as suggested by its name, it allows the definition of specifications parametric in data values that can be known only at runtime. For instance, the specifications for iterators built with the basic layer in Listings 1 and 2 can deal only with a statically fixed (and limited) number of iterators with a-priori known identifiers. To be able to monitor a dynamically changing set of iterators, a specification needs to depend on variables that have to be necessarily instantiated at runtime. The parametric layer provides a construct for declaring variables, delimiting their scope, and allowing their dynamic instantiation through event matching.

```
hasNext(id,res) matches {event:'func_post', name:'hasNext', targetId:id, res:res};
next(id) matches {event:'func_pre', name:'next', targetId:id};
newIter(id) matches {event:'func_post', name:'iterator', resId:id};

Main =
{let id; newIter(id)
  ((hasNext(id,true) next(id)) * hasNext(id,false) | Main)}?;
```

Listing 8: Parametric specification of iterators.

The `newIter` event type is parametric w.r.t. the object identifier of the newly created iterator returned by the invocation of the corresponding method. Accordingly, event types `hasNext` and `next` must trace the identifier of the iterator on which they are called, because several iterators may operate at the same time.

The parametricity of `Main` is expressed by the declaration of `id`, which is used in all event types; the left-most use (in `newIter(id)`) binds `id` with an iterator identifier, as soon as an event matches `newIter(id)`, that is, a method `iterator` is called and returns a new iterator identity; correspondingly, the remaining event types on the left-hand-side of the shuffle operator can only match method invocations on the iterator with the same identifier.

The specification can be scaled to arbitrary numbers of coexisting iterators thanks to the shuffle operator, since events involving distinct iterators are mutually independent; with recursion, the declaration of `id` can be arbitrarily nested, so that different object ids can be correctly managed at the same time, because the innermost declaration hides the outer ones.

The `?` operator applied to the whole specification allows the synthesized monitor to emit the *MaybeTrue* verdict when all iterations have been completed. As shown in Listing 6, the prefix closure operator can be used to relax the specification in case iterators are not required to be fully consumed.

2.5. Generic layer

The generic layer is added on top of the parametric one to favor modularity and further increase the expressive power of the RML. For instance, the specification in Listing 8 can be generalized to scale the definition to an arbitrary number of iterators.

```
// event types as in Listing 8
Iterator<id> =
  newIter(id) (hasNext(id,true) next(id)) * hasNext(id,false);
```

Listing 9: Parametric specification of iterators with a generic.

The specification `Iterator<id>` is generic in the identifier `id` of the iterator; parameter `id` will hold a data value passed as argument when `Iterator` is instantiated in the main specification.

Generic specifications can be quite expressive when combined with recursion and the conditional expression; `Msg<inf,sup>` given below defines the specification `msg(inf)msg(inf+1)..msg(sup)`, parametric in `inf` and `sup`.

```
Msg<inf,sup> =
  if(inf<=sup) msg(inf) Msg<inf+1,sup> else empty;
Main = Msg<1,4>*; // (msg(1)msg(2)msg(3)msg(4))*
```

Listing 10: Example of generic recursive specification with conditional expression.

3. Trace calculus

This section defines the trace calculus on which the semantics and the implementation of RML is based; an RML specification is compiled into a term of the trace calculus which, in fact, serves as IR for the implementation of RML. A monitor is associated with a term of the trace calculus, representing the current state of the monitor; a valid event triggers a state transition in the monitor, according to the rewriting rules, which define the semantics of the calculus presented in this section.

v	$::= l \mid \{k_1:v_1, \dots, k_n:v_n\} \mid [v_1, \dots, v_n]$	(data value)
b	$::= x \mid l \mid \{k_1:b_1, \dots, k_n:b_n\} \mid [b_1, \dots, b_n]$	(basic data expression)
θ	$::= \tau(b_1, \dots, b_n)$	(event type pattern)
t	$::= \epsilon$	(empty trace)
	$\mid \theta : t$	(prefix)
	$\mid t_1 \cdot t_2$	(concatenation)
	$\mid t_1 \wedge t_2$	(intersection)
	$\mid t_1 \vee t_2$	(union)
	$\mid t_1 \mid t_2$	(shuffle)
	$\mid t!$	(prefix closure)

Fig. 3. Syntax of the trace calculus; terms t are allowed to be regular.

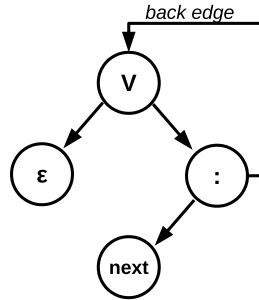


Fig. 4. Abstract syntax graph of the unique term t s.t. $t = \epsilon \vee (next : t)$.

3.1. Basic layer: trace expressions

We start the presentation of the calculus by defining the syntax and semantics of trace expressions, which are used as IR of the basic layer of RML, without support for parametric and generic specifications.

Syntax. The syntax of trace expressions is specified in Fig. 3. We assume that l, x, τ range over countably infinite sets of primitive literals, variables and event type names, respectively.

Event type patterns consist of an event type name τ applied to a possibly empty list of data expressions delimited by parentheses; the calculus supports primitive literals l of type number, string and boolean, object literals $\{k_1:b_1, \dots, k_n:b_n\}$ (where $k_1, \dots, k_n$ denote property keys), and array literals $[b_1, \dots, b_n]$. The meta-variable v (possibly with subscripts/superscripts) denotes data values, that is, ground data expressions with no variables. For simplicity, we abbreviate an event type pattern $\tau()$ with τ , when the list of arguments is empty.

The calculus is deliberately independent from the language used to define event types; correspondingly, the definition of the rewriting system of the calculus is parametric in the relation *match* assigning a semantics to event types (see below). For completeness, Section 4 provides a full formalization of RML event type definitions, together with the corresponding *match* relation, which, in this case, is modeled as a partial function (see the proofs in Appendix A); however, other ways of defining event types with their corresponding semantics could be considered. At the level of the calculus we only require the following minimal assumption on the language for event type definitions: event type definitions must be closed w.r.t. negation; in other words, for any event type τ , its negation $\bar{\tau}$ must be definable. Such an assumption ensures that a certain set of operators can be derived in the calculus (see Section 3.3); furthermore, if $\theta = \tau(b_1, \dots, b_n)$, then $\bar{\theta}$ denotes the event type pattern $\bar{\tau}(b_1, \dots, b_n)$.

The calculus shares some features of RML, with however some differences. The same binary left-associative operators as offered by RML (with the same precedence rules) are defined, but for concatenation the operator symbol $\cdot$ is used; furthermore, the right-associative prefixing operator $:$ has been introduced, with priority higher than all other binary operators.

The main difference with RML consists in the fact that the syntax of the calculus is more abstract: no explicit constructor for recursion is needed, because terms are assumed to be regular (a.k.a. rational or cyclic) [14–17]; that is, the depth of a term is allowed to be infinite, but the number of its different subtrees must be finite. This condition is proved to be equivalent [14] to requiring that a term can always be defined by a *finite* set of possibly recursive syntactic equations. In terms of implementation, the abstract syntax of a regular term can be implemented by a graph⁴ with back edges, while a tree is commonly employed to represent the abstract syntax of a standard term.

For instance, the RML definition $Star = \text{empty} \setminus / (next \ Star)$ can be translated into the trace expression t uniquely determined by the equation $t = \epsilon \vee (next : t)$, whose abstract syntax can be represented by the graph in Fig. 4.

⁴ The internal representation of cyclic terms in SWI-Prolog is indeed based on such approach.

$$\begin{array}{c}
\text{(pre)} \frac{}{\theta : t \xrightarrow{e} t'; \sigma} \quad \sigma = \text{match}(e, \theta) \quad \text{(or-l)} \frac{t_1 \xrightarrow{e} t'_1; \sigma}{t_1 \vee t_2 \xrightarrow{e} t'_1; \sigma} \\
\text{(or-r)} \frac{t_1 \not\xrightarrow{e} \quad t_2 \xrightarrow{e} t'_2; \sigma}{t_1 \vee t_2 \xrightarrow{e} t'_2; \sigma} \quad \text{(and)} \frac{t_1 \xrightarrow{e} t'_1; \sigma_1 \quad t_2 \xrightarrow{e} t'_2; \sigma_2}{t_1 \wedge t_2 \xrightarrow{e} t'_1 \wedge t'_2; \sigma} \quad \sigma = \sigma_1 \cup \sigma_2 \\
\text{(shuffle-l)} \frac{t_1 \xrightarrow{e} t'_1; \sigma}{t_1 | t_2 \xrightarrow{e} t'_1 | t_2; \sigma} \quad \text{(shuffle-r)} \frac{t_1 \not\xrightarrow{e} \quad t_2 \xrightarrow{e} t'_2; \sigma}{t_1 | t_2 \xrightarrow{e} t_1 | t'_2; \sigma} \\
\text{(cat-l)} \frac{t_1 \xrightarrow{e} t'_1; \sigma}{t_1 \cdot t_2 \xrightarrow{e} t'_1 \cdot t_2; \sigma} \quad \text{(cat-r)} \frac{t_1 \not\xrightarrow{e} \quad E(t_1) \quad t_2 \xrightarrow{e} t'_2; \sigma}{t_1 \cdot t_2 \xrightarrow{e} t'_2; \sigma} \\
\text{(l)} \frac{t \xrightarrow{e} t'; \sigma}{t! \xrightarrow{e} t'; \sigma} \quad \text{(e-l)} \frac{}{E(t!)} \\
\text{(e-e)} \frac{}{E(\epsilon)} \quad \text{(e-al)} \frac{E(t_1) \quad E(t_2)}{E(t_1 \text{ op } t_2)} \quad \text{op} \in \{[, \cdot, \wedge\} \\
\text{(e-or-l)} \frac{E(t_1)}{E(t_1 \vee t_2)} \quad \text{(e-or-r)} \frac{E(t_2)}{E(t_1 \vee t_2)}
\end{array}$$

Fig. 5. Semantics of trace expressions, positive judgments.

$$\begin{array}{c}
\text{(n-e)} \frac{}{\epsilon \not\xrightarrow{e}} \quad \text{(n-pre)} \frac{}{\theta : t \not\xrightarrow{e}} \quad \text{match}(e, \theta) \quad \text{undef} \\
\text{(n-or)} \frac{t_1 \not\xrightarrow{e} \quad t_2 \not\xrightarrow{e}}{t_1 \vee t_2 \not\xrightarrow{e}} \quad \text{(n-and-l)} \frac{t_1 \not\xrightarrow{e}}{t_1 \wedge t_2 \not\xrightarrow{e}} \\
\text{(n-and-r)} \frac{t_2 \not\xrightarrow{e}}{t_1 \wedge t_2 \not\xrightarrow{e}} \quad \text{(n-and)} \frac{t_1 \xrightarrow{e} t'_1; \sigma_1 \quad t_2 \xrightarrow{e} t'_2; \sigma_2}{t_1 \wedge t_2 \not\xrightarrow{e}} \quad \sigma_1 \cup \sigma_2 \quad \text{undef} \\
\text{(n-shuffle)} \frac{t_1 \not\xrightarrow{e} \quad t_2 \not\xrightarrow{e}}{t_1 | t_2 \not\xrightarrow{e}} \quad \text{(n-cat-l)} \frac{t_1 \not\xrightarrow{e} \quad NE(t_1)}{t_1 \cdot t_2 \not\xrightarrow{e}} \\
\text{(n-cat-r)} \frac{t_1 \not\xrightarrow{e} \quad t_2 \not\xrightarrow{e}}{t_1 \cdot t_2 \not\xrightarrow{e}} \quad \text{(n-l)} \frac{t \not\xrightarrow{e}}{t! \not\xrightarrow{e}} \quad \text{(ne-pre)} \frac{}{NE(\theta : t)} \\
\text{(n-or)} \frac{NE(t_1) \quad NE(t_2)}{NE(t_1 \vee t_2)} \quad \text{(ne-al)} \frac{\exists i \in \{1, 2\} \quad NE(t_i)}{NE(t_1 \text{ op } t_2)} \quad \text{op} \in \{[, \cdot, \wedge\}
\end{array}$$

Fig. 6. Semantics of trace expressions, negative judgments.

Besides no explicit recursion constructor, the calculus does not offer any feature to deal with definitions to identify terms with names. Finally, there are two minor differences: `empty` is denoted by the constant ϵ , and the two constants 1 and 0 are not part of the basic operators of the calculus, but are introduced only for convenience as a shorthand for the term defined by $t = \epsilon \vee (\tau : t) \vee (\bar{\tau} : t)$, and $\epsilon \wedge (\tau : \epsilon)$, respectively, for any event type⁵ τ ; that is, constants 1 and 0 are the calculus counterparts of the RML terms `all` and `none`, respectively.

A value substitution binds variables occurring in event type patterns with data values, hence, it is a finite domain partial map σ from a finite subset of variables $\text{dom}(\sigma)$ to the set of data values. The terms $\emptyset$, $\sigma|_S$ and $\sigma_{\setminus S}$ denote the empty substitution and the substitutions obtained from σ by restricting its domain to S and by removing S from its domain, respectively; in case S is a singleton set $\{x\}$, we simply write $\sigma|_x$ and $\sigma_{\setminus x}$. We denote with σt the term obtained by applying the substitution σ to all free occurrences of variables in t ; this can be defined by coinduction on t , where the most interesting case is when t is a variable:

$$\sigma x = \sigma(x) \text{ if } x \in \text{dom}(\sigma) \quad \sigma x = x \text{ otherwise}$$

For the remaining cases σt is the homomorphic extension. Substitution σt is fully formalized for the full calculus in the following sections.

Semantics. The semantics of the calculus is expressed by a labeled transition system, where labels range over a given universe of events. The transition relation is inductively defined by the rewriting rules in Fig. 5 and 6 for positive and negative judgments, respectively, together with the auxiliary predicates E and NE needed to deal with the empty trace. Negative judgments are needed to ensure that the semantics of the calculus is deterministic. Judgment $t_1 \xrightarrow{e} t_2; \sigma$ means that event e triggers the transition from term t_1 to t_2 with computed substitution σ , whereas $t \not\xrightarrow{e}$ means that event e does not trigger any transition starting from t (see proof of Lemma A.6).

Predicate E tests whether the empty trace belongs to the semantics of a term t , to properly deal with the rewriting rules of concatenation: if $E(t)$ is derivable, then the set of traces of t contains the empty trace, whereas if $NE(t)$ is derivable, then the set of traces of t does not contain the empty trace (see proof of Lemma A.1).

⁵ We recall that the event type pattern $\tau()$ is abbreviated with τ .

An event trace $\bar{e}$ is a possibly infinite sequence of events ranging over the universe of events; $e\bar{e}$ denotes the event trace starting with event e and continuing with the trace $\bar{e}$.

Definition 3.1. The semantics $\llbracket t \rrbracket$ of a term t is the set of event traces coinductively defined as follows:

- $e \in \llbracket t \rrbracket$ iff $E(t)$ is derivable;
- $e\bar{e} \in \llbracket t \rrbracket$ iff $t \xrightarrow{e} t'; \sigma$ is derivable, and $\bar{e} \in \llbracket \sigma t' \rrbracket$.

We write $t_1 \equiv t_2$ to mean that t_1 and t_2 are semantically equivalent, that is, $\llbracket t_1 \rrbracket = \llbracket t_2 \rrbracket$.

As mentioned at the beginning of this section, rule (pre) is parametric in the relation *match* defining the semantics of event types: we write $\sigma = \text{match}(e, \theta)$ to mean that event e matches event type pattern θ with computed substitution σ ; we say that $\text{match}(e, \theta)$ is undefined iff there is no σ s.t. $\sigma = \text{match}(e, \theta)$. We only require two constraints on *match*:

- consistency with the variables occurring in θ , denoted by $FV(\theta)$: if $\sigma = \text{match}(e, \theta)$, then $\text{dom}(\sigma) \subseteq FV(\theta)$; this implies that for any ground event type pattern θ , if $\sigma = \text{match}(e, \theta)$, then $\sigma = \emptyset$;
- consistency with the negation of event types: if $\text{match}(e, \theta)$ is undefined, then $\text{match}(e, \bar{\theta}) = \emptyset$, and if $\text{match}(e, \theta) = \sigma$, then $\text{match}(e, \bar{\theta})$ is undefined.

In case of RML, *match* is a partial function which formalizes the semantics of event type definitions (see Section 4); for instance, given the definition

```
open(fd) matches {event:'func_post', name:'fs.open', res:fd};
```

if e is the event

```
{event:'func_post', name:'fs.open', args:['f.txt', 'r'], res:42}
```

then $\sigma = \text{match}(e, \text{open}(fd))$ where $\sigma = \{fd \mapsto 42\}$, while $\text{match}(e', \text{open}(fd))$ is undefined, if e' is the following event:

```
{event:'func_pre', name:'fs.open', args:['f.txt', 'r']}.
```

Rules (or-l) and (or-r) define the semantics of $t_1 \vee t_2$; the allowed rewriting steps are those of t_1 plus those of t_2 , but in the latter case only when event e does not trigger any transition starting from t_1 .

Rules (shuffle-l) and (shuffle-r) are similar to those for union, but in this case the subterm which did not contribute is kept after the transition to correctly handle shuffle.

In the side condition of rule (and) for intersection, we write $\sigma = \sigma_1 \cup \sigma_2$ to mean that σ_1 and σ_2 can be successfully merged into σ , that is, $\text{dom}(\sigma) = \text{dom}(\sigma_1) \cup \text{dom}(\sigma_2)$, $\sigma|_{\text{dom}(\sigma_1)} = \sigma_1$ and $\sigma|_{\text{dom}(\sigma_2)} = \sigma_2$. We say that $\sigma_1 \cup \sigma_2$ is undefined if there is no σ s.t. $\sigma = \sigma_1 \cup \sigma_2$.

Rule (cat-l) states that if e triggers a transition starting from t_1 , then the same transition is triggered from $t_1 \cdot t_2$; t_2 is kept after the transition since it can contribute to further steps later on, because of concatenation; this can happen (rule (cat-r)) only under the two conditions that event e does not trigger any transition from t_1 (to ensure determinism), and the semantics of t_1 contains the empty trace (to correctly deal with concatenation).

In rule (!) allowed transitions are exactly the same of the operand, but $t!$ always contains the empty trace.

While all rules for $t \xrightarrow{e} t'$ must be disjoint to guarantee determinism, this is not required for rules defining $E(t)$: rules (e-or-l) and (e-or-r) overlap, but do not prevent determinism (see Theorem A.1).

For what concerns rules for $t \xrightarrow{e} t'$ and $NE(t)$ in Fig. 6, in rule (n-and), the side condition ' $\sigma_1 \cup \sigma_2$ undef' is the negation of the side condition of rule (and), hence, it is satisfied iff there exists no σ s.t. $\text{dom}(\sigma) = \text{dom}(\sigma_1) \cup \text{dom}(\sigma_2)$, $\sigma|_{\text{dom}(\sigma_1)} = \sigma_1$ and $\sigma|_{\text{dom}(\sigma_2)} = \sigma_2$.

For sake of simplicity, rules (n-cat-l) and (n-cat-r) overlap (they are both applicable when $t_1 \xrightarrow{e} t_2 \xrightarrow{e} t'$ and $NE(t_1)$ are derivable); again, this does not prevent determinism as proved in Theorem A.1; a similar comment applies to rule (ne-al) which in some cases can be successfully instantiated for both $i = 1$ and $i = 2$.

We conclude comments on Figs. 5 and 6 by observing that judgments $t \xrightarrow{e} t'$ and $t \xrightarrow{e} t'$ are mutually recursive, $t \xrightarrow{e} t'$ depends on $E(t)$, $t \xrightarrow{e} t'$ depends on $NE(t)$, while $E(t)$ and $NE(t)$ are defined independently.

3.2. Full calculus

We extend the calculus by introducing the more advanced constructs to support the two higher-level layers of RML: parametric and generic specifications.

Syntax. The syntax given in Section 3.1 is extended as defined in Fig. 7. Data expressions include unary and binary operators to compute new data values; the current implementation of RML supports value expressions with only standard numeric and boolean operators (see more details in Section 5).

$d ::=$	$b \mid u \mid d_1 \circ d_2$	(data expression)
$t ::=$	$\dots$	(previous productions as before)
	$\mid \{\text{let } x; t\}$	(parametric expression)
	$\mid g \langle d_1, \dots, d_n \rangle$	(generic application)
	$\mid \text{if } (d) t_1 \text{ else } t_2$	(conditional expression)
$g ::=$	$\langle x_1, \dots, x_n \rangle.t$	(generic expression)

Fig. 7. Syntax of the added constructs; terms t are allowed to be regular.

$$\begin{array}{c}
\text{(par-t)} \frac{t \xrightarrow{e} t'; \sigma}{\{\text{let } x; t\} \xrightarrow{e} \sigma|_x t'; \sigma|_{\setminus x}} \quad x \in \text{dom}(\sigma) \\
\text{(par-f)} \frac{t \xrightarrow{e} t'; \sigma}{\{\text{let } x; t\} \xrightarrow{e} \{\text{let } x; t'\}; \sigma} \quad x \notin \text{dom}(\sigma) \\
\text{(app)} \frac{\sigma t \xrightarrow{e} t'; \sigma'}{(\langle x_1, \dots, x_n \rangle.t) \langle d_1, \dots, d_n \rangle \xrightarrow{e} t'; \sigma'} \quad \sigma = \{x_1 \mapsto \text{ev}(d_1), \dots, x_n \mapsto \text{ev}(d_n)\} \\
\text{(cond-t)} \frac{t_1 \xrightarrow{e} t; \sigma}{\text{if } (d) t_1 \text{ else } t_2 \xrightarrow{e} t; \sigma} \quad \text{ev}(d) = \text{true} \\
\text{(cond-f)} \frac{t_2 \xrightarrow{e} t; \sigma}{\text{if } (d) t_1 \text{ else } t_2 \xrightarrow{e} t; \sigma} \quad \text{ev}(d) = \text{false} \quad \text{(n-par)} \frac{t \not\xrightarrow{e}}{\{\text{let } x; t\} \not\xrightarrow{e}} \\
\text{(n-app)} \frac{\sigma t \not\xrightarrow{e}}{(\langle x_1, \dots, x_n \rangle.t) \langle d_1, \dots, d_n \rangle \not\xrightarrow{e}} \quad \sigma = \{x_1 \mapsto \text{ev}(d_1), \dots, x_n \mapsto \text{ev}(d_n)\} \\
\text{(n-cond-t)} \frac{t_1 \not\xrightarrow{e}}{\text{if } (d) t_1 \text{ else } t_2 \not\xrightarrow{e}} \quad \text{ev}(d) = \text{true} \\
\text{(n-cond-f)} \frac{t_2 \not\xrightarrow{e}}{\text{if } (d) t_1 \text{ else } t_2 \not\xrightarrow{e}} \quad \text{ev}(d) = \text{false} \quad \text{(e-par)} \frac{E(t)}{E(\{\text{let } x; t\})} \\
\text{(e-app)} \frac{E(\sigma t)}{E((\langle x_1, \dots, x_n \rangle.t) \langle d_1, \dots, d_n \rangle)} \quad \sigma = \{x_1 \mapsto \text{ev}(d_1), \dots, x_n \mapsto \text{ev}(d_n)\} \\
\text{(e-cond-t)} \frac{E(t_1)}{E(\text{if } (d) t_1 \text{ else } t_2)} \quad \text{ev}(d) = \text{true} \\
\text{(e-cond-f)} \frac{E(t_2)}{E(\text{if } (d) t_1 \text{ else } t_2)} \quad \text{ev}(d) = \text{false} \quad \text{(ne-par)} \frac{NE(t)}{NE(\{\text{let } x; t\})} \\
\text{(ne-app)} \frac{NE(\sigma t)}{NE((\langle x_1, \dots, x_n \rangle.t) \langle d_1, \dots, d_n \rangle)} \quad \sigma = \{x_1 \mapsto \text{ev}(d_1), \dots, x_n \mapsto \text{ev}(d_n)\} \\
\text{(ne-cond-t)} \frac{NE(t_1)}{NE(\text{if } (d) t_1 \text{ else } t_2)} \quad \text{ev}(d) = \text{true} \\
\text{(ne-cond-f)} \frac{NE(t_2)}{NE(\text{if } (d) t_1 \text{ else } t_2)} \quad \text{ev}(d) = \text{false}
\end{array}$$

Fig. 8. Semantics of the added constructs: positive and negative judgments.

The syntax for parametric and generic expressions is reminiscent of the corresponding RML constructs, although simpler; as in RML, generics are not first-class.

Semantics. All positive and negative judgments defining the semantics of the added constructs are specified in Fig. 8. In the transition of a parametric expression $\{\text{let } x; t\}$, variable x is substituted by applying $\sigma|_x$ (the substitution σ restricted to x) to t' , if the substitution σ computed after the transition from t contains x in its domain (rule (param-t)); consequently, the variable x is removed from the domain of σ ($\sigma|_{\setminus x}$); otherwise, the declaration of x is kept (rule (param-f)).

Application of substitution to parametric and generic expressions works as expected:

$$\sigma \{\text{let } x; t\} = \{\text{let } x; \sigma|_x t\} \quad \sigma (\langle x_1, \dots, x_n \rangle.t) = \langle x_1, \dots, x_n \rangle.\sigma|_{\setminus \{x_1, \dots, x_n\}} t$$

The full definition of substitution application to calculus terms can be found in Fig. 9. The definition is coinductive since the terms are regular, hence, not well-founded. It is immediate to show that for any event type pattern θ and substitution σ , $\sigma \overline{\sigma \theta} = \sigma \theta$, since $\sigma \tau(b_1, \dots, b_n) = \tau(\sigma b_1, \dots, \sigma b_n) = \overline{\tau}(\sigma b_1, \dots, \sigma b_n) = \sigma \overline{\tau}(b_1, \dots, b_n) = \sigma \tau(b_1, \dots, b_n)$.

When a generic specification is applied, arguments are evaluated with the auxiliary function ev , and the event has to trigger a transition starting from the term obtained from the body of the generic specification by substituting all parameters

$$\begin{aligned}
\sigma x &= \sigma(x) \text{ if } x \in \text{dom}(\sigma) \\
\sigma x &= x \text{ if } x \notin \text{dom}(\sigma) \\
\sigma l &= l \\
\sigma \{k_1:b_1, \dots, k_n:b_n\} &= \{k_1:\sigma b_1, \dots, k_n:\sigma b_n\} \\
\sigma [b_1, \dots, b_n] &= [\sigma b_1, \dots, \sigma b_n] \\
\sigma (\tau(b_1, \dots, b_n)) &= \tau(\sigma b_1, \dots, \sigma b_n) \\
\sigma \epsilon &= \epsilon \\
\sigma (\theta : t) &= \sigma \theta : \sigma t \\
\sigma (t_1 \text{ op } t_2) &= \sigma t_1 \text{ op } \sigma t_2 \text{ for } \text{op} \in \{\cdot, \wedge, \vee, |\} \\
\sigma (t!) &= (\sigma t)! \\
\sigma (u \text{ d}) &= u \text{ d} \\
\sigma (d_1 \text{ o } d_2) &= \sigma d_1 \text{ o } \sigma d_2 \\
\sigma \{\text{let } x; t\} &= \{\text{let } x; \sigma_x t\} \\
\sigma (g\langle d_1, \dots, d_n \rangle) &= \sigma g\langle \sigma d_1, \dots, \sigma d_n \rangle \\
\sigma (\text{if } (d) \text{ } t_1 \text{ else } t_2) &= \text{if } (\sigma d) \text{ } \sigma t_1 \text{ else } \sigma t_2 \\
\sigma (\langle x_1, \dots, x_n \rangle . t) &= \langle x_1, \dots, x_n \rangle . \sigma_{\setminus \{x_1, \dots, x_n\}} t
\end{aligned}$$

Fig. 9. Coinductive definition of application of substitution to terms.

$$\begin{array}{ccc}
(\text{val}) \frac{}{ev(v) = v} & (\text{u-op}) \frac{ev(d) = v}{ev(u \text{ d}) = u(v)} & (\text{b-op}) \frac{ev(d_1) = v_1 \quad ev(d_2) = v_2}{ev(d_1 \text{ o } d_2) = v_1 \text{ o } v_2}
\end{array}$$

Fig. 10. Evaluation of data expressions.

with the corresponding values of the arguments. The definition of function ev to evaluate data expressions is straightforward and can be found in Fig. 10.

3.3. Derived operators

In this section we extend the calculus with several useful operators which are also available in RML, as shown in Section 2.3.4; although they are all derivable, such operators are directly implemented in the interpreter of the calculus for efficiency reasons.

Standard postfix operators. The expressions $t?$, $t+$, and $t*$ are equivalent to $\epsilon \vee t$, $t \cdot t_*$ and t_* , respectively, where t_* is the trace expression uniquely determined by the equation $t_* = \epsilon \vee (t \cdot t_*)$.

The semantics of postfix operators can be provided directly with the following rules:

$$\begin{array}{ccc}
(?) \frac{t \xrightarrow{e} t'; \sigma}{t? \xrightarrow{e} t'; \sigma} & (+) \frac{t \xrightarrow{e} t'; \sigma}{t+ \xrightarrow{e} t' \cdot t_*; \sigma} & (*) \frac{t \xrightarrow{e} t'; \sigma}{t* \xrightarrow{e} t' \cdot t_*; \sigma} \\
(n-?) \frac{t \not\xrightarrow{e}}{t? \not\xrightarrow{e}} & (n-+) \frac{t \not\xrightarrow{e}}{t+ \not\xrightarrow{e}} & (n-*) \frac{t \not\xrightarrow{e}}{t* \not\xrightarrow{e}} \\
(e-?) \frac{}{E(t?)} & (e-+) \frac{E(t)}{E(t+)} & (e-*) \frac{}{E(t*)} \quad (ne-+) \frac{NE(t)}{NE(t+)}
\end{array}$$

For all operators, the transitions allowed are the same as for t . Both $t?$ and $t*$ always contain the empty trace, while for $t+$ this is possible only if the same holds for t . Both $t+$ and $t*$ concatenate the trace expression t' resulting from the transition of the original one with t_* , in order to allow more iterations of the trace expression.

The standard postfix operators are derivable as shown by the following laws formally proved in Appendix A:

$$t? \equiv \epsilon \vee t \quad t+ \equiv t \cdot t_* \quad t* \equiv t_*$$

where $t_* = \epsilon \vee (t \cdot t_*)$.

Conditional filter. This operator is a generalization of the $\gg$ operator, as mentioned in Section 2.3.4: $\theta \gg t_1 : t_2$ specifies all traces which belong to t_1 when only events matching θ are kept, and they belong to t_2 when all events matching θ are filtered out.

Conditional filter is derivable as shown by the following law formally proved in Appendix A:

$$\theta \gg t_1 : t_2 \equiv ((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2)$$

where $\bar{\theta}$ denotes the negation of the event type pattern θ ; in this case traces of disjoint events are shuffled, since the operands of the shuffle operator specify traces of events that must match θ , and $\bar{\theta}$, respectively.

As already shown in Section 2.3.4, unconditional filter $\theta \gg t$ is an abbreviation for $\theta \gg t : 1$, that is, $((\theta : \epsilon) * \wedge t) \mid ((\bar{\theta} : \epsilon) * \wedge 1)$, which can be proved equivalent to $((\theta : \epsilon) * \wedge t) \mid (\bar{\theta} : \epsilon) *$.

$$\begin{array}{c}
\text{(filter-t)} \frac{t_1 \xrightarrow{e} t'_1; \sigma'}{\theta \gg t_1 : t_2 \xrightarrow{e} \theta \gg t'_1 : t_2; \sigma''} \sigma = \text{match}(e, \theta), \sigma'' = \sigma \cup \sigma' \\
\text{(filter-f)} \frac{t_2 \xrightarrow{e} t'_2; \sigma}{\theta \gg t_1 : t_2 \xrightarrow{e} \theta \gg t_1 : t'_2; \sigma} \text{match}(e, \theta) \text{ undef} \\
\text{(n-filter-t)} \frac{t_1 \not\xrightarrow{e}}{\theta \gg t_1 : t_2 \not\xrightarrow{e}} \exists \sigma \sigma = \text{match}(e, \theta) \\
\text{(n-filter-t2)} \frac{t_1 \xrightarrow{e} t'_1; \sigma'}{\theta \gg t_1 : t_2 \not\xrightarrow{e}} \sigma = \text{match}(e, \theta), \sigma \cup \sigma' \text{ undef} \\
\text{(n-filter-f)} \frac{t_2 \not\xrightarrow{e}}{\theta \gg t_1 : t_2 \not\xrightarrow{e}} \text{match}(e, \theta) \text{ undef} \\
\text{(e-filter)} \frac{E(t_1) \quad E(t_2)}{E(\theta \gg t_1 : t_2)} \quad \text{(ne-filter)} \frac{\exists i \in \{1, 2\} NE(t_i)}{NE(\theta \gg t_1 : t_2)}
\end{array}$$

Fig. 11. Semantics of filter.

The semantics of conditional filter is directly defined by the rules in Fig. 11. When the observed event matches θ , it also has to be a valid prefix for t_1 . Substitutions produced by these two steps need to be successfully merged, because event e must match both θ and an event type pattern in t_1 . If the event does not match θ then event e is expected to trigger a transition starting from t_2 . Finally, $\theta \gg t_1 : t_2$ contains the empty trace only if both trace expressions t_1 and t_2 contain it.

3.4. Laws

This section presents several basic properties satisfied by the operators of the calculus; most of them are exploited by the implementation for optimization purposes (see the examples provided in Section 6); all formal proofs can be found in Appendix A.

Identity of the shuffle. The singleton set ϵ containing the empty trace is the identity of the shuffle:

$$\epsilon \mid t \equiv t \equiv t \mid \epsilon.$$

Identity of the intersection. The set of all traces 1 is the identity of the intersection:

$$1 \wedge t \equiv t \equiv t \wedge 1.$$

Identity of the concatenation. The singleton set ϵ containing the empty trace is the identity of the concatenation:

$$\epsilon \cdot t \equiv t \equiv t \cdot \epsilon.$$

Invariance of 1 w.r.t. filter. The following law holds for all event type patterns θ :

$$\theta \gg 1 \equiv 1.$$

Relationship between the $$ and $+$ operators.* The usual law connecting the $*$ and $+$ operators holds:

$$t + \equiv t \cdot t *.$$

Except for the last one, the other laws are relevant for optimization purposes, as shown by the experimental results in Section 8; the last rule is used to easily deduce that $+$ is a derived operator from the claim that $*$ is derived as well.

As a final remark, although the deterministic semantics of the calculus allows generation of more efficient monitors, there are properties that would hold for a non-deterministic version which are no longer valid; for instance, in the trace calculus union and shuffle are not commutative.

4. Semantics of event types in RML

This section presents the formalization of the language used in RML for event type definitions, thus providing an instantiation of the *match* relation on which the semantics of the trace calculus is based.

$\bar{\Theta} ::= \Theta_1; \dots; \Theta_n;$	(event type definitions)
$\Theta ::= \tau(x_1, \dots, x_n) \text{ match } op$	(positive basic clause)
$\quad \tau(x_1, \dots, x_n) \neg \text{match } op$	(negative basic clause)
$\quad \tau(x_1, \dots, x_n) \text{ match } etp_1 \dots etp_n$	(positive derived clause)
$\quad \tau(x_1, \dots, x_n) \neg \text{match } etp_1 \dots etp_n$	(negative derived clause)
$vp ::= x l op ap w$	(value pattern)
$op ::= \{k_1:vp_1, \dots, k_n:vp_n\}$	(object pattern)
$ap ::= [vp_1, \dots, vp_n] [vp_1, \dots, vp_n, el]$	(array pattern)
$etp ::= \tau(vp_1, \dots, vp_n)$	(extended event type pattern)

Fig. 12. Syntax of event type definitions.

Syntax. Fig. 12 presents the syntax of the language for event type definitions; they consist of sequences of clauses where an event type can be defined by more clauses.

The head $\tau(x_1, \dots, x_n)$ of an event type clause consists of the name τ of the event type and its parameters $x_1, \dots, x_n$ which are assumed to be distinct, and are the only variables that can occur in the body of the clause.

A clause can be either *positive* or *negative*; correspondingly, the body of the clause is introduced by the keywords *match* and $\neg \text{match}$. In terms of concrete syntax,

```
open(fd) matches {event:'func_post', name:'fs.open', res:fd};
no_open(fd) not matches {event:'func_post', name:'fs.open', res:fd};
```

are positive and negative clauses, respectively.

Event patterns constitute the bodies of clauses, and specify the events that have to be matched by the defined event type; they can be either *object patterns*, in *basic* clauses, or disjunctions of *extended event type patterns* in *derived* clauses. In both cases, patterns can of course contain variables, possibly with repeated occurrences of the same variable.

The structure of object patterns in the body of basic clauses closely resembles the shape of events, which are assumed to be represented by objects. An object pattern $\{k_1:vp_1, \dots, k_n:vp_n\}$ specifies all object values that contain at least property keys $k_1, \dots, k_n$ (hence, object patterns are open) associated with data values $v_1, \dots, v_n$, respectively matching value patterns $vp_1, \dots, vp_n$; property keys $k_1, \dots, k_n$ are assumed to be distinct, while their order is immaterial.

A derived clause is expressed in terms of other event types; recursion in derived clauses is not allowed since it would not bring any benefit. The body of a derived clause consists of the disjunction $etp_1 | \dots | etp_n$ of extended event type patterns, that is, event type patterns as defined in Fig. 3, with the difference⁶ that they are also allowed to contain *wildcard* and *ellipsis* occurrences (see below).

Value patterns are very similar to basic data expressions (and hence, to data values, see Fig. 3), but they can also contain the wildcard pattern (abstract syntax w , corresponding to the more concrete one $'_'$) and the ellipsis (abstract syntax el , corresponding to the more concrete one $'\dots'$) at the end of array patterns (denoted by el): the former is used to match any value without binding a variable, while the latter allows matching of arrays with variable length. For instance, the following examples introduced in Section 2, exploit wildcard and ellipsis:

```
sensor_values(time) matches {temperature:_, humidity:_, timestamp:time};
head_req(reqId) matches {event:'func_post', name:'http.request', args:[{method:'HEAD'}, ...], res:reqId};
```

For sake of simplicity, the head of a clause must always contain the parentheses even when the defined event type has no parameters; furthermore the formalization of value patterns we present here does not include union of value patterns $vp_1 | \dots | vp_n$, a convenient feature offered by RML which allows a more compact syntax in certain situations.

For instance, the following RML definition (already shown in Section 2)

```
relevant matches
{event:'func_post', name:'fs.open'} |
{event:'func_pre', name:'fs.write' | 'fs.close'};
```

is not supported by the syntax given in Fig. 12; however, it can be translated (see Section 7.2 for more details on the RML compiler) in the following equivalent abstract syntax:

```
relevant() matches {event:'func_post', name:'fs.open'}
relevant() matches {event:'func_pre', name:'fs.write'}
relevant() matches {event:'func_pre', name:'fs.close'}
```

Pattern substitution. In the rest of the section we will use a notion of pattern substitution ς which is similar to that adopted in Section 3, with the difference that variables are mapped to value patterns, rather than to data values; pattern substitutions are needed to manage parameters of event types in derived clauses.

⁶ Technically speaking, there are no obstacles in allowing extended event type patterns also in trace specifications, both in the trace calculus and in RML; however, we have preferred to simplify the syntax of event type patterns in trace specifications to favor readability.

$$\begin{aligned}
\zeta x &= \zeta(x) \quad (\text{if } x \in \text{dom}(\zeta)) \\
\zeta x &= x \quad (\text{if } x \notin \text{dom}(\zeta)) \\
\zeta l &= l \\
\zeta w &= w \\
\zeta \{k_1:vp_1, \dots, k_n:vp_n\} &= \{k_1:\zeta vp_1, \dots, k_n:\zeta vp_n\} \\
\zeta [vp_1, \dots, vp_n] &= [\zeta vp_1, \dots, \zeta vp_n] \\
\zeta [vp_1, \dots, vp_n, el] &= [\zeta vp_1, \dots, \zeta vp_n, el] \\
\zeta (\tau(vp_1, \dots, vp_n)) &= \tau(\zeta vp_1, \dots, \zeta vp_n)
\end{aligned}$$

Fig. 13. Inductive definition of pattern substitution.

$$\begin{aligned}
& \text{(var)} \frac{}{v:x \rightsquigarrow \{x \mapsto v\}} \quad \text{(wild)} \frac{}{v:w \rightsquigarrow \emptyset} \quad \text{(val)} \frac{}{v:v \rightsquigarrow \emptyset} \quad \text{(valf)} \frac{}{v:v' \rightsquigarrow \text{fail}} \quad v \neq v' \\
& \text{(obj)} \frac{\forall i \in \{1, \dots, n\} \quad v_i:vp_i \rightsquigarrow \sigma_i}{\{k_1:v_1, \dots, k_{n+q}:v_{n+q}\}:\{k_1:vp_1, \dots, k_n:vp_n\} \rightsquigarrow \sigma} \quad \sigma = \sigma_1 \cup \dots \cup \sigma_n \\
& \text{(objf1)} \frac{\forall i \in \{1, \dots, n\} \quad v_i:vp_i \rightsquigarrow \sigma_i}{\{k_1:v_1, \dots, k_{n+q}:v_{n+q}\}:\{k_1:vp_1, \dots, k_n:vp_n\} \rightsquigarrow \text{fail}} \quad \sigma_1 \cup \dots \cup \sigma_n \text{ undef} \\
& \text{(objf2)} \frac{\exists i \in \{1, \dots, n\} \quad v_i:vp_i \rightsquigarrow \text{fail}}{\{k_1:v_1, \dots, k_{n+q}:v_{n+q}\}:\{k_1:vp_1, \dots, k_n:vp_n\} \rightsquigarrow \text{fail}} \\
& \text{(objf3)} \frac{}{v:\{k_1:vp_1, \dots, k_n:vp_n\} \rightsquigarrow \text{fail}} \quad v \neq \{k_1:v_1, \dots, k_{n+q}:v_{n+q}\} \\
& \text{(arr)} \frac{\forall i \in \{1, \dots, n\} \quad v_i:vp_i \rightsquigarrow \sigma_i}{[v_1, \dots, v_n]:[vp_1, \dots, vp_n] \rightsquigarrow \sigma} \quad \sigma = \sigma_1 \cup \dots \cup \sigma_n \\
& \text{(arre)} \frac{[v_1, \dots, v_n]:[vp_1, \dots, vp_n] \rightsquigarrow \sigma^\perp}{[v_1, \dots, v_{n+q}]:[vp_1, \dots, vp_n, el] \rightsquigarrow \sigma^\perp} \\
& \text{(arrf1)} \frac{\forall i \in \{1, \dots, n\} \quad v_i:vp_i \rightsquigarrow \sigma_i}{[v_1, \dots, v_n]:[vp_1, \dots, vp_n] \rightsquigarrow \text{fail}} \quad \sigma_1 \cup \dots \cup \sigma_n \text{ undef} \\
& \text{(arrf2)} \frac{\exists i \in \{1, \dots, n\} \quad v_i:vp_i \rightsquigarrow \text{fail}}{[v_1, \dots, v_n]:[vp_1, \dots, vp_n] \rightsquigarrow \text{fail}} \\
& \text{(arrf3)} \frac{}{v:[vp_1, \dots, vp_n] \rightsquigarrow \text{fail}} \quad v \neq [v_1, \dots, v_n] \\
& \text{(arref)} \frac{}{v:[vp_1, \dots, vp_q, el] \rightsquigarrow \text{fail}} \quad (v = [v_1, \dots, v_n] \text{ and } n < q), \text{ or } v \neq [v_1, \dots, v_n]
\end{aligned}$$

Fig. 14. Inductive definition of value pattern matching.

Application of a pattern substitution ζ to a value pattern vp and an extended event type pattern etp is inductively defined in Fig. 13.

In the rest of the section we use the notation $\{x_1 \mapsto vp_1, \dots, x_n \mapsto vp_n\}$ to denote the pattern substitution ζ s.t. $\text{dom}(\zeta) = \{x_1, \dots, x_n\}$ and $\zeta(x_1) = vp_1, \dots, \zeta(x_n) = vp_n$; an analogous notation $\{x_1 \mapsto v_1, \dots, x_n \mapsto v_n\}$ is used for value substitutions.

Value pattern matching. Matching of data values v (that is, the observed events) against value patterns vp is formalized by the judgment $v:vp \rightsquigarrow \sigma^\perp$ (see Fig. 14): $\sigma^\perp$ denotes either the computed value substitution σ , in case matching succeeds, or *fail*, in case of failure; this latter option needs to be made explicit to support negative event type definitions.

Similarly as rule (and) in Fig. 5, the side condition $\sigma = \sigma_1 \cup \dots \cup \sigma_n$ in rules (obj) and (arr) means that $\sigma_1, \dots, \sigma_n$ can be successfully merged into σ , that is, $\text{dom}(\sigma) = \text{dom}(\sigma_1) \cup \dots \cup \text{dom}(\sigma_n)$, and $\sigma|_{\text{dom}(\sigma_i)} = \sigma_i$ for all $i \in \{1, \dots, n\}$; consequently, in rules (objf1) and (arrf1), the side condition ' $\sigma_1 \cup \dots \cup \sigma_n \text{ undef}$ ' is the negation of the previous one, hence, it is satisfied iff there exists no σ s.t. $\text{dom}(\sigma) = \text{dom}(\sigma_1) \cup \dots \cup \text{dom}(\sigma_n)$ and $\sigma|_{\text{dom}(\sigma_i)} = \sigma_i$ for all $i \in \{1, \dots, n\}$.

An object value matches an object pattern (rule (obj)), as long as it contains at least all property keys of the pattern (that is $n, q \geq 0$) with values matching the correspondingly associated value patterns, and the corresponding computed substitutions can be safely merged together. Matching with object pattern can fail for the following reasons: (a) the substitutions computed for the values and patterns associated with each property key cannot be successfully merged together (rule (objf1)); for instance, value $\{args:[1], res:2\}$ and pattern $\{args:[x], res:x\}$ do not match because $[1]$ and $[x]$ match with substitution $\{x \mapsto 1\}$, while 2 and x match with substitution $\{x \mapsto 2\}$; (b) there is a value which does not match the corresponding value pattern for a required property key (rule (objf2)); for instance, value $\{name:'fs.open', res:1\}$ and pattern $\{name:'fs.close', res:x\}$ do not match because '*fs.open*' and '*fs.close*' do not match; (c) the data value is an object that does not have all required properties, or is not an object value at all (rule (objf3)); for instance, values $\{args:[1]\}$

$$\begin{array}{c}
\text{(mtch-main)} \frac{mdec_{\bar{\Theta}}(e, \bar{\Theta}, etp) = \sigma^\perp}{match_{\bar{\Theta}}(e, etp) = \sigma^\perp} \quad \text{(mtch-1)} \frac{mdec_{\bar{\Theta}}(e, \Theta, etp) = \sigma}{mdec_{\bar{\Theta}}(e, (\Theta; \dots), etp) = \sigma} \\
\text{(mtch-2)} \frac{mdec_{\bar{\Theta}}(e, \Theta, etp) = fail \quad mdec_{\bar{\Theta}}(e, \bar{\Theta}', etp) = \sigma^\perp}{mdec_{\bar{\Theta}}(e, (\Theta; \bar{\Theta}'), etp) = \sigma^\perp} \\
\text{(mtch-fail)} \frac{}{mdec_{\bar{\Theta}}(e, \epsilon, etp) = fail} \\
\text{(mtch-op)} \frac{e: \zeta op \rightsquigarrow \sigma^\perp}{mdec_{\bar{\Theta}}(e, \Theta, \tau(vp_1, \dots, vp_n)) = \sigma^\perp} \quad \begin{array}{l} \Theta = \tau(x_1, \dots, x_n) \text{ match } op \\ \zeta = \{x_1 \mapsto vp_1, \dots, x_n \mapsto vp_n\} \end{array} \\
\text{(mtch-nop)} \frac{e: \zeta op \rightsquigarrow fail}{mdec_{\bar{\Theta}}(e, \Theta, \tau(vp_1, \dots, vp_n)) = \emptyset} \quad \begin{array}{l} \Theta = \tau(x_1, \dots, x_n) \neg\text{match } op \\ \zeta = \{x_1 \mapsto vp_1, \dots, x_n \mapsto vp_n\} \end{array} \\
\text{(mtch-der)} \frac{\forall j \in \{1, \dots, i-1\} \text{ match}_{\bar{\Theta}}(e, \zeta etp_j) = fail \quad match_{\bar{\Theta}}(e, \zeta etp_i) = \sigma}{mdec_{\bar{\Theta}}(e, \Theta, \tau(vp_1, \dots, vp_n)) = \sigma} \quad \begin{array}{l} \Theta = \tau(x_1, \dots, x_n) \text{ match } etp_1 | \dots | etp_k \\ \zeta = \{x_1 \mapsto vp_1, \dots, x_n \mapsto vp_n\} \\ 1 \leq i \leq k \end{array} \\
\text{(mtch-nder)} \frac{\forall i \in \{1, \dots, k\} \text{ match}_{\bar{\Theta}}(e, \zeta etp_i) = fail}{mdec_{\bar{\Theta}}(e, \Theta, \tau(vp_1, \dots, vp_n)) = \emptyset} \quad \begin{array}{l} \Theta = \tau(x_1, \dots, x_n) \neg\text{match } etp_1 | \dots | etp_k \\ \zeta = \{x_1 \mapsto vp_1, \dots, x_n \mapsto vp_n\} \end{array} \\
\text{(mtch-f1)} \frac{}{mdec_{\bar{\Theta}}(e, \Theta, \tau(vp_1, \dots, vp_n)) = fail} \quad \Theta \neq \tau(x_1, \dots, x_n) \dots \\
\text{(mtch-f2)} \frac{e: \zeta op \rightsquigarrow \sigma}{mdec_{\bar{\Theta}}(e, \Theta, \tau(vp_1, \dots, vp_n)) = fail} \quad \begin{array}{l} \Theta = \tau(x_1, \dots, x_n) \neg\text{match } op \\ \zeta = \{x_1 \mapsto vp_1, \dots, x_n \mapsto vp_n\} \end{array} \\
\text{(mtch-f3)} \frac{\forall i \in \{1, \dots, k\} \text{ match}_{\bar{\Theta}}(e, \zeta etp_i) = fail}{mdec_{\bar{\Theta}}(e, \Theta, \tau(vp_1, \dots, vp_n)) = fail} \quad \begin{array}{l} \Theta = \tau(x_1, \dots, x_n) \text{ match } etp_1 | \dots | etp_k \\ \zeta = \{x_1 \mapsto vp_1, \dots, x_n \mapsto vp_n\} \end{array} \\
\text{(mtch-f4)} \frac{\exists i \in \{1, \dots, k\} \text{ match}_{\bar{\Theta}}(e, \zeta etp_i) = \sigma}{mdec_{\bar{\Theta}}(e, \Theta, \tau(vp_1, \dots, vp_n)) = fail} \quad \begin{array}{l} \Theta = \tau(x_1, \dots, x_n) \neg\text{match } etp_1 | \dots | etp_k \\ \zeta = \{x_1 \mapsto vp_1, \dots, x_n \mapsto vp_n\} \end{array}
\end{array}$$

Fig. 15. Inductive definition of *match* for extended event type pattern matching.

and $[1, 2, 3]$ and pattern $\{args:[x], res: y\}$ do not match because $\{args:[1]\}$ lacks property *res* and $[1, 2, 3]$ is not an object, respectively.

Rule (arr) is similar to rule (obj): an array value matches an array pattern without ellipsis, as long as the lengths of the array and of the array pattern are the same, all values of the array match the corresponding value patterns, and the corresponding computed substitutions can be safely merged together.

Rule (arre) relaxes this requirement for array patterns with ellipsis, allowing the array data value to have more elements than needed (that is $n, q \geq 0$); the rule can be partly reduced to rule (arr) by removing from the array value all additional elements, however rule (arref) is still needed to manage some failures that arise only for array patterns with ellipsis.

Failure on array patterns without ellipsis are managed by rules (arrf1), (arrf2) and (arrf3), which are the analogous of rules (objf1), (objf2) and (objf3) for object patterns. The last rule (arref) deals only with array patterns with ellipsis: matching fails if the value is an array with less elements than those required by the pattern, or is not an array at all. For instance, value $[1, 2]$ does not match pattern $[x]$ because it has two elements instead of one, while it matches $[x, \dots]$, since the pattern allows array values to contain more elements; finally, $[\]$ and $\{x:42\}$ do not match $[x, \dots]$, because $[\]$ has no elements, while pattern $[x, \dots]$ requires arrays to have at least one element, and $\{x:42\}$ is not an array, respectively.

Extended event type pattern matching. Matching of an event e against an extended event type pattern etp is defined in Fig. 15 by the following three different judgments; as happens for value pattern matching, $\sigma^\perp$ denotes either the computed value substitution σ , in case matching succeeds, or *fail*, in case of failure.

- $match_{\bar{\Theta}}(e, etp) = \sigma^\perp$ is the main judgment, meaning that either the event e matches the extended event type pattern etp w.r.t. the event type definitions $\bar{\Theta}$ with computed substitution σ , when $\sigma^\perp = \sigma$, or e does not match etp w.r.t. $\bar{\Theta}$ when $\sigma^\perp = fail$. The subscript $\bar{\Theta}$ is dropped when implicit, as happens, for instance, in rule (pre) in Fig. 5;
- $mdec_{\bar{\Theta}}(e, \bar{\Theta}', etp) = \sigma^\perp$ means that the event type definitions $\bar{\Theta}'$ contribute with $\sigma^\perp$ to the matching of e against etp w.r.t. the event type definitions $\bar{\Theta}$. This auxiliary judgment is needed to keep track of the type definitions considered so far, to ensure a deterministic semantics of matching; indeed, definitions are inspected in left-to-right order and matching succeeds only for the first successful definition, therefore no backtracking is performed and a unique computed substitution is returned in case of success. The subscript $\bar{\Theta}$ is needed to correctly manage derived clauses (see rules (mtch-der), (mtch-nder), (mtch-f3) and (mtch-f4));

- $mdec_{\bar{\Theta}}(e, \Theta, etp) = \sigma^\perp$ means that the event type definition Θ contributes with $\sigma^\perp$ to the matching of e against etp w.r.t. the event type definitions $\bar{\Theta}$.

Rule (mtch-main) checks whether there exists a definition among $\bar{\Theta}$ that successfully contributes to the matching; rules (mtch-1), (mtch-2) and (mtch-fail) allow left-to-right inspection of type definitions: if the left-most definition succeeds, then the corresponding computed substitution is returned (rule (mtch-1)), otherwise the remaining definitions (if any) are considered (rule (mtch-2)); finally, if there are no other definitions to inspect (that is, $\bar{\Theta} = \epsilon$), then the matching fails (rule (mtch-fail)).

For instance, if we assume that the event type `open` of arity 1 is defined by the following two clauses

```
open(r) matches {event:'func_post', name:'fs.open', res:r};
open(x) matches {name:'fs.open', args:[x]};
```

then event $\{event: 'func_post', name: 'fs.open', args: ['foo'], res: 42\}$ and the pattern `open(val)` match with substitution $\{val \mapsto 42\}$, thanks to rules (mtch-main) and (mtch-1), among others, while matching with substitution $\{val \mapsto 'foo'\}$ is not derivable, because rule (mtch-2) cannot be applied, since the first clause above contributes successfully to matching and, hence, the first premise of (mtch-2) is not satisfied.

On the contrary, $\{event: 'func_pre', name: 'fs.open', args: ['foo']\}$ matches with substitution $\{val \mapsto 'foo'\}$, thanks to rules (mtch-main) and (mtch-2), among others, because matching fails with the first clause above and is successful with the second one.

Finally, matching between $\{name: 'fs.close'\}$ and `open(val)` fails, thanks to rules (mtch-main), (mtch-2) and (mtch-fail), among others, because matching fails with both clauses above.

Rule (mtch-op) handles positive basic clauses: the extended event type pattern $\tau(vp_1, \dots, vp_n)$ must match the name and the number of parameters of the head $\tau(x_1, \dots, x_n)$; if so, a corresponding pattern substitution ζ is computed and applied to the body of the clause (ζop), and, finally, matching of the event e (which is an object value) against the value pattern ζop is checked.

Rule (mtch-nop) is the dual of (mtch-op), handling negative basic clauses: matching succeeds iff the event e does not match against the value pattern ζop ; in case of success, the empty substitution is returned, analogously to what happens in logic programming: indeed, while positive clauses are existentially quantified definitions and the computed substitutions specify the witnesses for such quantifications, negative clauses are universally quantified definitions.

Rule (mtch-der) manages positive derived clauses: the first extended event type pattern in the body of the clause matching the event is deterministically searched in left-to-right order and the matching fails if no extended event type pattern succeeds; rule (mtch-nder) is the dual of (mtch-der), as happens for (mtch-op) and (mtch-nop): matching succeeds iff there are no matching extended event type patterns in the body of the clause and the empty substitution is returned in case of success.

All the remaining rules deal with failures: a positive basic clause fails if the extended event type pattern $\tau(vp_1, \dots, vp_n)$ does not match the name or the number of parameters of the head of the clause (side condition $\Theta \neq \tau(x_1, \dots, x_n) \dots$ of rule (mtch-f1)); for instance, if the only defined clauses are those introduced above for the event type `open` of arity 1, then neither of them can be successfully used to match an event with the patterns `open('foo', 42)` and `close(42)`: in the former case the number of arguments (2) in the pattern does not match the number of parameters (1) in the clauses, in the latter the name `close` do not match the name `open` of the event type defined by the two clauses.

A negative basic clause fails if the event matches the instantiated body ζop (rule (mtch-f2)); for instance, if the event type `not_open` of arity 1 is only defined by the following clause:

```
not_open(x) not matches {name:'fs.open', args:[x]};
```

then $\{name: 'fs.open', args: ['foo']\}$ matches neither `not_open('foo')`, nor `not_open(y)`: in the former case the substitution $\zeta = \{x \mapsto 'foo'\}$ is applied to $\{name: 'fs.open', args: [x]\}$ to get $\{name: 'fs.open', args: ['foo']\}$ which successfully matches the event with the empty computed substitution σ ; in the latter case, $\zeta' = \{x \mapsto y\}$ is applied to $\{name: 'fs.open', args: [x]\}$ to get $\{name: 'fs.open', args: [y]\}$ which successfully matches the event with the computed substitution $\sigma' = \{y \mapsto 'foo'\}$.

A positive derived clause fails if all extended event type patterns of its body fail after they have been instantiated with the substitution ζ (rule (mtch-f3)); for instance, given the following clauses

```
open matches {event:'func_post', name:'fs.open'}
close matches {event:'func_pre', name:'fs.close'};
relevant matches open | close;
```

then event $\{event: 'send'\}$ and `relevant()` do not match because the event matches neither `open()` nor `close()`.

Finally, a negative derived clause fails if any of the extended event type patterns of its body matches after instantiation with the substitution ζ (rule (mtch-f4)); for instance, if `irrelevant` is defined by

```
irrelevant not matches open | close;
```

where constant event types `open` and `close` are defined as above, then event $\{event: 'func_post', name: 'fs.open'\}$ and `irrelevant()` do not match because the event matches `open()`.

$RS ::=$	$Main = RE; RD_1; \dots RD_n;$	(RML specification)
$RD ::=$	$Id = RE \mid Id(\vec{x}) = RE$	(RML definition)
$rb ::=$	$x \mid l$	(RML basic data expression)
$\vartheta ::=$	$\tau(rb_1, \dots, rb_n)$	(RML event type pattern)
$rd ::=$	$rb \mid u \mid rd_1 \mid o \mid rd_2$	(RML data expression)
$RE ::=$	empty	(empty trace)
	$\mid all$	(universe)
	$\mid none$	(none)
	$\mid \vartheta$	(event type pattern)
	$\mid Id$	(specification name)
	$\mid Id(rd_1, \dots, rd_n)$	(generic instantiation)
	$\mid RE_1 RE_2$	(concatenation)
	$\mid RE_1 \wedge RE_2$	(intersection)
	$\mid RE_1 \vee RE_2$	(union)
	$\mid RE_1 \mid RE_2$	(shuffle)
	$\mid \{let\ x; RE\}$	(parametric block)
	$\mid RE?$	(optional)
	$\mid RE+$	(postfix plus)
	$\mid RE^*$	(Kleene star)
	$\mid RE!$	(prefix closure)
	$\mid \vartheta \gg RE_1 : RE_2$	(conditional filter)
	$\mid \vartheta \gg RE$	(unconditional filter)
	$\mid if\ (rd)\ RE_1\ else\ RE_2$	(conditional expression)

Fig. 16. Syntax of RML; terms RE are finite.

5. RML formalization

This section provides the formalization of RML syntax and semantics; following the modular approach taken for the trace calculus in Section 3, the definition of RML is deliberately independent from the language used to define event types, which, however, has been formalized in Section 4. This independence is obtained “for free” by defining the semantics of RML directly by translation into the trace calculus; in this way, an RML specification can be associated with a semantics, independently of the definition of the used event types.

The syntax of the language is defined in Fig. 16. It is in line with the examples in Section 2 and 6, except for the minor detail that in the concrete syntax implemented by the compiler the main specification is not required to be the first definition.

The syntax implicitly assumes that names defined in an RML specification are all distinct, and the same assumption holds for the parameters of generics. In the definition and instantiation of a generic we implicitly assume that the sequence of parameters or arguments cannot be empty, hence, at the syntactic level there is a clear separation between non-generic and generic definitions.

RML and the trace calculus exhibit the following syntactic differences:

- in RML terms RE are inductively defined in the standard way, hence, they are finite, but non-generic and generic definitions are supported with the corresponding space name for specifications with identifiers $Id_1, \dots, Id_n$; this is also the concrete standard way to deal with recursion, which is treated more abstractly in the trace calculus by means of regular terms. However, this more concrete syntax would also allow unproductive definitions as, for instance, $Id = Id$, whose translation in the trace calculus is undetermined; for this reason, we assume that the syntax of RML specification in Fig. 16 is restricted to *productive* definitions (see below).
- RML supports a simpler notion of (basic) data expression (and, correspondingly, of event type pattern); compare the definition of rb and rd in Fig. 16 with that of b and d in Fig. 7: RML data expressions can only use primitive literals l , while data expressions of the trace calculus can also contain object and array literals. The reason of such a minor limitation is only implementative, since at the moment the RML compiler supports only this restricted syntax.
- The trace calculus provides both the concatenation and prefixing operators; the latter is needed for the formal definition of the semantics of the calculus, as it explicitly introduces event type patterns in specifications, allowing events to be consumed by the generated monitors, and thus corresponding to the base case of the rewriting semantics (see rule (pre) in Fig. 5); in contrast, RML favors a more user friendly syntax where concatenation is expressed by juxtaposition and prefixing is not required because event type patterns ϑ are automatically compiled into an equivalent trace expression.

As mentioned above, the translation of RML is restricted to productive definitions.

Definition 5.1. A dependency sequence in an RML specification RS is a finite list $Id_1, \dots, Id_n$ of specification names defined in RS s.t.

$$\begin{array}{c}
\text{(main)} \frac{RS \vdash RE \rightsquigarrow t}{\vdash RS \rightsquigarrow t} \quad RS \equiv \text{Main} = RE; Id_1(\bar{x}_1) = RE_1; \dots Id_n(\bar{x}_n) = RE_n; \\
\text{(empty)} \frac{}{RS \vdash \text{empty} \rightsquigarrow \epsilon} \quad \text{(all)} \frac{}{RS \vdash \text{all} \rightsquigarrow 1} \quad \text{(none)} \frac{}{RS \vdash \text{none} \rightsquigarrow 0} \\
\text{(ev-ty-pat)} \frac{}{RS \vdash \vartheta \rightsquigarrow \vartheta : \epsilon} \quad \text{(spec-name)} \frac{RS \vdash RE \rightsquigarrow t}{RS \vdash Id \rightsquigarrow t} \quad Id = RE \text{ in } RS \\
\text{(gen-inst)} \frac{RS \vdash RE \rightsquigarrow t}{RS \vdash Id(\bar{r}d_1, \dots, \bar{r}d_n) \rightsquigarrow \langle (x_1, \dots, x_n).t \rangle (\bar{r}d_1, \dots, \bar{r}d_n)} \quad Id(x_1, \dots, x_n) = RE \text{ in } RS \\
\text{(concat)} \frac{RS \vdash RE_1 \rightsquigarrow t_1 \quad RS \vdash RE_2 \rightsquigarrow t_2}{RS \vdash RE_1 RE_2 \rightsquigarrow t_1 \cdot t_2} \quad \text{(and)} \frac{RS \vdash RE_1 \rightsquigarrow t_1 \quad RS \vdash RE_2 \rightsquigarrow t_2}{RS \vdash RE_1 \wedge RE_2 \rightsquigarrow t_1 \wedge t_2} \\
\text{(or)} \frac{RS \vdash RE_1 \rightsquigarrow t_1 \quad RS \vdash RE_2 \rightsquigarrow t_2}{RS \vdash RE_1 \vee RE_2 \rightsquigarrow t_1 \vee t_2} \quad \text{(shuffle)} \frac{RS \vdash RE_1 \rightsquigarrow t_1 \quad RS \vdash RE_2 \rightsquigarrow t_2}{RS \vdash RE_1 | RE_2 \rightsquigarrow t_1 | t_2} \\
\text{(pref-clos)} \frac{RS \vdash RE \rightsquigarrow t}{RS \vdash RE! \rightsquigarrow t!} \quad \text{(block)} \frac{RS \vdash RE \rightsquigarrow t}{RS \vdash \{\text{let } x; RE\} \rightsquigarrow \{\text{let } x; t\}} \\
\text{(plu)} \frac{RS \vdash RE \rightsquigarrow t}{RS \vdash RE+ \rightsquigarrow t+} \quad \text{(star)} \frac{RS \vdash RE \rightsquigarrow t}{RS \vdash RE* \rightsquigarrow t*} \quad \text{(optional)} \frac{RS \vdash RE \rightsquigarrow t}{RS \vdash RE? \rightsquigarrow t?} \\
\text{(filter-1)} \frac{RS \vdash RE \rightsquigarrow t}{RS \vdash \vartheta \gg RE \rightsquigarrow \vartheta \gg t} \quad \text{(filter-2)} \frac{RS \vdash RE_1 \rightsquigarrow t_1 \quad RS \vdash RE_2 \rightsquigarrow t_2}{RS \vdash \vartheta \gg RE_1 : RE_2 \rightsquigarrow \vartheta \gg t_1 : t_2} \\
\text{(if)} \frac{RS \vdash RE_1 \rightsquigarrow t_1 \quad RS \vdash RE_2 \rightsquigarrow t_2}{RS \vdash \text{if } (rd) RE_1 \text{ else } RE_2 \rightsquigarrow \text{if } (rd) t_1 \text{ else } t_2}
\end{array}$$

Fig. 17. Coinductive definition of compilation of RML into the trace calculus.

- for all $i \in \{1, \dots, n-1\}$, Id_{i+1} occurs in the definition of Id_i in RS ;
- all $Id_1, \dots, Id_n$ are required to be distinct, except for Id_1 and Id_n that are allowed to be equal; in such a case the dependency sequence is called *cyclic*.

As shown by the translation defined below, any generic definition is productive, hence, the only source of unproductivity may come from definitions of non-generic specifications.

Definition 5.2. An RML specification RS is *productive* iff for all cyclic dependency sequences $Id_1, \dots, Id_n$ in RS there exists $i \in \{1, \dots, n-1\}$ s.t. the definition of Id_i in RS does **not** have shape $Id_i = Id_{i+1}$.

The translation of RML into the trace calculus is defined by the rules in Fig. 17 through the following judgments:

- the main judgment $\vdash RS \rightsquigarrow t$ states that the RML specification RS is translated into the term t of the trace calculus;
- the auxiliary judgment $RS \vdash RE \rightsquigarrow t$ states that the RML expression RE is translated into the term t of the trace calculus, w.r.t. the RML specification RS ; this latter is needed to correctly translate a generic instantiation $Id(\bar{r}d_1, \dots, \bar{r}d_n)$ that necessarily refers to the proper definition of Id in RS .

Because of recursion definitions in RS (that is, cyclic dependency sequences in RS), the rules for the judgment $RS \vdash RE \rightsquigarrow t$ (which is defined recursively) have to be interpreted coinductively [18].

Rule (main) is the only one defining the judgment $\vdash RS \rightsquigarrow t$: the definition RE of Main is retrieved from the RML specification RS and is translated w.r.t. the specification itself RS by means of the auxiliary judgment $RS \vdash RE \rightsquigarrow t$.

Besides the requirement on productivity, which is implicitly assumed, the side conditions of rules (spec-name) and (gen-inst) explicitly force two other restrictions: a name of a non-generic definition can be referenced only if it is defined as such in RS , and, analogously, a generic definition can be instantiated only if it is defined as such in RS , and the number of arguments coincide with the number of parameters; in case such side conditions are not verified, the rules cannot be applied, therefore no judgment can be derived and, hence, no translation is provided.

Rule (ev-ty-pat) uses the prefixing operator and the constant ϵ to convert the event type pattern ϑ in a valid expression in the trace calculus (see the comments above); since an RML event type pattern is also valid in the trace calculus (see the comment above), ϑ is kept unchanged in the translation. The same happens for the translation of RML data expressions in rules (gen-inst) and (if). The rest of the rules are trivial.

We can now state the main property that ensures that, under the above mentioned syntactic assumptions, the judgment $RS \vdash RE \rightsquigarrow t$ corresponds to a total function.

Proposition 5.1. Let RS be a productive RML specification and RE an RML expression s.t. all references to non-generic and generic definitions are correct, that is, if Id occurs in RE or the right-hand-side of a definition in RS , then RS contains $Id = RE'$ for some RE' , and if $Id(\bar{r}d_1, \dots, \bar{r}d_n)$ occurs in RE or the right-hand-side of a definition, then RS contains $Id(x_1, \dots, x_n) = RE'$ for some RE' and $x_1, \dots, x_n$. Then there exists a unique term t of the trace calculus s.t. $RS \vdash RE \rightsquigarrow t$ is derivable.

Proof. Because the rules of $RS \vdash RE \rightsquigarrow t$ are interpreted coinductively, $RS \vdash RE \rightsquigarrow t$ is derivable iff there exists a possibly infinite derivation for it [18]. Since the rules cover all possible syntactic cases and the only side conditions, which are in rules (spec-name) and (gen-inst), are always satisfied by the assumption that in RE and RS all references to non-generic and generic definitions are correct, it is always possible to build a valid derivation for $RS \vdash RE \rightsquigarrow t$ for a certain t .

For what concerns the uniqueness of t , because RML terms are finite, by induction on RE we can prove that if the derivation for $RS \vdash RE \rightsquigarrow t$ does not involve rules (spec-name) and (gen-inst), then t is unique and the derivation is necessarily finite; this is due to the fact that for all other rules, premises always refer to subterms of the term RE in the conclusion. However, if any of the rules (spec-name) and (gen-inst) is used, then the derivation can be infinite, but regular, because the definitions in RS are finite; this can only happen if there exists a cyclic dependency sequence in RS . Furthermore, since the trace calculus term in the conclusion of rule (gen-inst) is always guarded by the generic application operator, while the term t in the conclusion of rule (spec-name) is not guarded, the only case when $RS \vdash RE \rightsquigarrow t$ can be derived for different terms t may occur when there exists a cyclic dependency sequence $Id_1, \dots, Id_n$ in RS s.t. the definitions of all Id_i have shape $Id_i = Id_{i+1}$, but this contradicts the assumption that RS is productive. $\square$

6. Specification patterns

In this section we explore several specification patterns that can be adapted to different scenarios to verify properties that are often required for the correct execution of a system. We first consider the problem of monitoring correct acquisition and release of shared resources to avoid issues such as memory leaks and data races. Then we show how generics allow translation of state machines into RML specifications to support quantitative monitoring. Finally, we consider specification patterns for LIFO and FIFO properties, which are commonly required in several scenarios, especially in concurrency and networks, and which cannot be statically verified, at least in their most general formulation, for theoretical limitations. For instance, nested locks [19] are used in practice to guarantee absence of data races in multi-threaded programs; their correctness is based on a LIFO property which cannot be proved with model checking for all classes of programs, because of undecidability results [20]. Communicating finite-state automata with FIFO channels are used to model common Internet protocols as TCP, but verifying asynchronously communicating software is undecidable for FIFO buffers communication in the most general case [21,22]. Interestingly, FIFO properties correspond to sets of traces which are not context-free, hence cannot be expressed and checked by several RV DSLs. All specifications shown in this section have been tested⁷ with the implementation of RML; the corresponding benchmarks are presented in Section 8.

6.1. Resource management

From the parametric specification of iterators in Listing 8 one can extract a first example of specification pattern for a typical property to be verified when a system employs an arbitrary number of statically unknown resources. In this specific case resources are iterators which are acquired with an event of type `newIter(id)`, and implicitly released with an event of type `hasNext(id, false)`.

By disregarding the specific properties of iterators, we can derive the following general specification pattern.

```
// acquire(id), release(id), use(id) to be defined
Main = {let id; acquire(id) (use(id)* release(id) | Main)}?;
```

Listing 11: A pattern for resource management.

By properly defining the event types `acquire(id)`, `release(id)` and `use(id)`, we can adapt the specification above to several contexts; for instance, in the case of iterators, event types can be defined as follows:

```
acquire(id) matches newIter(id);
release(id) matches hasNext(id, false);
use(id) matches next(id) | hasNext(id, true);
```

The specification in Listing 11 checks that resources are properly acquired before their use and then are finally released, but no assumption is made on their use; if additional constraints are required on how resources are used, then the sub-specification `use(id)*` needs to be refined, as already done for iterators.

The following event type definitions show how the specification in Listing 11 can be adapted to verify file management in Node.js:

```
acquire(id) matches
{event:'func_post', name:'fs.open', res:id};
release(id) matches
{event:'func_pre', name:'fs.close', args:[id]};
use(id) matches
```

⁷ https://github.com/RMLatDIBRIS/running_examples/tree/master/journal-paper.

```
{event: 'func_pre', name: 'fs.read' | 'fs.write' | 'fs.fchmod',
  args: [id, ...]};
```

Recursive combination with the shuffle operator allows monitoring of an arbitrary number of statically unknown independent resources. In particular, if n resources are allocated, then the specification is unfolded through recursion into the shuffle of n specifications, each corresponding to a different resource, plus the `Main` specification itself. Whenever an event of type `release(id)` occurs, the number of shuffled specifications is decreased by one, thanks to the optimizations based on the laws proved in Section 3.4 (in particular $e \mid t \equiv t$). Such optimizations are effectively exploited by the RML implementation to generate a monitor able to run with time complexity linear in the length of the monitored trace, assuming that the number of simultaneously allocated resources is bounded by an arbitrary constant.

The specification pattern in Listing 11 does not enforce exclusive use of resources; indeed, it accepts a trace with two consecutive events of type `acquire(id)` with the same `id`, even though no event of type `release(id)` has occurred between the two. This is fine for the two considered cases of iterators and files, because one can safely assume⁸ that the runtime system correctly manages these types of resources: whenever an iterator is created or a file is opened, a new object identifier or file descriptor is returned. However, there are cases where it is important to verify exclusive access to multiple resources, for instance if one wants to check that a lock free application is thread safe. In such cases a variation of the specification pattern is required, as shown in Listing 12.

```
// acquire(id), use(id), release(id) to be defined
acqRel(id) matches acquire(id) | release(id);

Main =
  {let id; acquire(id)
    ((Main | use(id) * release(id)) ^
     acqRel(id) >> release(id) all)
  }?;
```

Listing 12: Parametric specification of exclusive access to multiple resources.

The specification is a refinement of that shown in Listing 11 where an additional constraint has been added with the intersection operator, to require that two consecutive events of type `acquire(id)` with the same `id` cannot occur if no event of type `release(id)` is observed between them. This is expressed with the specification `acqRel(id) >> release(id) all` requiring any event of type `acquire(id)` or `release(id)` to verify `release(id) all`; this is equivalent to asking that the first event matching `acquire(id)` or `release(id)` must necessarily match `release(id)`, that is, an event matching `acquire(id)` cannot occur before an event matching `release(id)`. The universe of all traces `all` is used to specify that no other requirement needs to be verified by the constraint after an event of type `release(id)` has been observed; further acquisition of the same resource `id` will be recursively verified by `Main`.

The condition `acqRel(id) >> release(id) all` can be equivalently specified by `notAcqRel(id) * release(id) all`, where `notAcqRel` is the negation of `acqRel`; the filter operator does not require the introduction of a negative predicate.

As for the specification of Listing 11, the implementation of RML generates a monitor able to run with time complexity linear in the length of the monitored trace, assuming that the number of simultaneously allocated resources is bounded by an arbitrary constant.

The specification scales well⁹ to the more general case where also the entities that access the resources are tracked.

6.2. Specifications with state variables

Generics and conditional expressions can be employed to define specifications depending on data values which need to be updated during monitoring; in this way, state machines can be directly translated into equivalent RML specifications.

Let us consider for instance the problem of verifying that certain operations only use a limited amount of resources; this can be specified with the following pattern.

```
// available(total), use(total,used) to be defined
Use<total> =
  if (total>0) {let used; use(total,used) Use<total-used>}
  else empty;
Main = {let total; (available(total) Use<total>)!};
```

Listing 13: Use of a limited amount of resources.

⁸ Unless the purpose of the monitor is just the verification of the correctness of the runtime system.

⁹ See <https://rmlatdibris.github.io/examples/resource.html#exclusive-access-to-resources>.

An event of type `available(total)` corresponds to checking the availability of a certain resource, where `total` specifies the maximum usable amount; events consuming such resource match `use(total, used)`, where `total` is the amount currently available, and `used` is the consumed quantity, with `used < total`.

With the prefix closure operator the monitor emits the verdict *MaybeTrue* not only when the whole resource amount is used but also when there is still some amount left.

DSLs supporting quantitative monitoring [23] combine numerical aggregation operations with conventional RV formalisms to collect quantitative, statistical data for analysis of streams of data; although RML has been designed as a general purpose DSL for runtime monitoring and verification, specification patterns with state variables partially support quantitative monitoring.

6.3. LIFO properties

We now consider patterns for runtime verification of LIFO properties; based on them, it is also possible to provide specifications to prove the correctness of different stack implementations supporting several operations. For simplicity, the first example we provide is parametric in the data values associated with the stack events, but works only for a single stack; the pattern is then scaled to the more general case of multiple stacks.

The following specification ensures that `pop` can only be called on a non-empty stack, and correctly returns the expected element.

```
// push(val), pop(val) to be defined
Main = { let val; push(val) Main pop(val) }*;
```

Listing 14: Parametric specification of a single stack.

Except for the declaration of variable `val` needed for dealing with parametricity, the specification resembles a context-free grammar. The specification assumes that the stack is initially empty, and verdict *MaybeTrue* can be returned only when the stack is empty; the prefix closure operator can be used to emit the same verdict even when the monitored program terminates with a non-empty stack.

The specification above can be extended with other operations; for instance, uses of the `size` operator can be monitored with a state variable as illustrated in Listing 15. An event matches `size(s)` iff operation `size` has returned `s` as result.

```
// push(val), pop(val), size(s) to be defined
Stack<s> = size(s) Stack<s> v
{ let val; push(val) Stack<s+1> pop(val) Stack<s> }?;
Main = Stack<0>;
```

Listing 15: Parametric specification of a single stack with size.

Since the stack is initially assumed to be empty, the main specification instantiates the generic `Stack<s>` with 0.

To scale the specification to multiple stacks, the event types `new` and `free` have to be introduced to monitor their dynamic allocation and deallocation, and the event types `push`, `pop` and `size` need to be extended to arity 2, to keep track of the id of the stack on which the operation is executed.

```
// new(id), free(id) to be defined
// push(id, val), pop(id, val), size(id, s) to be defined
Stack<id,s> = size(id,s) Stack<id,s> v
{let val;push(id,val)Stack<id,s+1>pop(id,val)Stack<id, s>}?;
Main = {let id;new(id) (Main|(Stack<id,0>free(id))) }?;
```

Listing 16: Parametric specification of multiple stacks.

The generic `Stack<id,s>` is the generalization of `Stack<s>` in Listing 15; the main specification is based on the pattern of Listing 11 to manage allocation and deallocation of an arbitrary number of stacks.

When an element is popped (for all specifications, Listing 14, 15 and 16) or a stack is deallocated (for the specification in Listing 16), optimizations applied by the generated monitor (see the laws $t \cdot \epsilon \equiv t$ and $t \mid \epsilon \equiv t$ in Section 3.4) guarantee that the time complexity of its execution is linear (see Section 8) in the length of the monitored trace, assuming that the number of simultaneously allocated stacks is bounded by an arbitrary constant.

Verdict *MaybeTrue* can be emitted only when all allocated stacks have been emptied and, then, deallocated; the specification can be weakened by using the prefix closure operator. Equivalent versions of the specifications defined above exploit

the filter and intersection operators¹⁰ to provide even more modular solutions that scale better when additional stack operations have to be monitored.

6.4. FIFO properties

We now turn to consider patterns for runtime verification of FIFO properties; based on them, it is also possible to provide specifications to prove the correctness of different queue implementations supporting several operations.

Specifying FIFO properties is challenging, because the corresponding set of traces is not a context-free language; indeed, some of the RV DSLs presented in Section 9 cannot express such kind of properties.

We can prove that FIFO properties are not context-free in general by exploiting the fact that the language $L_{FIFO} = \{a^m b^n c^m d^n \mid m > 0, n > 0\}$ is not context-free. This can be shown by directly applying the Bar-Hillel lemma (the pumping lemma for context-free languages) [24], stating that for any context-free language L there exists a pumping length $p > 0$ such that every string $s \in L$ of length $|s| = n$ s.t. $n \geq p$ can be decomposed as $s = uvwxy$ such that

1. $|vx| > 0$;
2. $|vwx| \leq p$;
3. $uv^n wx^n y \in L$, for all $n \geq 0$.

The proof proceeds by contradiction as follows. If we assume that L_{FIFO} is context-free, then we can apply the Bar-Hillel lemma to the string $a^p b^p c^p d^p$, where p is the pumping length of L_{FIFO} . Because $|vwx| \leq p$ by condition 2 of the lemma, we can deduce that the string vx contains 'a' symbols iff it does not contain 'c' symbols, and contains 'b' symbols iff it does not contain 'd' symbols, therefore, by condition 1 (vx cannot be empty), uvw cannot belong to L_{FIFO} , thus contradicting condition 3 for $n = 0$.

Now let us consider to monitor the following distinct events observed on an initially empty FIFO queue over a set that contains at least two different values v_1 and v_2 :

```
enq(v) // value v has been inserted on the queue
deq(e) // value v has been removed from the queue
```

Let T_{FIFO} denote the set of all finite traces on such events which can only be observed from the correct implementation of a FIFO queue and which lead to an empty queue.

As an example, $\text{enq}(v_1) \text{enq}(v_2) \text{deq}(v_1) \text{deq}(v_1) \notin T_{FIFO}$ because the second removed element should be v_2 , and $\text{enq}(v_1) \text{enq}(v_2) \text{deq}(v_1) \notin T_{FIFO}$ because after the last observed event the queue is not empty.

Let us abbreviate $\text{enq}(v_1)$ and $\text{enq}(v_2)$ with a and b , and $\text{deq}(v_1)$ and $\text{deq}(v_2)$ with c and d , respectively; let R be the set of traces defined by the regular expression $a^+ b^+ c^+ d^+$; then, by definition, $T_{FIFO} \cap R = L_{FIFO}$. Because L_{FIFO} is not context-free and the intersection of a context-free language with a regular language is still a context-free language [25], we conclude that T_{FIFO} cannot be context-free.

Listing 17 defines a specification for correct FIFO queues with insertion and removal operations; only a single queue is monitored, but the specification can be scaled to the case of multiple queues dynamically allocated and deallocated in a similar way as done for stacks in Listing 16. The specification is based on the two event types $\text{enq}(\text{val})$ (value val has been enqueued) and $\text{deq}(\text{val})$ (value val has been dequeued); from the latter, the event type deq is derived (a value has been dequeued).

```
// enq(val), deq(val) to be defined
deq matches deq(_);

Main = {let val; enq(val) ((deq|Main) ^ (deq >> deq(val) all))}?
```

Listing 17: Parametric specification of a single queue.

Since the queue is initially empty, every correct trace starts with an event matching $\text{enq}(\text{val})$, for a certain value val . Then, the rest of the trace must verify the conjunction of the following two sub-specifications:

1. $(\text{deq}|\text{Main})$: the trace is a correct trace of Main where, additionally, sooner or later an event matching deq has to be observed, to remove the previously inserted value; hence, $(\text{deq}|\text{Main})$ keeps track of the number of events of type deq that are required to be observed to guarantee that the queue will be eventually emptied;
2. $(\text{deq} >> \text{deq}(\text{val}) \text{all})$: the first observed event matching deq , if any, must correspond to the removal of the previously inserted value val .

¹⁰ See <https://rmlatdibris.github.io/examples/lifo.html#approach-by-decomposition>.

The two sub-specifications are both required: the former requires events of type `deq` to occur, but does not impose any order on them, while the latter ensures their correct order, but does require them to occur. For this reason, $(\text{deq} \mid \text{Main})$ works equally well as $(\text{deq}(\text{val}) \mid \text{Main})$ because the value of the removed element is already checked by $(\text{deq} \gg \text{deq}(\text{val}) \text{ all})$.

The deterministic semantics of RML (in particular, of the shuffle in this example) is crucial to ensure the correctness of the specification in Listing 17; if the two rewriting rules for the shuffle operator in Fig. 5 (shuffle-l) and (shuffle-r) could be applied non-deterministically, then the specification would accept also non-correct traces; for instance, the *MaybeTrue* verdict would be emitted for the trace consisting of six events, matching `enq(1)`, `enq(1)`, `enq(2)`, `deq(1)`, `deq(2)` and `deq(1)`, respectively. Indeed, this happens also with the deterministic semantics if we replace $(\text{deq} \mid \text{Main})$ with $(\text{Main} \mid \text{deq})$ or $(\text{Main} \mid \text{deq}(\text{val}))$, so that *Main* has higher precedence.

Finally, the operator $?$ is used to allow the *MaybeTrue* verdict to be only emitted when the queue is empty, as done for the specifications of stacks in Listing 15 and 16; again, the prefix closure operator can be employed to emit *MaybeTrue* also in case of non-empty queues.

We now show how the specification of FIFO queues can be extended to monitor also the `size` operator; differently from the approach followed for stacks in Listing 15, here we adopt a more modular solution that scales better when extending the specification to monitor additional queue operations; this is achieved by decomposing the specification into simpler ones, thanks to the intersection and filter operators.

```
// event types enq(val), deq(val), size(s) to be defined
enq matches enq(_);
deq matches deq(_);
enqDeq matches enq | deq;

Queue={let val;enq(val)((deq|Queue) ^ (deq>>deq(val) all))}?;
Size<s> =
  ((size(s)Size<s>) v (enq Size<s+1>) v deq Size<s-1>)?;
Main = (enqDeq>>Queue) ^ Size<0>;
```

Listing 18: Specification of a queue with size monitoring.

Queue defines the same specification of Listing 17, while the generic *Size<s>* exploits the state variable *s*, as already done in Listing 15 and 16, to correctly keep track of the size of the queue.

The two specifications *Queue* and *Size<s>* allow a strict separation of concerns: the former is focused on the correct behavior of the enqueue and dequeue operators, while the latter is concerned with the correctness of the size operator only; for instance, in *Size<s-1>* no check is performed to ensure that the generic is instantiated with a non-negative value, because *Queue* is responsible for ensuring that no dequeue operation is performed on empty queues.

The two specifications can be combined together thanks to the intersection operator; the filter operator is used to disregard events of type `size(_)` which are not relevant for *Queue*. As in the previous example, the queue is assumed to be initially empty, and the *MaybeTrue* verdict can be emitted only when the queue is empty.

When an element is dequeued (for both specifications, Listing 17 and 18), optimizations applied by the generated monitor (see the laws $\epsilon \mid t \equiv t$, $\theta \gg 1 \equiv 1$ and $t \wedge 1 \equiv t$ in Section 3.4) guarantee that the time complexity of its execution is linear (see Section 8) in the length of the monitored trace, assuming that the number of simultaneously allocated queues is bounded by an arbitrary constant.

Several variations¹¹ of the specifications above can be considered for other queue operations, for randomized queues and for queues with no repetitions.

7. Implementation

This section provides some details of the implementation of RML hosted on GitHub.¹² The implementation of the RML system consists of many modules using different languages and frameworks, and an overview of the architecture is depicted in Fig. 18. The final goal is to generate from an RML specification a monitor implemented in Prolog, the language of choice for the implementation of the rewriting system presented in Section 3.

The GitHub repository includes:

- a *compiler* from RML to Prolog;
- a Prolog *monitor* implementing the RML semantics;
- an example of a real-world *instrumentation* developed for Node.js¹³ (the cross-platform JavaScript runtime environment).

¹¹ See <https://rmlatdibris.github.io/examples/fifo.html> and Section 8.

¹² <https://github.com/RMLatDIBRIS>.

¹³ <https://nodejs.org/en/>.

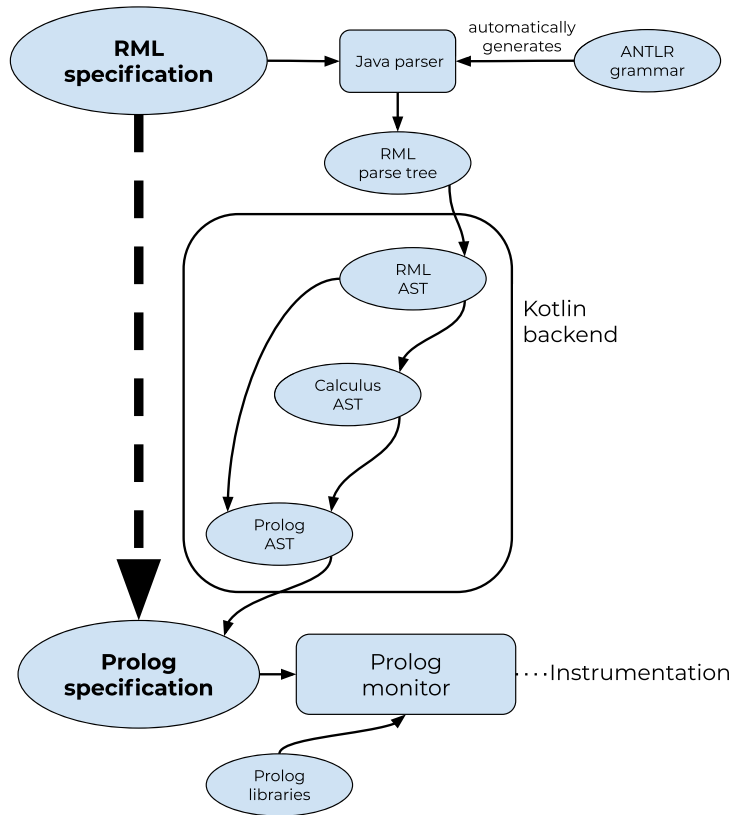


Fig. 18. Overview of the RML compiler architecture.

A website¹⁴ devoted to RML and its usage is also available.

7.1. Compiler

As it is customary in language implementation, the first step is lexical and syntactical analysis of the source code. This is implemented using *ANTLR*¹⁵ [26], an advanced, powerful and flexible parser generator. The syntax of RML is formalized as an ANTLR grammar: examples of lexical and syntactical (parts of) of the grammar are shown in Listings 19 and 20.

```

ELLIPSIS: '...' ;
INT: [0-9]+ ;
FLOAT: INT '.' INT ;
STRING: '\\' [ a-zA-Z0-9_.* '\\ ' ] ;

```

Listing 19: Portion of ANTLR lexical grammar for RML.

Lexical rules are mostly based on standard regular expression operators, while ANTLR parsing grammar is more interesting, as it offers quite a few features that are useful for the implementation of RML:

- it is built on top of a widely understood formalism, namely context-free grammars;
- productions can be decorated with names (introduced by #): this is used by the parser generator to introduce Java classes with meaningful names to implement the different kinds of nodes of the parse tree in the produced parser;
- associativity can be easily changed (see concatenation);
- left-recursion, which is often a problem in parsing, is automatically handled by ANTLR (the only limitation is related to mutually left-recursive rules);
- when parts of the production are assigned a name (as in `leftBranch=exp`), the generated parser will produce a parse tree whose nodes will be objects with fields named accordingly.

¹⁴ <https://rmlatdibris.github.io/>.

¹⁵ <https://www.antlr.org/>.

```
// RML expression syntax
exp: exp '*' # starExp
    | exp '+' # plusExp
    | exp '?' # optionalExp
    | exp '!' # closureExp
    | <assoc=right> exp exp # catExp
    | exp '/' exp # andExp
    | exp '\\/' exp # orExp
    | exp '|' exp # shufExp
    | evtype '>>' leftBranch=exp (':' rightBranch=exp)? # filterExp
    | 'empty' # emptyExp
    | 'all' # allExp
    | '{' 'let' evtypeVar (',' evtypeVar)* ';' exp '}' # blockExp
    | 'if' '(' dataExp ')' exp 'else' exp # ifElseExp
    | expId ('<' dataExp (',' dataExp)* '>')? # varExp
    | evtype # evtypeExp
    | '(' exp ')' # parenExp
;
```

Listing 20: Portion of ANTLR parsing grammar for RML.

When fed with a grammar, ANTLR generates a parser; many languages are supported, but in our case Java is used. Such parser produces a parse tree supporting the visitor pattern [27], which allows users to traverse the tree according to their needs.

While the automatically generated parser is a Java program, the software we developed for the internals of the compiler is written in *Kotlin*.¹⁶ Kotlin is a modern, concise, statically typed, object-oriented programming language that runs on the JVM and is fully interoperable with Java.

The parse tree generated by ANTLR is a low-level representation of the RML specification, but still includes many parsing-related information. The next step of the compilation process is to translate the parse tree into a more manageable abstract syntax tree (AST) implemented in Kotlin. The language is well suited to the task, and AST classes are encoded in just a few lines, as shown in Listing 21. Such classes are produced by the Kotlin code which interfaces with Java and implements the visit of the parse tree. Sticking to Java for the whole project would have led to a lot of boilerplate code, as Java is much more verbose.

AST classes are implemented with a combination of *data classes* and *sealed classes*. The former feature makes the Kotlin compiler automatically generate all common class methods (as `toString` and `equals`) and assigns to the instances a value (as opposed to reference) semantics, when compared to other instances. The latter ensures that the whole class hierarchy is known at compile-time; this, combined with Kotlin type inference algorithm and its functional programming support, allows the developer to choose a functional style based on pattern matching. This comes in handy when implementing languages and manipulating AST, tasks that are often more suited to a functional programming style.

At this point, the specification is encoded in a compact, high-level Kotlin AST encoding the RML structure. Here the compilation pipeline is divided in two parts (see Fig. 18). Event type definitions are directly compiled into a Prolog AST, while specification definitions are converted to another AST encoding the syntax of the trace calculus (Fig. 3). The additional step favors modularity by using the trace calculus as intermediate representation; in this way optimizations can be implemented more easily.

The two lines then meet again and produce the AST of the resulting Prolog program. Since the Prolog dialect of choice is fixed (SWI-Prolog¹⁷) the AST is closely related to its particular syntax, and exploits it whenever convenient. This allows the RML compiler to produce a human-readable Prolog program that can be inspected if needed (see Listing 22).

The ASTs for Prolog and the trace calculus are structured in the same way as the RML AST (see Listing 21).

7.2. Prolog semantics

The main reason why Prolog has been the language of choice for the implementation of the rewriting semantics relies on the underlying programming paradigm. Logic programming is based on rules and unification, thus a logic program is extremely similar to an inference system, and the translation of the rewriting rules is natural. Consider for instance rule (and) of Fig. 5:

$$\text{(and)} \frac{t_1 \xrightarrow{e} t'_1; \sigma_1 \quad t_2 \xrightarrow{e} t'_2; \sigma_2}{t_1 \wedge t_2 \xrightarrow{e} t'_1 \wedge t'_2; \sigma} \sigma = \sigma_1 \cup \sigma_2$$

In our Prolog implementation this is represented by the following clause:

¹⁶ <https://kotlinlang.org/>.

¹⁷ <https://www.swi-prolog.org/>.

```
sealed class Expression

// regex-like unary operators
data class StarExpression(val exp: Expression): Expression()
data class PlusExpression(val exp: Expression): Expression()
data class OptionalExpression(val exp: Expression): Expression()

data class PrefixClosureExpression(val exp: Expression): Expression()

// binary operators
data class ConcatExpression(val left: Expression,
    val right: Expression): Expression()
data class AndExpression(val left: Expression,
    val right: Expression): Expression()
data class OrExpression(val left: Expression,
    val right: Expression): Expression()
data class ShuffleExpression(val left: Expression,
    val right: Expression): Expression()

// conditional operators
data class FilterExpression(val eventType: EventType,
    val filteredExpression: Expression,
    val unfilteredExpression: Expression?): Expression()
data class IfElseExpression(val condition: DataExpression,
    val thenExpression: Expression,
    val elseExpression: Expression): Expression()
```

Listing 21: A code snippet from the Kotlin implementation of the RML AST nodes.

```
:- module(spec, [(trace_expression/2), (match/2)]).
:- use_module(monitor(deep_subdict)).
:- use_module(library(clpr)).
match(_event, push_et(Val)) :- deep_subdict(_event, _{event:"func_pre", name:"mypush", args:[Val]}).
match(_event, pop_et(Val)) :- deep_subdict(_event, _{event:"func_post", name:"mypop", res:Val}).
match(_event, relevant_et) :- match(_event, push_et(_)).
match(_event, relevant_et) :- match(_event, pop_et(_)).
match(_event, any_et) :- deep_subdict(_event, _{}).
match(_event, none_et) :- not(match(_event, any_et)).
trace_expression('Main', Main) :- (Main=((relevant_et>>Stack);1)),
    (Stack=star(var(val), (push_et(var(val))*(Stack*pop_et(var(val)))))).
```

Listing 22: Example of an RML specification compiled into Prolog.

```
next(T1/T2, O, T, S) :- !,
    next(T1, O, T3, S1),
    next(T2, O, T4, S2),
    merge(S1, S2, S),
    conj(T3, T4, T).
```

The atoms in the clause have the following meaning:

- ! (known as “cut” in logic programming) tells the Prolog interpreter not to backtrack in case of failure, since there are no other rules applicable for the intersection operator;
- next(T1, O, T3, S1) corresponds to the premise $t_1 \xrightarrow{e} t'_1; \sigma_1$;
- next(T2, O, T4, S2) corresponds to the premise $t_2 \xrightarrow{e} t'_2; \sigma_2$;
- merge(S1, S2, S): corresponds to the side-condition $\sigma = \sigma_1 \cup \sigma_2$;
- conj(T3, T4, T) applies the optimizations based on the laws of the calculus presented in Section 3.4 to simplify, when possible, the term $t'_1 \wedge t'_2$.

Parametric variables need to be implemented with care, as variables in the specification must not be confused with Prolog variables. In a previous version of the implementation we tried to encode parametric specifications with Prolog variables, but that clutters the implementation with bookkeeping code and meta-programming features, to avoid unsound interactions with unification. The current implementation follows a simpler approach and encodes a specification variable x as a term `var(x)` (note that x is a symbol name and not a variable identifier, which cannot start with a lowercase letter in Prolog).

Substitutions are implemented with association lists, that is, lists of terms of shape `var_name=term`. Association lists are widely used in Prolog, and the whole list library can be used to manipulate them, helping with the implementation of all substitution operations as “application” and “merge”.

Since the terms of the trace calculus are regular, it is extremely useful to have language support for cyclic terms. SWI-Prolog directly provides that by allowing unification of a variable x with a term containing x itself. For instance, unification $x = f(x)$ succeeds and instantiates variable x with the (only) regular term satisfying that equation.

One of the main reasons why we chose SWI-Prolog in particular as a dialect, is its native support for *coinductive logic programming* [28]. Since terms are regular, substitution application is coinductively defined on them (Fig. 9). In coinductive logic programming, predicates are labeled either as inductive (by default) or coinductive. When coinductive predicates are involved in the derivation, after each resolution step the interpreter checks whether the selected atom of the goal unifies with any of the previously resolved ones, and if it does, then a cycle is detected and the resolution succeeds by coinduction and produces the substitution resulting from that unification.

Substitution application to prefix terms $ET1:T1$, for instance, is defined by the following Prolog clause:

```
apply_sub_trace_exp(S, ET1:T1, ET2:T2) :- !,
    apply_sub_event_type(S, ET1, ET2),
    apply_sub_trace_exp(S, T1, T2).
```

If we consider the cyclic SWI-Prolog term (corresponding to a valid regular term of the trace calculus) defined by the unification $T1=ET1:T1$, then the standard inductive resolution would result in an infinite loop, where the interpreter keeps trying to apply substitution s to term $T1$.

However, in SWI-Prolog, we can mark the predicate as coinductive:

```
:- use_module(library(coinduction)).
:- coinductive apply_sub_trace_exp/3.
```

This allows us to get the correct coinductive (and, thus, terminating) semantics when substitutions are applied to regular terms.

The *match* relation derived from the event type definitions, according to the semantics in Section 4, and used in the rewriting rules for prefixing in Fig. 5 and 6, is defined by the Prolog predicate *match*, whose clauses are automatically generated by the RML compiler. For instance, let us consider the following event type definition:

```
operation(fd) matches {
    event: 'func_pre',
    name: 'fs.write' | 'fs.read',
    args: [fd, ...]
};
```

The RML compiler then produces the following clauses:

```
match(_event, operation_et(Fd)) :- deep_subdict(_event, _{event:"func_pre", name:"fs.write", args:[Fd|_]}).
match(_event, operation_et(Fd)) :- deep_subdict(_event, _{event:"func_pre", name:"fs.read", args:[Fd|_]}).
```

Differently from RML variables in specification terms, here it is correct to translate variables occurring in event type definitions into proper Prolog variables, since we actually want them to unify with (part of) the event sent by the instrumentation and bound to *_event*.

SWI-Prolog recently introduced dictionaries, that is, object-like terms having shape `tag{field1: term1, ..., fieldN: termN}`. Event patterns are conveniently represented with dictionaries, because of their object-like structure.

Unfortunately, at the moment the SWI-Prolog standard library for dictionaries does not implement a predicate to check whether a dictionary is a “deep” sub-dictionary of another one by recursively traversing down the dictionary structure. We implemented such a feature as part of the library that can be used by the generated Prolog monitor, and exported it as predicate *deep_subdict*. Recalling the event type matching semantics, *deep_subdict* allows an event to match a pattern even if it has *more* fields than requested.

Dictionaries are in turn exploited by the SWI-Prolog standard library `library(http/json)` for JSON (de)serialization and manipulation, which is used to inspect the events (that are expected to be in JSON format) received from the instrumentation.

Finally, union value patterns as `'fs.write' | 'fs.write'` are implemented by unfolding them to produce a set of patterns covering all the possible combinations, without containing any union operator themselves. Since the previous pattern is used only once in the event type definition of *operation(fd)* above, only two corresponding Prolog clauses are generated.

7.3. Monitor

After an RML specification has been compiled into Prolog, another Prolog module, the *monitor*, loads the Prolog specification and then starts handling events in JSON format received from the instrumentation: by using the developed library mentioned above to implement the rewriting semantics of the trace calculus, the monitor checks whether a rewrite step is allowed for the incoming event, and, if so, updates its state with the Prolog term representing the new term obtained by rewriting.

Our tool can generate monitors working according to two different modes.

In *offline* mode, the monitor reads a trace from a log file, which is expected to contain a single JSON event per line, then verifies the trace and outputs a verdict. Offline monitoring is still considered to be runtime verification: even if the verification process happens after termination of the program, we are still analyzing a single execution of the program.

In *online* mode, the monitor listens on an input channel for events, and verifies them one at a time, responding every time with a verdict and updating its internal state according to the rewriting semantics. Online mode is especially interesting for runtime verification of distributed and heterogeneous systems, when multiple components (possibly implemented using different technologies) are instrumented to connect to the monitor. At current time, two types of communications are supported:

HTTP: The monitor works as a web server waiting for HTTP requests. HTTP is a widely understood and implemented protocol, thus it is generally easy for instrumented components to send events to the monitor. However, an entirely new HTTP request is made every time, possibly affecting performance for high event flow-rate.

WebSocket: The monitor opens a WebSocket communication channel and waits for events; this mode is highly recommended for high event flow-rate, because in this case communication is more efficient since the connection stays active once established.

Both protocols are supported by the SWI-Prolog standard library.

8. Experiments

In this section we report on the experiments conducted with the implementation of RML, with the examples shown¹⁸ in Section 6, others that have been omitted in Section 6 for space reasons and a more substantial case study based on the iterator pattern [27].

The main aim of these experiments is to empirically show that for all examples the execution time of the generated monitor is linear either in the size of the trace (that is, the number of monitored events) when the size of the monitored data¹⁹ is bounded, or in the size of the monitored data when the trace length is fixed. All experiments have been run on an Intel(R) Core(TM) i7-7700HQ CPU at 2.80 GHz with a 16 GB RAM, running SWI-Prolog version 8.0.3 for x86_64-linux and Linux Ubuntu 18.04.2 LTS, kernel version 4.15.0-54-generic.

Since our purpose is to test the efficiency of the generated monitors, they have been run offline on the log files containing the events generated by the instrumented code. All log files have been generated by running Node.js scripts instrumented with *Jalangi2*²⁰ [29], a tool supporting source code instrumentation of JavaScript. The tool is able to trace several kinds of events related to the execution of JavaScript code by invoking a corresponding callback that can be properly defined to customize the instrumentation to the specific aims. In our case, the only action performed by the instrumentation is logging events in JSON format to a file, and the only events that are logged are function calls and returns; for instance, the call `enqueue(42)` and the return from call `dequeue()` with value 42 generate the following event²¹ objects, in JSON format:

```
{event: 'func_pre', name: 'enqueue', args: [42]}
{event: 'func_post', name: 'dequeue', args: [], res: 42}.
```

To better simulate a real scenario, the log files contain all generated events, and not just those relevant for the verification, therefore all the specifications used in the experiments contain a top level filter operator to consider only the events that need to be checked; this is what typically happens in a monitored system, where different and possibly unrelated properties are verified simultaneously.

8.1. Resource management

Besides the examples in Listing 11 and 12, we have also included the specification to verify the basic operations on sets for adding and removing elements; as in many set interfaces implemented by standard libraries, both operations return `true` iff the corresponding operation has modified the set.

The specification of a set with basic operations (see Listing 23) is a minor adaptation of the pattern used for exclusive access to multiple resources provided in Listing 12: the elements of the set play the role of the resources, adding a new element e to the set corresponds to acquiring a new resource (event type `acquire(e)`), removing an element e which was in the set is equivalent to releasing it (event type `release(e)`), while adding an element e which is already in the set is considered as a use of an already allocated resource (event type `use(e)`). The only event that has no counter-part in the specification of Listing 12 is the removal of an element e which does not belong to the set; this amounts to considering also those events that are allowed to involve resources that are not yet allocated (event type `no_use(e)`).

¹⁸ Except for Listing 13, which does not pose any challenge in terms of execution time.

¹⁹ Since the notion of data size depends on the specific example, more details are provided with the comments to the benchmarks.

²⁰ <https://github.com/Samsung/jalangi2>.

²¹ The generated objects actually contain additional properties which, however, are not exploited in the presented examples, and, therefore, are omitted for sake of simplicity.

```
// event types acquire(e1), use(e1), release(e1), no_use(e1) to be defined

no_use matches no_use(_);
acqRelNoUse(e1) matches acquire(e1) | release(e1) | no_use(e1);

Main = (no_use* {let e1; acquire(e1)
  ((Main | use(e1)* release(e1)) ^
    acqRelNoUse(e1) >> (release(e1) all))});
```

Listing 23: Specification of a single set with operations for adding/removing elements.

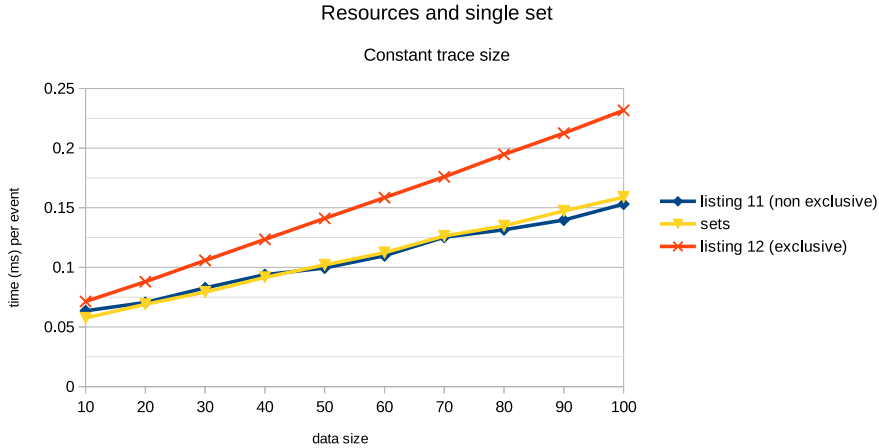


Fig. 19. Resource management, variable data size.

For the measurements concerning the specifications of resource management, the data size corresponds to the maximum number of simultaneously acquired resources (specifically for Listing 23 this actually corresponds to the size of the set); indeed, the size of the term of the trace calculus representing the state of the monitor is proportional to this parameter.

The instrumented code used for all experiments reported in Fig. 19 and 20 always generates correct traces, to consider the worst-case scenario where monitors have to inspect all the events before terminating. To further stress monitors, the code repeats the following steps several times²²: first acquire the maximum number of resources (corresponding to the data size), then use them, and finally release all of them in inverse order. In this way, and thanks to the interpreter optimizations based on the laws of the calculus presented in Section 3.4, the size of the term representing the current state of the monitor repeatedly increases to its largest size and then decreases to its initial size. The charts in Fig. 19 summarize the measurements for the monitors generated from the specifications in Listing 11, 12 and 23, with variable data size (between 10 and 100) and constant trace size²³ (20K events for Listing 11 and 12, and 50K for Listing 23). Despite for each of the three listings the size of the traces is kept constant when the data size increases, the sizes for Listing 11 and 12 and Listing 23 differ; nonetheless, the charts are comparable since the total times required by the monitors are normalized by dividing them by the size of the analyzed trace; that is, the time needed for processing a single event is considered; similar comments apply to all the other figures in this section. To be able to compare measurements, in all charts presented in this section the Y-values contain the time (in ms) required for monitoring a single event, rather than the total time required by the monitors for checking an entire trace.

All three charts show that the time required for monitoring a single event linearly increases with the data size. The charts in Fig. 20 concern the measurements for the same generated monitors, but with variable trace size (between 10K and 70K events) and constant data size 50. The charts show that the time required for monitoring a single event does not depend on the size of the trace, hence, when the data size is bounded, the total time required by the monitors for checking an entire trace linearly increases with its size.

8.2. LIFO properties

For the specifications in Listing 14 and 15 of a single stack, the data size corresponds to the maximum number of elements in the stack, while in case of the specification in Listing 16 where multiple stacks are verified, the data size is the maximum of the sum of the sizes of all checked stacks. Because the verified properties and the definitions of data size

²² The number of iterations depends on the tested property and the corresponding size of the trace, but it typically varies between 100 and 10k.

²³ The size of the traces cannot be arbitrarily chosen if one considers the worst-case scenario where the generated traces must be correct.

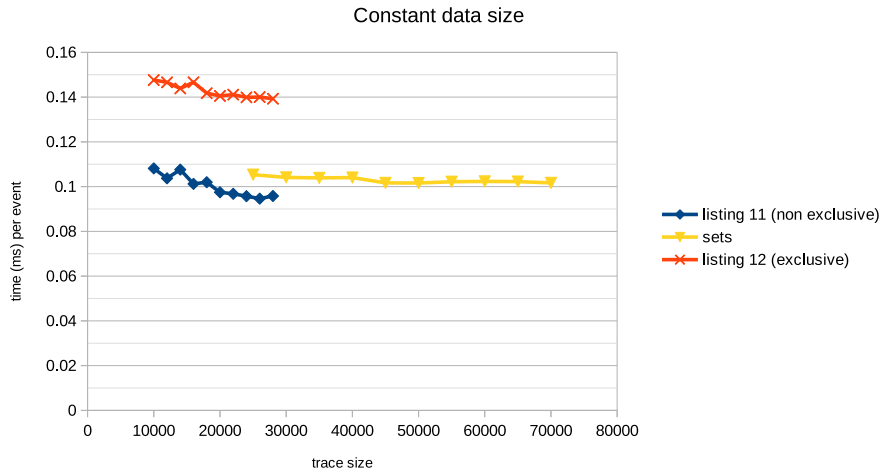


Fig. 20. Resource management, variable trace size.

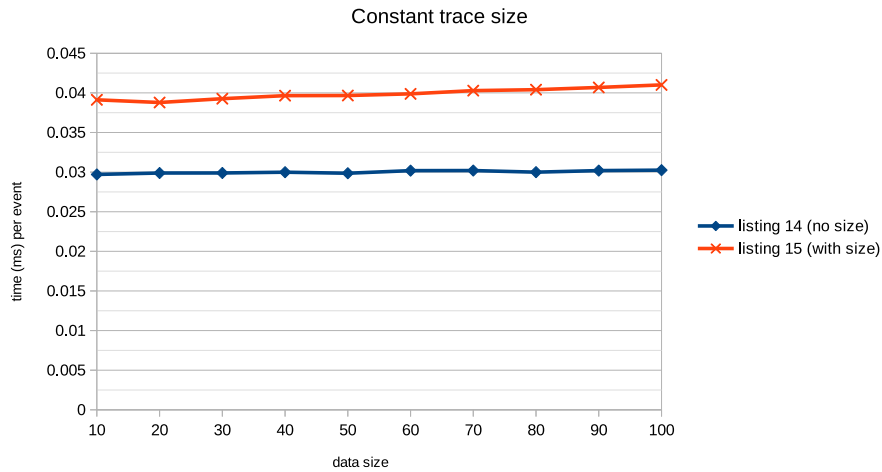


Fig. 21. Single stack, variable data size.

are substantially different, the performances of the generated monitors are not comparable and, hence, the charts for the specifications of a single stack are kept separate from that of multiple stacks.

As in the previous examples, the Node.js scripts used to generate the event traces repeat the following steps several times. For specification in Listing 14, after creating an empty stack, elements are pushed until the maximum value for data size is reached, then all elements are popped from the stack until the stack is empty; again, thanks to the interpreter optimizations based on the laws of the calculus presented in Section 3.4, the size of the term representing the current state of the monitor repeatedly increases to its greatest size and then decreases to its initial size.

For specifications in Listing 15 and 16 which consider also the size operation, the corresponding function is called immediately after each push and pop operation.

For the specification on multiple stacks the structure of the script is more complex, since an outer loop is needed to deal with dynamic allocation and deallocation; stacks are repeatedly created and filled with elements, then they are emptied and deallocated, following the same allocation order. In this way, for each event the monitor has to always inspect the entire specification corresponding to its current state.

The charts in Fig. 21 show that for the specification of a single stack with variable data size (between 10 and 100) and constant²⁴ trace size, the time for monitoring a single event is basically constant. This is due to the fact that for each event the monitors always need to inspect only a fixed-size portion of the specification corresponding to their current state. However, since the size of the specification linearly increases with that of the stack, an overhead for dynamic memory allocation leads to a minimal performance degradation when the size of stack increases, as especially witnessed by the chart for Listing 15.

²⁴ 200K events for Listing 14 and 150K for Listing 15.

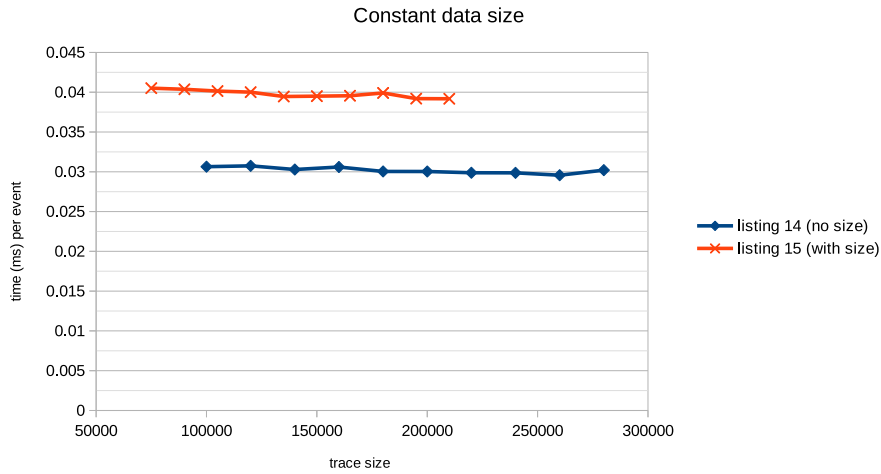


Fig. 22. Single stack, variable trace size.

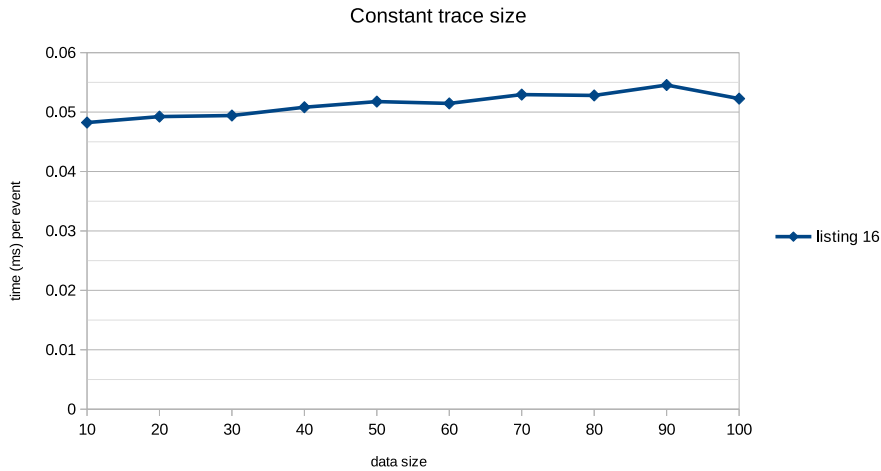


Fig. 23. Multiple stacks, variable data size.

The measurements for variable trace size (between 50K and 300K events) and constant data size 50 in Fig. 22 confirm that also for these specifications the total time required by the monitors for checking entire traces linearly increases with their size, when the data size is bounded.

For the specification of multiple stacks, when the data size is variable (between 10 and 100) and the trace size is constant (120K events, see Fig. 23), the time increases linearly with the data size, since in this case the monitor cannot inspect a fixed-size part of the specification representing its current state, when checking pop operations; this is directly related to the fact that stacks are emptied in the same order in which they have been allocated to generate worst case traces.

Also for this specification, the chart in Fig. 24 shows that, when the data size is constant (50), the total time required by the monitor for checking entire traces linearly increases with their size (between 50K and 170K events), although it is slowly decreasing as the trace size increases.

8.3. FIFO properties

Besides the specifications presented in Listing 17 and 18, we have included in the experiments monitors generated from other three variations of the specification of Listing 17: random queues, where elements are enqueued at random positions, queues with no repetitions, where an element is not enqueued if already present in the queue, and the combination of both options.

The corresponding specifications are shown in Listing 24, 25 and 26.

The Node.js scripts developed to generate the events follow the same pattern as for the examples on LIFO properties: an empty queue is created, then filled with elements and finally emptied by dequeuing all elements, and these steps are repeated several times.

The charts in Fig. 25 and 26 exhibit a similar behavior to those for the examples on resource management.

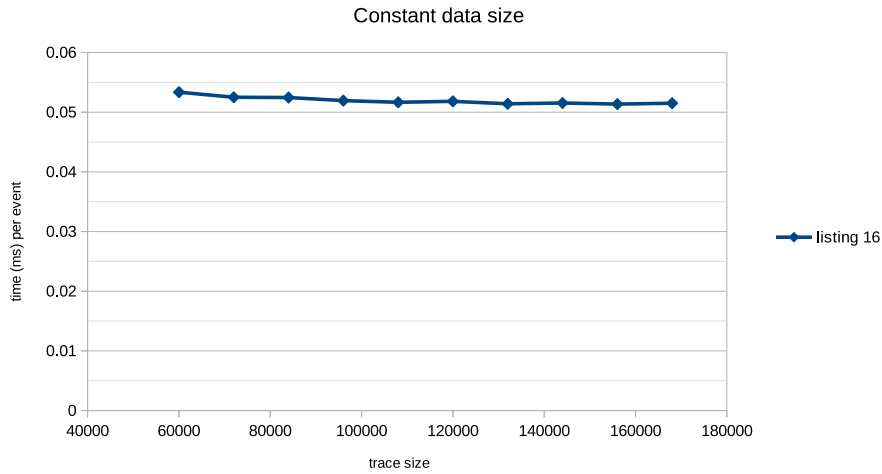


Fig. 24. Multiple stacks, variable trace size.

```
// event types enq(val), deq(val) to be defined
deq matches deq(_);

Main={let val; enq(val) ((enq(val)* deq(Main) ^ (deq>>deq(val) all)))?;
```

Listing 24: Specification a single queue with no repetitions.

```
// event types enq(val), deq(val) to be defined

Main = {let val; enq(val) ( deq(val) | Main)}?;
```

Listing 25: Specification of a single random queue.

```
// event types enq(val), deq(val) to be defined

Main = {let val; enq(val) (enq(val)* deq(val) | Main)}?;
```

Listing 26: Specification of a random queue with no repetitions.

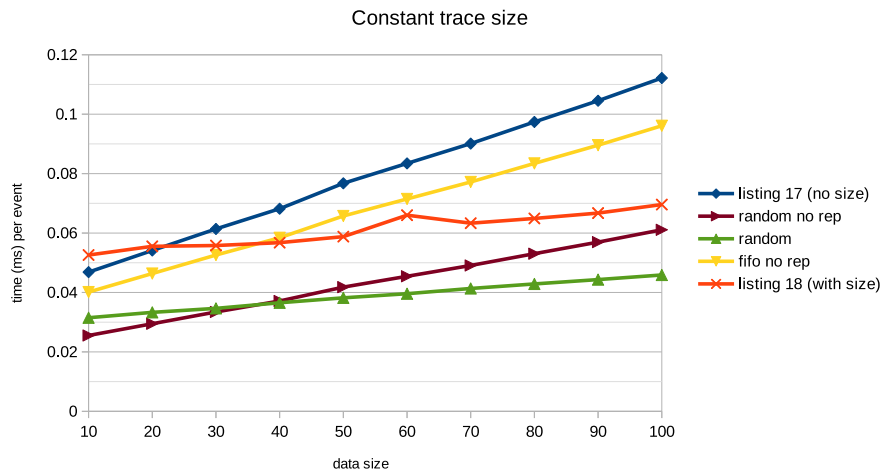


Fig. 25. Single queue, variable data size.

In Fig. 25 the charts²⁵ show that the time required for monitoring a single event linearly increases with the data size (between 10 and 100). The charts in Fig. 25 exhibit significant variation in their gradients; in particular, the gradient is

²⁵ The constant size of the generated traces is 40K events for Listing 17, 30K for Listing 18, 46K for Listing 24, 80K for Listing 25 and 73K for Listing 26.

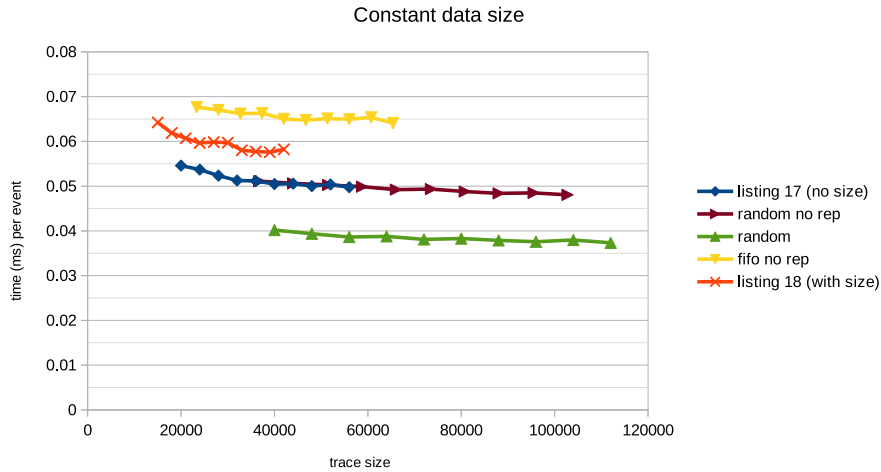


Fig. 26. Single queue, variable trace size.

```

1 let s=new Set()
2 s.add(1)
3 for(let v of s){console.log(v);s.delete(v);s.add(v)} // loops forever

1 # prints 1 if Python version < 3.6
2 # prints 1 42 42 42 42 if Python version >= 3.6
3 d={'x':1}
4 for k in d:
5     print(d[k])
6     del d[k]
7     d[k]=42

```

Fig. 27. Unsafe use of an iterator in JavaScript and Python.

lower for Listings 25 (random queue) and 18 (FIFO queue with size): in the former case, we conjecture this is due to the simplicity²⁶ of the specification, while in the latter a possible explanation is given by the fact that the time required by the generated monitor to check events of type `size` is independent from the size of the queue, differently from what happens for the other events of type `enq` and `deq`.

Fig. 26 exhibits that the total time required by the monitors for checking entire traces linearly increases with their size (between 10K and 110K events), when the data size is constant (50).

8.4. Safe iterators

Iterators are one of the most frequently used design patterns [27], to the degree that all mainstream object-oriented languages offer a native support for them. The interface of iterators is quite simple and its correct use can be easily specified in RML, as already shown in Listings 1 and 8; however, while it is easy to check iterators in isolation, it is more problematic to verify correct interaction with their containers, in case they are mutable objects. Indeed, differently from the previous examples presented in this section, which are focused on single interfaces, here we face the more challenging problem of verifying the correct interaction of objects implementing different interfaces.

The main problem with iterators and their mutable containers is given by the fact that in general it is not safe to modify a container while there are iterators active on it, because this could lead to unpredictable, counter-intuitive or undefined behavior; such an issue is more serious with multithreading, but it manifests also with single threading as shown in Fig. 27.

In the upper JavaScript code, a set is created with just one element, and iteration over it is performed with the specific shorthand syntax for managing iterators in JavaScript; at each iteration step, the current element of the set is first removed and then immediately re-inserted; in this case the, maybe counter-intuitive, outcome of the execution is an infinite loop, because in JavaScript elements of iterators are always processed in insertion order.

In the lower Python code, a similar example is shown based on dictionaries: depending on the version of the interpreter, a different behavior is obtained, definitely unpredictable in case of Python 3.6 or higher.

²⁶ For Listing 26 (random queue with no repetitions) the gradient slightly increases because the specification becomes a bit more complex to manage repetitions.

```

1 List<Integer> l = new LinkedList<>();
2 l.add(42);
3 Iterator<Integer> it = l.iterator();
4 int i = 0;
5 do {
6     if (i % 2 == 0) l.remove(0); // removes the first element of l
7     else l.add(i); // appends i to the end of l
8     i++;
9 } while (i != 0);
10 while (it.hasNext())
11     System.out.println(it.next()); // prints null

```

Fig. 28. Failure of a fail-fast Java iterator.

In the Java API several iterators are fail-fast: they throw an exception on a best-effort basis whenever a non-permissible modification is performed on their collections; for instance, structural modifications on lists (that is, operations which affect the size of the list) are not allowed when using iterators, while, for instance, the `set` operation, which just change the element at an existing index without modifying the length of the list, is permitted. While it is certainly not possible to guarantee failure with unsynchronized concurrent modifications, there are also single threading examples where fail-fast iterators fail to throw an exception, as shown in Fig. 28.

The example is based on the fact that the implementation of fail-fast iterators of `LinkedList` uses an integer modification counter, and, thus, neglects problems with overflow²⁷ in two's complement arithmetic. For this reason, the iteration at line 5 terminates with the list containing just the element -1, after having alternatively removed and inserted elements from and in the list; despite the list has been structurally modified, the statement at line 11 does not throw an exception but produces the undefined output `null`.

To avoid the problems shown above, we consider a more elaborate specification of iterators which, besides checking the correct use of their interface, also verifies the correct interaction with their containers, thus forbidding calls to methods on an iterator `it_id` whose container was subject to a modification unsafe for `it_id` (that is, a structural modification) after its creation.

```

// event types hasNext(it_id,res), next(it_id), newIter(cont_id,it_id), freeIter(it_id), unsafe(cont_id) to
// be defined

newIter(it_id) matches newIter(_,it_id);
hasNext_or_next(it_id) matches hasNext(it_id,_) | next(it_id);
iterator matches newIter(_) | hasNext_or_next(_) | freeIter(_);
not_newIter not matches newIter(_);
unsafe_or_iter(cont_id,it_id) matches
    hasNext_or_next(it_id) | freeIter(it_id) | unsafe(cont_id);

Main = Safe ^ iterator >> Iterators;

Safe = not_newIter*
    {let cont_id,it_id; newIter(cont_id,it_id) (SafeIter<cont_id,it_id> ^ Safe)}?;
SafeIter<cont_id,it_id> = unsafe_or_iter(cont_id,it_id) >> hasNext_or_next(it_id) * unsafe(cont_id) *
    freeIter(it_id) all;

Iterators = {let it_id; newIter(it_id) (Iterator<it_id> freeIter(it_id) | Iterators)}?;
Iterator<it_id> = ((hasNext(it_id,true) + next(it_id)) * hasNext(it_id,false) +) !;

```

Listing 27: Specification of multiple safe iterators over multiple containers.

The full specification, available in Listing 27, deals with the most general case of multiple safe iterators over multiple containers. It is based on the following basic event types, whose definition has been deliberately omitted and needs to be adapted to the different scenarios (mainly depending on the considered container and on the source programming language of the verified software):

- `hasNext(it_id,res)`, `next(it_id)` and `newIter(cont_id,it_id)` have the same semantics as in Listing 8;
- `freeIter(it_id)`: iterator `it_id` is no longer active;
- `unsafe(cont_id)`: a modification unsafe for iterators has been performed on the container `cont_id`; as an example, `unsafe(cont_id)` for Java lists should match method calls on `cont_id` for `add` or `remove`, but not `set` (non structural modification), `get` or `size` (no modification at all).

²⁷ Indeed, `Integer.MAX_VALUE+1==Integer.MIN_VALUE`.

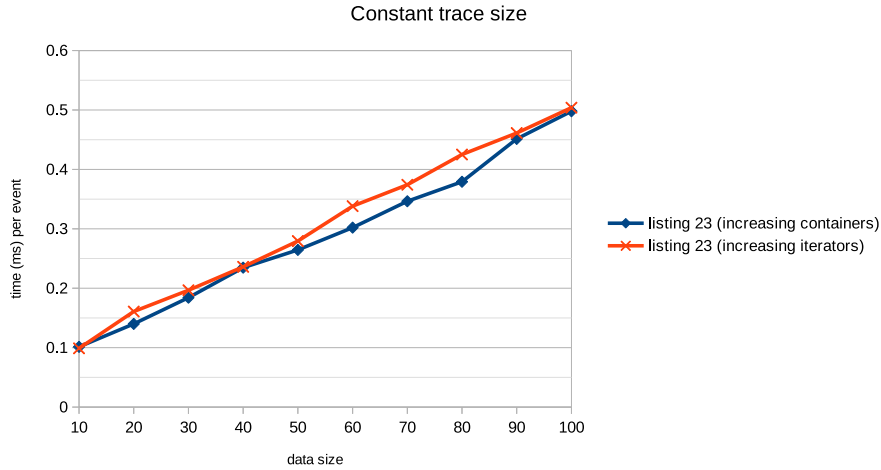


Fig. 29. Safe iterators, variable data size.

The main specification is obtained by compositional combination of two sub-specifications through the intersection operator; the former, *Safe*, verifies that containers and iterators interact safely, the latter *iterator* >> *Iterators* (restricted to operations of the iterator interface) checks that iterators are used correctly in isolation.

In *Safe*, checks must be performed as soon as a new iterator *it_id* is created over a container *cont_id* (event type pattern *newIter(cont_id, it_id)*); for the remaining events, no verification is required (*not_newIter**). The check is parametric on both *cont_id* and *it_id* (*SafeIter<cont_id, it_id>*): whenever a relevant operation on *cont_id* or *it_id* is performed (filter with event type pattern *unsafe_or_iter(cont_id, it_id)*), it must be verified that all operations on iterator *it_id* (*hasNext_or_next(it_id)**) must occur before any unsafe operation on *cont_id* (*unsafe(cont_id)**). Finally, no other check is required after *it_id* is no longer active (*freeIter(it_id) all*); checks on a specific iterator active over a specific container must be performed in conjunction with the checks for the other iterators active over some container, as specified in *Safe*, which is recursively combined with itself through the intersection operator. In this case the law stating that *all* is the identity of the intersection (see Section 3.4) is essential for optimization purposes.

In comparison with the specification of Listing 8, *Iterators* checks a weakest condition, allowing method *hasNext* to be called more times before *next*, and iterators to be deallocated before the end of the whole iteration; furthermore, event type *freeIter(it_id)* must be necessarily considered, since this is crucial for the *Safe* specification.

The Node.js script which generates the correct event traces used for the experiments repeats the following steps several times. A maximum number M_i of iterators are created per a maximum number M_c of containers (that is, a total of $M_i \cdot M_c$ iterators are created), then a fixed number of unsafe operations are performed on a container from which no iterator has been created; after such steps, all created iterators are iterated till completion and, then, a fixed number of unsafe operations are performed for all containers from which iterators are created. Finally, all created iterators are deallocated.

The chart in Fig. 29 reports the measurements for the monitor generated from the specification in Listing 27, with variable data size (between 10 and 100) and constant trace size (100K events); the data size consists of the maximum number of simultaneously active iterators, that is, $M_i \cdot M_c$. Two different measurements are considered: one where $M_i = 10$ and M_c ranges over the interval $[1..10]$, and the symmetric one where M_c is kept constant. For both cases the charts show a similar behavior: the time required to monitor a single event increases linearly with the maximum number of simultaneously active iterators (that is, $M_i \cdot M_c$).

The chart in Fig. 30 reports the measurements for the monitor generated from the specification in Listing 27, with variable trace size (between 40K and 110K events) and constant data size (500 with $M_i = 10$ and $M_c = 50$). The chart shows that the time required to monitor a single event does not depend on the length of the trace.

9. Related work

This section describes the main approaches to RV that have been proposed in the scientific literature. Even if the field is still relatively young (almost all contributions to RV have been published in the last 20 years), a considerable number of languages, tools, architectures and views have been proposed. A precise comparison between RML and every other tool would be both extremely hard and possibly not really useful, due to many profound differences: application domains, generality of the approach, and abstraction levels are only some of the identifying characteristics of RV tools.

The following sections will briefly describe the main techniques, their pros and cons, some state-of-the-art tools and their difference with respect to RML, comparing their design choices and their consequences. This is not meant to be a precise, complete classification of RV tools. This is a complex research task by itself; such a taxonomy has been recently proposed by Falcone et al. [30], and a graphical representation is given in Fig. 31. With respect to such taxonomy, we can identify RML as follows.

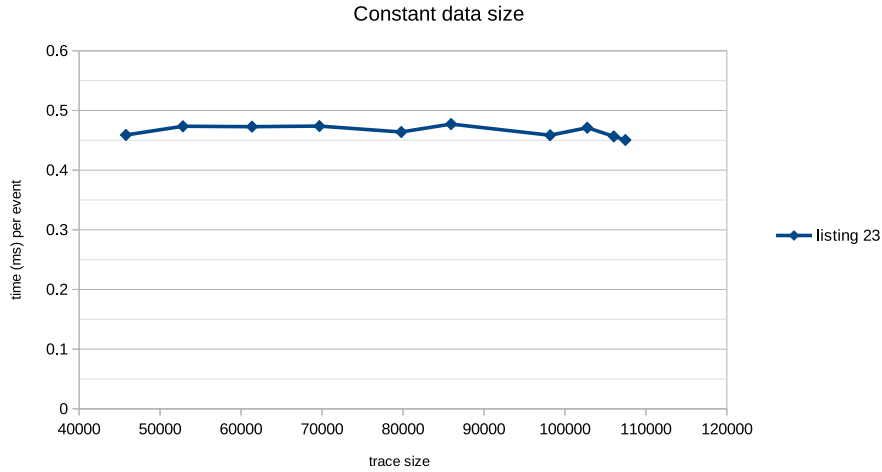


Fig. 30. Safe iterators, variable trace size.

Specification The *system model*, that is, the abstraction level at which the program is observed, is defined by the instrumentation; RML specifications are always *explicit*, as they are written by users and there are no implicit properties checked by the tool.

Monitor RML monitors employ an operational, rewriting-based *decision procedure* (the rewriting semantics); such monitors are *explicitly generated* by our compiler, and *directly executed* as SWI-Prolog code.

Deployment The current implementation of RML monitors can work both *offline* and *online*; regarding the latter, the monitor itself works in a synchronous way, though the instrumentation can handle the communication with the monitor also in an asynchronous manner. The monitor is placed *outline*, meaning it is a different program executed in a different address space; the instrumentation can be *inline*, as it happens with our Node.js instrumentation, which is an instance of a *software instrumentation* (as opposed to hardware ones). Finally, RML monitors are *centralized*, as properties globally describe the correct behavior of a system.

Reaction Our monitors are *passive*, since they only provide specification outputs (verdicts). However, such outcomes are delivered to the instrumentation layer, which may be either passive or active; the latter would affect the execution of the program under scrutiny to some extent.

Trace Traces [31] both have a *role* in the theoretical model (where they are possibly infinite sequences of events) and in the practical observation (where they are necessarily finite sequences); everything else about the kind of information encoded in events entirely depends on the instrumentation (the *evaluation* can either refer to points or intervals in time, though in all of our examples we considered events about single execution points).

Interference Again, this depends on the instrumentation. For our Node.js instrumentation, the interference is semantically *non-invasive*, as the code instrumentation step does not change the behavior of the program. Regarding performances, however, the overhead cannot be zero as reported in Section 8.

Application area RML is designed as an *analysis* technique, complementary to testing, static analysis and possibly other tools.

The distinction between monitors and instrumentation layers is not common in RV literature: indeed even an authoritative work as the taxonomy above freely mixes the two concepts. However, we believe the distinction is worth the additional effort for maximal flexibility.

Other general, comprehensive and authoritative surveys on RV have been done by Delgado et al. [32], Havelund and Goldberg [33], Leucker and Schallhart [1], Sokolsky et al. [34], Falcone et al. [11], Bartocci et al. [35].

9.1. Temporal logics

Since RV has its roots in model checking, it is not surprising that logic-based formalisms previously introduced in the context of the latter have been applied to the former.

Temporal logics (first studied by Prior [36]) are a family of modal logics where modalities are used to qualify propositions in terms of time, like “property ϕ will always/eventually hold” or “property ϕ (always) held”. Some temporal logics only refer to the past or to the future, while others allow for both.

Linear Temporal Logic (LTL) [37], is one of the most used formalism in verification. Its modalities refer to events in the future; the two fundamental operators are $X\phi$ (“ ϕ has to hold at the next state”) and $\phi U \psi$ (“ ϕ has to hold at least until ψ does, which must happen at the current or a future time”). Note that the standard semantics of LTL is defined on infinite traces only. Finally, “linear” comes from the fact that only one possible path is taken into account.



Fig. 31. “A Taxonomy for Classifying Runtime Verification Tools” [30].

To account for the incremental nature of RV monitors (as opposed to static formal verification), a three-valued semantics for LTL, named LTL_3 has been proposed [38]. Beyond the basic “true” and “false” truth values, a third “inconclusive” one is considered (LTL specification syntax is unchanged, only the semantics is modified to take into account the new value). This allows one to distinguish the satisfaction/violation of the desired property (“false”) from the lack of sufficient evidence among the events observed so far (“inconclusive”), making this semantics more suited to RV. Differently from LTL, the semantics of LTL_3 is defined on finite prefixes, making it more suitable for comparison with other RV formalisms. Further development of LTL_3 led to $RV-LTL$ [2], a 4-valued semantics on which RML monitor verdicts are based on (see Section 2.3.3).

The expressive power of LTL is the same as of star-free ω -regular languages [39]. When restricted to finite traces, RML is much more expressive than LTL as any regular expression can be trivially translated to it; however, on infinite traces, the comparison is slightly more intricate since RML and LTL_3 have incomparable expressiveness [40].

Since RML can describe non-regular languages, it can express some properties LTL cannot. On the other hand, the U operator has no precise counterpart in RML, as happens in general in RV where monitors can inspect only a finite prefix of the execution trace. A naive translation of $\phi U \psi$ would be $\phi^* \psi$, which is, however, incorrect, because it can also accept an infinite sequence of events matching ϕ , without ensuring that an event matching ψ will eventually occur. Considering the LTL formula $\phi U \psi$ is not monitorable with respect to the infinite sequence of events matching ϕ (because the verdict will be inconclusive for all finite prefixes [41]), this expressiveness feature is not useful for RV purposes.

There exist many extensions of LTL that deal with time in a more quantitative way (as opposed to the strictly qualitative approach of standard LTL) without increasing the expressive power, like *interval temporal logic* [42], *metric temporal logic* [43] and *timed LTL* [38]. Other proposals go beyond regularity [44] and even context-free languages [45].

Regardless of the formal expressiveness, RML and temporal logics are essentially different: RML has no direct support for time. However, if the instrumentation provides timestamps as part of the monitored events, then time constraints can be expressed by exploiting parametricity, generics and conditionals.

More insights on the use of temporal logics in RV can be found in the survey papers of Leucker and Schallhart [1] and Bartocci et al. [35].

LTl and other temporal logics have been embedded in a more expressive framework, namely *rechML* [46], which is a variant of the *modal μ -calculus* [47]. This allowed the formal study of monitorability [13] in a general framework, to derive results for free about any formalism that can be expressed in such calculi. It would be interesting to study whether the RML trace calculus could be derivable to get theoretical results that are missing from this presentation. Unfortunately, it is not clear whether our calculus and *rechML* are comparable at all. For instance, *rechML* is a fixed-point logic including both least and greatest fixpoint operators, while our calculus implicitly uses a greatest fixpoint semantics for recursion. On the other hand, *rechML* does not include a shuffle operator, and we are not aware of a way to derive it from other operators (indeed it is part of our core calculus).

9.2. Assertions

Assertions are predicates over program variables, used to ensure some expected property holds. Their use in verification has been introduced with *Hoare logic* [48], where imperative statements are associated with preconditions and postconditions that are expected and need to hold in order for the program to be proved correct. Today, assertions are a common programming tool that developers use to document, test and ensure expected properties in specific execution points. They can be supported either at the language level or as libraries.

Further development led to *design by contract*, first introduced in the context of the Eiffel²⁸ language, which embraces it [49]. In this methodology, software components explicitly state, at the code level, which preconditions are expected from client code, which postconditions are ensured after usage, and which invariants are continuously held true.²⁹ In an object-oriented setting, for instance, method signatures would be decorated with preconditions and postconditions, and classes would explicitly contain their invariants.

When they are not statically implemented (via typical formal methods techniques), assertion-based frameworks can be understood as an instance of RV, effectively monitoring and verifying the program at execution time. This approach has indeed been followed by some RV tools.

Java Modeling Language (JML) [50] is a verification tool specifically tailored for Java programs, where methods and classes are decorated with assertions, using first-order logic extended with arithmetic and boolean expressions, plus some other features. The *OpenJML*³⁰ implementation [51] supports runtime assertion checking. From an RV perspective, the tool is focused on data-oriented properties, as it allows one to assess the correctness of manipulated data in specific code locations. The implementation relies on Java bytecode³¹ instrumentation.

In order for RML to support runtime assertion checking of pre/post-conditions and invariants in object-oriented programming, the instrumentation layer needs to monitor at least all method calls (and returns), and possibly constructors (to first evaluate the invariants). Given such infrastructure, one can validate parameters and results of methods, as previously shown in the object-oriented examples in Section 6, by adding constraints in event type definitions. This idea suggests that simple assertions³² in the design-by-contract style could be automatically translated into RML event type definitions, while the main specification is used to verify control-oriented properties, by allowing only some valid sequences of method calls. Since in RML the specification is separated from the code, integration with assertion checking of pre/post-conditions would require appropriate event type definitions capturing the relevant variables from the program state (that is, from the JSON-serialized received event). This (arguably small) additional complexity seems unavoidable for language-independent tools, as properties are not connected to any particular code location. Examples in Section 6 on stacks and queues have shown that for some data types one can specify in RML data-oriented properties more abstractly at the interface level, independently of the specific implementation defined in the corresponding classes.

The notion of ghost variable in JML (and other tools) refers to variables used only in the specification but not in the observed program. While this notion makes sense when one needs to distinguish between program variables and specification variables, all RML variables can be understood as ghost ones. However, first-order logic based formalisms like JML also support universal quantification over ghost variables, though with restrictions to avoid undecidability. RML does not support universal quantification over new variables, that should be managed by explicit constraints in event type definitions, again with proper restrictions to allow tractability; however, this keeps RML matching implementation reasonably fast, since only pattern matching and basic expressions need to be supported.

On the other hand, with RML it is easy to monitor control-oriented properties and correct API usage [52,53]. Furthermore, if the instrumentation works at a finer granularity, RML can be used to verify properties when specific types of statements are executed. Essentially, the abstraction level entirely depends on the instrumentation layer.

Another object-oriented RV tool, SAGA³³ [54], targets Java programs with a different formalism based on attribute grammars. Context-free grammars are well suited for specifying protocols and control-oriented properties, while attributes are

²⁸ <https://www.eiffel.org/>.

²⁹ More precisely, invariants need to hold after construction and both before and after routine calls from outside the component. It is possible for them not to hold during the execution of code from the component itself.

³⁰ <http://www.openjml.org/>.

³¹ The bytecode is the target language of the Java compiler, and the instruction set of the Java Virtual Machine (JVM).

³² For instance, assertions that do not require first-order quantification or ghost variables; see more related comments below.

³³ <https://www.cwi.nl/innovation/software/saga-a-run-time-verifier-for-java-programs/saga-a-run-time-verifier-for-java-programs>.

used to also support data-oriented specifications. From such attributes, assertions to be checked at runtime are generated, and the code is instrumented using Rascal [55], a DSL devoted to source code manipulation.

The choice of context-free grammars as fundamental building blocks may look similar to what is done with RML recursive definitions; however, SAGA does not directly support the shuffle and intersection operators; intersection can be recovered by associating more specifications with the same class (or other entity), but it is not clear how an RML recursive specification involving the intersection operator could be translated into SAGA.

SAGA formalism requires to declare in the source code which events (method calls, for instance) need to be monitored. This resembles RML event types, which, however, are more general in nature and are parametric, while in SAGA data is accessed by mixing code with the specification.

9.3. State machines

High-level (runtime) verification languages are often translated to low-level, more directly executable ones. This is the approach of RML as well, where specifications are compiled down to the trace calculus terms, for which a rewriting semantics is defined, that in turn drives the implementation. This step is sometimes called monitor synthesis. While this allows one to write down the specification at a higher abstraction level, it clearly needs more work and tools to be developed (and optimized).

A different approach consists in formalizing the specification using *state machines* (a.k.a. automata or finite-state machines). Though the core concept of a finite set of states and a (possibly input-driven) transition function between them is always there, in the field of automata theory different formalizations and extensions bring the expressiveness anywhere from simple deterministic finite automata to Turing machines. From an RV perspective, they can be understood as executable specifications (this is also true for rewriting based approaches like RML).

An example of such formalisms is *DATE* (Dynamic Automata with Timers and Events [56]), an extension of the finite-state automata computational model based on communicating automata with timers and transitions triggered by observed events. This is the basis of *LARVA*³⁴ [57], a Java RV tool focused on control-flow and real-time properties, exploiting the expressiveness of the underlying system (DATE). The tool also supports other logics, like *QDDC* (Quantified Discrete-time Duration Calculus [58]), *LUSTRE* [59], and *counter-example traces* [60]; still, all the supported logics are internally translated to it.

The main feature of LARVA that is missing in RML is the support for temporized properties, as observed events can trigger timers for other expected events. Currently the only way RML can monitor real-time properties is with the support of an instrumentation adding timestamps to serialized events; then, by adding arithmetic constraints in event type definitions, it is possible to verify time constraints, although with some limitation: monitors could emit a negative verdict only after the expected event (or another one) is actually observed, and not as soon as the time-out has expired. In the worst case, this could postpone the final verdict indefinitely, unless the instrumentation layer could send clock-based time-out events to the monitor, but this would lead to efficiency and synchronization problems, especially in distributed systems.

On the other hand, the parametric verification support of RML is more general. LARVA scope mechanism works at the object level, thus parametricity is based on *trace slicing* and implemented by spawning new monitors and associating them with different objects. The RML approach is different as specifications are always global, and they can be parametric with respect to any observed data: this makes it easier to write down parametric specifications that are still related to the whole system, or to a family of otherwise unrelated (and possibly dynamically discovered) objects. Limitations of the parametric trace slicing approach described above, as well as possible generalizations to overcome them, have been explored by Chen and Rosu [61], Barringer et al. [62], Reger et al. [63]. In RML we decided not to reason on trace slicing but rather to keep just one (global) specification and syntactically instantiate parameters as soon as they are discovered, a task simplified by our design choice of having variable declarations together with a notion of scope for them.

Finally, the goals of the two tools are very different to start with. While RML strives to be system-independent, LARVA is devoted to Java verification, and the implementation relies on AspectJ³⁵ [64] as an “instrumentation” layer, which allows one to inject code (the monitor) to be executed at specific locations in the program under scrutiny. Aspect-oriented programming [65] is a paradigm that allows the developer to add new behavior to existing code without modifying it, enhancing modularity and separation of concerns; AspectJ brings these concepts to Java.

9.4. Monitor-oriented programming

Similarly as RML, which does not depend on the monitored system (not even on its instrumentation), other proposals exist in the literature that introduce different levels of separation of concerns. *Monitor-oriented programming*³⁶ (MOP [66]) is an infrastructure for RV that is neither tied to any particular programming language nor to a single specification language; indeed, it has been instantiated on a number of formalisms. In order to add support for new logics, one has to develop an

³⁴ <http://www.cs.um.edu.mt/svrg/Tools/LARVA/>.

³⁵ <https://www.eclipse.org/aspectj/>.

³⁶ http://fsl.cs.illinois.edu/index.php/Monitoring-Oriented_Programming.

appropriate plug-in converting specifications to one of the format supported by the MOP instance of the language of choice (see, for instance, [67]), so that the infrastructure can synthesize an appropriate monitor.

MOP can also be understood as a software development methodology where the program to be verified is actually aware of the monitoring process, and the two entities can interact with each other. This approach has also been suggested in the context of *runtime reflection* [68]. This is fundamentally different from RML, where the monitor basically sits on top of the existing program and is passive in nature.

*JavaMOP*³⁷ [69] is an RV framework for Java programs which is based on an extension of AspectJ, and follows the MOP approach; through AspectJ aspects it is possible to mix property specifications with instrumentation code, hence separation between specification and instrumentation is not supported; this is reflected also in the way parametricity is supported: as in other systems, no real abstraction is provided at the specification level to define the scope of data value variables, but traces are automatically sliced according to parameter instances, and each slice is checked by a separate monitor, associated with a specification. As a consequence, the user has no control on data value variables, and this limitation can only be solved by mixing formal specifications with programming by performing checks in the instrumentation.

JavaMOP is not system agnostic since only Java applications can be monitored, but it supports several formalisms for specifications, including extended regular expressions, context-free grammars, finite state machines, and different kinds of temporal logics; however none of them allow definitions of non context-free properties.

State machine systems are sometimes employed in mixed system together with other types of formalism. *TraceContract*³⁸ [70], for instance, offers a mix of finite state machines and temporal logic features. It is implemented and available as a Scala internal DSL.

As expressive as finite state machines, regular expressions have been used in RV for their simple semantics and their well-understood meaning among software developers. *Tracematches*³⁹ [71] uses regular expressions augmented with variables to get parametricity. The tool is implemented in AspectJ and is devoted to Java programs verification, especially regarding correct usage of APIs. This is similar to what we proposed [53,52] using the precursor of RML, namely parametric trace expressions [8]. Indeed, quite some common API usage patterns can be formalized using regular languages, though more operators (like shuffle and intersection) allow more flexibility and expressive power, as shown in Section 6.

9.5. Rule systems

The core idea behind rule-based verification systems is to encode properties in a set of “rules”, with rules being composed of conditions that need to hold in order to apply the rule, events that can trigger the application of the rule, and statements about what holds after such transition. Different formalisms extend this basic concept in different directions.

RuleR [72] approach is based on dynamic activation and deactivation of rules: observed events can trigger rules that in turn can change the set of applicable rules in subsequent steps. RuleR has been proposed as a simpler, more essential and easier alternative to Eagle [73], a highly expressive first-order logic with fixpoint and linear temporal operators, in order to get a more effective RV tool. For this reason we focus on the successor.

Just like in RML, RuleR supports data parameterization, so that specifications can depend on observed data from the event. Furthermore, it also includes a different form of parametric specifications, namely, rules parameterized over other rules. This form of higher-order specifications are currently not supported in RML; extending the trace calculus in such a way should be done with care to guarantee decidability of a single transition step.

PollyRV (also named *polyRV* and *polyLarva*⁴⁰ [74,75]) uses a guarded-command style specification language. Properties are expressed as a list of event-condition-action rules. Whenever an event (possibly having parameters) is generated by the system, the list of monitor rules is scanned for rule matches relating to that event. If a match is found, the expression specified in the condition of the rule is evaluated and, if satisfied, the action is triggered.

PollyRV supports the monitoring of real-time specifications. It is system side language agnostic and provides a configurable system-monitor boundary using explicit tagging of monitoring code locality (system or monitor).

Comparing rule-based system to RML is not easy from the usage point of view, as the underlying concepts are very different. Rule systems are arguably lower-level formalisms: the state is explicitly manipulated and one must reason mostly on single event steps. This often leads to longer specifications, as also acknowledged by the authors [76]. On the other hand, RML offers a higher abstraction level, and language operators work at the trace level.

Formal expressive power is conjectured to be the same; both RML and RuleR can express non context-free properties. Furthermore, they both have counting ability over natural numbers, which suggests Turing-completeness when generics (in the sense of RML) are employed. PollyRV seems to be less expressive; its syntax, as described in [77], does not seem to support operators for regular expressions as the Kleene star. However, monitors can generate new sub-monitors at runtime and this opens up many possibilities to extend the computational power of the formalism.

Rule systems are connected to rule based artificial intelligence models, as they share the underlying deductive approach, based on deriving new knowledge from known facts. *LogFire* [76], for instance, uses an algorithm from the artificial in-

³⁷ <http://fsl.cs.illinois.edu/index.php/javaMOP4>.

³⁸ <https://github.com/havelund/tracecontract>.

³⁹ <https://patricklam.ca/research/tracematch/>.

⁴⁰ <http://www.cs.um.edu.mt/svrg/Tools/PollyRV/>.

telligence community called *Rete* [78]. By using different inference engines and more efficient ways to store information about the observed data, the implementation of such systems can be greatly optimized, also becoming suitable for online RV purposes.

As it happens with other approaches, these systems can either be implemented on their own or embedded in existing languages as libraries. For instance, RuleR is implemented in Java, while LogFire is provided as an internal Scala⁴¹ DSL, a task for which Scala is very well suited thanks to its set of features. The same implementation choice (Scala internal DSL) has been made for TraceContract [70], which also supports some basic rule-oriented features.

9.6. Static and dynamic analysis

Effective and efficient combination of static and dynamic analysis techniques is a currently investigated research problem, which poses many challenges and is highly dependent on the formalism used and the kind of system under observation. Despite the challenges, it is a natural step towards software correctness: every property that is too impractical (or impossible) to enforce statically, is postponed to dynamic analysis, ideally getting the best of both worlds. In this view, RV serves as a complementary technique to more traditional ones: static analysis, formal methods, type systems, etc.

PQL⁴² (*Program Query Language* [79]) is a tool that allows the user to define patterns of execution to be matched against the flow of the observed program. Statically, the analyzer finds the relevant code points that need to be instrumented, thus minimizing the runtime overhead; this optimization is not uncommon in RV tools; a similar analysis has also been developed for Tracematches [80]. At runtime, a monitor generated from the property (as a finite state machine) matches the sequence of observed events against the specification. PQL is mainly concerned with security properties and resource leaks.

Regarding its expressiveness, basic queries can be compared to context-free grammars. However, queries are patterns that can be instantiated by an arbitrary number of matching existing objects. While this form of parametricity increases the expressiveness (intersection of context-free grammars, according to [79]), it seems to share similar limitations to those of the trace slicing approach previously discussed. In this sense, PQL properties are more local, as opposed to the global approach of RML.

Another example of state-of-the-art combination of static and dynamic analysis framework is *Unified Static and Runtime Verification of Object-Oriented Software*⁴³ (*StaRVOORs* [81]). In this case the combination has the goal of reducing the original specification by simplifying what can be proved statically, and produce a runtime specification for what needs dynamic analysis. Thus, rather than optimizing the instrumentation, here the specification itself is “optimized”.

The framework combines the static deductive verifier KeY⁴⁴ [82] with the RV tool LARVA [57]. The formalism of choice for the specification is *ppDATE* [83], an extension of DATE [56] that allows both control- and data-oriented properties to be stated, decorating automaton states with Hoare triples. From this, JML [50] annotations are produced, and the deductive verifier uses both to prove correctness, or if it is not possible, only partial proofs. In the latter case, the specification is refined to encode what needs to be checked at runtime, compiled to DATE and used at runtime with LARVA [57].

Our proposal goes towards the direction of integrating static and runtime verification by using the same formalism, trace expressions, for representing properties to be checked both statically and at runtime. The goal is to recognize “wrong assumptions” made by the designers of the model to be statically checked, during the RV stage [84,85]. In fact, when applying formal verification to a system that interacts with the real world, we must use a model of the environment. This model represents an abstraction of the actual environment, but is necessarily incomplete and hence presents an issue for system verification. If the actual environment matches the model, then the verification is correct; however, if the environment falls outside the abstraction captured by the model, then we cannot guarantee that the system is well-behaved. A solution to this problem consists in exploiting the model of the environment for statically verifying the system’s behavior and, if the verification succeeds, using it also for validating the model against the real environment via runtime verification. This requires to use the very same formalism for both stages.

We demonstrated the feasibility of this approach by presenting its implementation on top of a framework integrating the Agent Java PathFinder model checker [86]. Trace expressions are used to model the environment for both static formal verification and runtime verification. The generalization from trace expressions to RML is not an issue, given the one-to-one correspondence between the two formalisms and our familiarity with moving from one to the other, and is indeed under way.

9.7. Process calculi

Process calculi (a.k.a. process algebras) are a formalism family developed to precisely describe the behavior of concurrent systems. The main entities are processes (single computing units) that can both work independently and communicate (and synchronize) with each other. Different choices of operators and different restrictions about dynamic creation of processes

⁴¹ <https://www.scala-lang.org/>.

⁴² <http://pql.sourceforge.net/>.

⁴³ <https://starvoors.github.io/>.

⁴⁴ <https://www.key-project.org/>.

Table 1

RV tools comparison.

	System agnostic	Strong sep. instr. & spec.	Full Parametricity	Non CF-prop.	Formalization	Error Recovery	External DSL	Generics	Implementation technologies
RML	✓	✓	✓	✓	full		✓	✓	Java, ANTLR, Kotlin, Prolog
JavaMOP				✓		✓			Java, AspectJ
RuleR	✓		✓	✓	basis		✓		Java (proof of concept)
PollyRV	✓	✓	✓			✓	✓		Java, AspectJ
TraceContract	✓		✓	✓					Scala
PQL				✓		✓	✓		Java, Python, Datalog
JASSDA							✓		Java
Larva					basis	✓	✓		Java, AspectJ
SAGA			✓	✓					Java, ANTLR, Rascal

and communication channels led to a number of systems. Notable examples include *Communicating Sequential Processes* (CSP [87]), the *Calculus of Communicating Systems* (CCS [88]) and the π -calculus [89].

Despite the different goals and techniques involved, some RV formalisms and tools have been inspired by process calculi, since both the communities try to model software behavior.

*Java with Assertions Debugging Architecture*⁴⁵ (Jassda [90]) uses a CSP-like syntax to express properties, enriched with a notation to deal with set of events as a whole. This resembles the event types of RML which can be understood, in a sense, as set of events (those matching the event type pattern). As it happens with many other tools in object-oriented RV, parametricity is limited to trace slicing based on instances.

Though process calculi are quite expressive and can model complex interactions, they miss some operators that are useful in verification. Most importantly, intersection is usually not included since it makes little sense in that context. When specifying expected properties however, intersection can be crucial: it can both increase expressiveness (context-free languages are not closed under intersection, for instance) and support modular specifications, as shown in several RML examples in Section 6.

9.8. Stream RV

Beyond ensuring correctness of the observed execution of the program under scrutiny, it is sometimes useful to also collect quantitative, statistical data for analysis. In this case one needs to treat a sequence of events as a single *stream*, and *stream languages* that define dependencies between streams and compute the output stream in an incremental way are needed.

The concept was pioneered by *Lola*⁴⁶ [91], a monitoring language allowing users to manipulate streams of events and create new ones by specifying them with equations. The expressiveness is higher than temporal logics and finite state machines, and some context-free properties can be encoded. Furthermore, in *Lola 2.0* [92] parametricity support is added.

While initially developed in the context of synchronous systems monitoring (a system clock is assumed), stream RV has been generalized to deal with asynchronous systems. *TeSSLa*⁴⁷ [93,94] is a temporal stream specification language supporting events with timestamps, to monitor asynchronous, cyber-physical, real-time systems (a global order over events is still assumed).

A lower level formalism named *data transducers* [23] has been recently proposed, with the goal of defining a machine language for stream computations.

The goals of ensuring correctness and computing metrics are quite different, and thus the features are not immediately comparable. However, even though RML scope is traditional RV, generics and data expressions allow one to compute some values along the way (though no history of previous events is directly supported). Thanks to parametric event types, such computations can be easily computed from the observed data.

Transducers have also been used in runtime enforcement to anticipate incorrect behavior and allow acting on the system under scrutiny in a timely manner to avoid errors [95].

9.9. RV tools comparison

In this section we compare RML with eight tools based on the approaches surveyed in the previous sections, namely JavaMOP [69], RuleR [72], PollyRV [74,75], TraceContract [70], Program Query Language (PQL) [79], JASSDA [90], LARVA [57], and SAGA [54].

Below we briefly summarize their main features according to some characteristics relevant for an RV tool, namely system agnosticism, strong separation between instrumentation and specification, full parametricity, ability to express non

⁴⁵ <http://jassda.sourceforge.net/>.

⁴⁶ <https://www.react.uni-saarland.de/tools/lola/>.

⁴⁷ <https://www.tessla.io/>.

context-free properties, formalization of the semantics, error recovery capabilities, external DSL use, generic definitions, and implementation technologies. We say that a tool is based on an external DSL if the underlying specification language is a DSL external to the implementing programming language; this is in contrast with tools based on a real programming language or an internal DSL [70] (that is, embedded in the programming language they are developed in) which require their users to have more advanced programming skills.

JavaMOP, implemented using Java and AspectJ, follows the MOP approach; it does not separate specification and instrumentation and is not system agnostic since only Java applications can be monitored. It supports several formalisms for specifications which are less expressive than context-free grammars; however, non context-free properties can be monitored by directly using Java.

RuleR is based on an external DSL and is system agnostic and language independent, but it is not based on a universally recognized data format for events to support strong separation between specification and instrumentation. It allows specification of non context-free properties, its propositional trace semantics has been formalized, but to our knowledge no usable tool is currently available, although the monitoring algorithm at the basis of the core of RuleR has been implemented as a Java proof of concept.

PollyRV is system agnostic and keeps instrumentation and specification separated. A PollyRV runtime monitoring framework is generated in a two-step sequence. In the first step the PollyRV monitor is created, as a Java program, from the high-level specification written in an external DSL. The second step then handles the generation of aspect code which is to be weaved into the target system. PollyRV supports error recovery capabilities and parametricity, but not generics, and the expressive power of the formalism seems to be that of context-free languages.

TraceContract [70] is a Scala API which is classified as internal DSL. The API is system agnostic, but requires users to be acquainted with a complex language such as Scala. TraceContract allows users to write fully parametric specifications based on state machines. No support for generic specifications and strong separation between instrumentation and specification could be found. One of the main drawbacks of TraceContract is its higher learning curve that could prevent its adoption by users preferring simpler DSLs with more limited potential for introduction of programming errors.

PQL is an expressive external DSL supporting RV of open-source Java applications with code execution triggered upon violation; it allows specifications of properties covering the closure of context-free languages combined with intersection; however, it does not support shuffle, parametricity and generics. PQL is implemented as a jar file and executed as a Python script. Static analysis is performed by translating PQL queries into Datalog [96].

JASSDA is an RV framework for Java programs based on CSP-like specifications and implemented in Java. JASSDA is very simple but does not support concatenation and parametricity is obtained through slicing.

LARVA has been expressly designed for checking real-time properties of Java programs. Properties are specified with dynamic automata with timers and events (DATEs, their basic semantics is available [97]); in particular, it supports symbolic states to guard transitions, replication of automata, and CCS-like communication between automata. Code instrumentation is based on AspectJ.

SAGA is a framework for RV of Java programs based on attribute grammars extended with assertions; it supports parametricity and allows users to mix specifications with code instrumentation, hence, it does not keep specification and instrumentation separated, but it allows the specification of non context-free properties by exploiting the full computational power of Java. Its implementation is based on Java, ANTLR and Rascal.

A summary of the comparisons above is shown in Table 1. Except for RML, none of the considered approaches supports generic specifications and comes with a complete formal semantics, although for some other tools a basis of formal semantics is provided. W.r.t this last point, the rigor of the full RML formalization represents a powerful means to guarantee that “runtime-monitors do indeed monitor the desired specification” [98]. The strong separation between specification and instrumentation is supported by some tools, but without a universally recognized data format for events; few of them are system agnostic, although in some cases this is hampered by the lack of separation with the instrumentation. As far as implementation is concerned, the RML architecture exploits a coherent combination of sensible technologies; few tools are as sophisticated as ours.

While the survey by Falcone et al. [30] explicitly lists, for each of the 20 tools under review, both the language used for specifying properties and the targeted SUS (mainly applications in C and in Java), no information on the languages and technologies used for implementing the RV tool itself is provided. We associated these features with the eight tools we compared with RML, and the information emerging from them is that Java is the most widely used language, often in connection with AspectJ.

We are aware of few verification tools that exploit Prolog, and more in general computational logics, as RML does. As discussed by Calegari et al. [99], most of them, like MCAPL [100], MCK [101], MCMAS [102], SCIFF [103] and SALMA [104], come from the multi-agent systems domain and aim at their static verification.

10. Conclusion

We have presented RML, an RV DSL with the following main features:

- It is system agnostic and keeps the specification and the instrumentation levels strongly separate, through the universally recognized data format JSON to represent events, thus favoring reuse of specifications and interoperability between the generated monitors and the components under scrutiny.
- It is highly expressive, supporting parametric specifications, non-context free properties, and concise specifications of non-trivial properties.
- It supports generic specifications to enhance expressivity and specification reuse.
- It is simple and equipped with a formal semantics based on a trace calculus and its labeled transition system; such a calculus drives also the implementation of the calculus. The language simplicity is obtained thanks to a sophisticated architecture, where many different technologies are seamlessly integrated together. Among them, computational logic (SWI-Prolog in particular) allowed us to implement the calculus in an efficient, easy to inspect, and semantics-compliant way.
- It has been used in real Node.js and Node-RED applications [105] and to specify properties within the ROSMonitoring framework which supports RV of robotic applications developed using the Robot Operating System, ROS. ROSMonitoring has been applied to a case study based on a simulation of the Mars Curiosity Rover [106].

Before RML had been designed and implemented, trace expressions – previously named (Attribute) Global Types –, upon which RML builds, had already been used to specify properties for monitors built on top of both JADE [10] and Jason [107] multi-agent systems. We used them for specifying the FYPA (Find Your Path, Agent!) interaction protocol driving a JADE multi-agent system used by the Ansaldo STS company for allocating platforms and tracks to trains inside Italian stations [108], to distribute the RV process over many different monitors [109–112], to verify fail-uncontrolled and ambient intelligence systems [113]. Trace expressions were used to boost agents' self-adaptation capabilities [114], also in the health domain [115].

There are several directions for future work to enhance the language.

Despite real RML applications exist, more experiments are needed to effectively assess the usability of the language; in particular, we are planning to employ RML for supporting RV of Internet of Things, and to study how the language can support RV of real-time properties.

For what concerns the implementation, we are planning to work on the implementation of RML to make the language more usable: there are several static checks that can be implemented to allow the user to detect bugs in the specifications concerning, for instance, the correct use of variables and specification names.

More theoretical aspects concern the expressivity of the language, whose limits are still unknown; a first step towards this research line would consist in formally comparing the expressive power of RML with some of the RV DSL presented in Section 9.

CRediT authorship contribution statement

Davide Ancona: Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing. **Luca Franceschini:** Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing. **Angelo Ferrando:** Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing. **Viviana Mascardi:** Conceptualization, Methodology, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Formal proofs

This section contains the formal proofs of all results stated in the previous sections.

A.1. Derived operations

Standard postfix operators.

$$t? = \epsilon \vee t$$

Proof. By rules (?), (e-?), (or-r), (n- ϵ), (e- ϵ) and (e-or-l) we can deduce that $E(t?)$ and $E(\epsilon \vee t)$ are both derivable, and if $t \xrightarrow{e} t'; \sigma$ is derivable, then both $t? \xrightarrow{e} t'; \sigma$ and $\epsilon \vee t \xrightarrow{e} t'; \sigma$ are derivable as well; furthermore, since there are no other applicable rewriting rules for $t?$ and $\epsilon \vee t$, then both $t? \xrightarrow{e} t'; \sigma$ and $\epsilon \vee t \xrightarrow{e} t'; \sigma$ imply $t \xrightarrow{e} t'; \sigma$. From these facts and Definition 3.1 we have:

- $\epsilon \in \llbracket t? \rrbracket$ and $\epsilon \in \llbracket \epsilon \vee t \rrbracket$;

- $e\bar{e} \in \llbracket t? \rrbracket$ iff $t? \xrightarrow{e} t'; \sigma$ is derivable and $\bar{e} \in \llbracket \sigma t' \rrbracket$ iff $t \xrightarrow{e} t'; \sigma$ is derivable and $\bar{e} \in \llbracket \sigma t' \rrbracket$ iff $\epsilon \vee t \xrightarrow{e} t'; \sigma$ is derivable and $\bar{e} \in \llbracket \sigma t' \rrbracket$ iff $e\bar{e} \in \llbracket \epsilon \vee t \rrbracket$.

Therefore we can conclude $\llbracket t? \rrbracket = \llbracket \epsilon \vee t \rrbracket$. $\square$

For the $+$ and $*$ operators we prove the following stronger result:

$$t_0 \cdot t_* \equiv t_0 \cdot t_* \quad (\text{A.1})$$

where t_* denotes the unique term t' s.t. $t' = \epsilon \vee (t \cdot t')$.

Proof. By rules (e-al), (e-*), (e-or-l) and (e- ϵ) we can deduce that if $E(t_0)$ is derivable, then $E(t_0 \cdot t_*)$ and $E(t_0 \cdot t_*)$ are also derivable, and the opposite implication holds as well, since there are no other applicable rules.

By rule (cat-l), if $t_0 \xrightarrow{e} t'_0; \sigma$ is derivable, then $t_0 \cdot t_* \xrightarrow{e} t'_0 \cdot t_*; \sigma$ and $t_0 \cdot t_* \xrightarrow{e} t'_0 \cdot t_*; \sigma$ are derivable; by rules (cat-r), (*), (or-r), (n- ϵ) and (cat-l), if $t_0 \not\xrightarrow{e} t'_0; \sigma$, $E(t_0)$ and $t \xrightarrow{e} t'; \sigma$ are derivable, then $t_0 \cdot t_* \xrightarrow{e} t' \cdot t_*; \sigma$ and $t_0 \cdot t_* \xrightarrow{e} t' \cdot t_*; \sigma$ are derivable.

Because it is not possible to build a finite derivation for $t \cdot t_* \xrightarrow{e} t''; \sigma$ by applying only rule (cat-r), even under the hypothesis that $t \not\xrightarrow{e} t''; \sigma$ and $E(t)$ are both derivable, we can conclude that if $t \cdot t_* \xrightarrow{e} t''; \sigma$ is derivable, then rule (cat-l) has to be necessarily used to obtain a finite derivation, $t \xrightarrow{e} t'; \sigma$ is derivable and $t'' = t' \cdot t_*$; therefore, if $t_0 \cdot t_* \xrightarrow{e} t''; \sigma$ is derivable, then either $t_0 \xrightarrow{e} t'_0; \sigma$ is derivable and $t'' = t'_0 \cdot t_*$, or $t_0 \not\xrightarrow{e} t'_0; \sigma$, $E(t_0)$ and $t \xrightarrow{e} t'; \sigma$ are derivable and $t'' = t' \cdot t_*$.

Furthermore, since there is only one rule for t_* , we have that if $t_* \xrightarrow{e} t''; \sigma$ is derivable, then $t \xrightarrow{e} t'; \sigma$ is derivable and $t'' = t' \cdot t_*$; therefore, if $t_0 \cdot t_* \xrightarrow{e} t''; \sigma$ is derivable, then either $t_0 \xrightarrow{e} t'_0; \sigma$ is derivable and $t'' = t'_0 \cdot t_*$, or $t_0 \not\xrightarrow{e} t'_0; \sigma$, $E(t_0)$ and $t \xrightarrow{e} t'; \sigma$ are derivable and $t'' = t' \cdot t_*$.

From these facts and Definition 3.1 we have:

- $\epsilon \in \llbracket t_0 \cdot t_* \rrbracket$ iff $E(t_0 \cdot t_*)$ derivable iff $E(t_0)$ derivable iff $E(t_0 \cdot t_*)$ derivable iff $\epsilon \in \llbracket t_0 \cdot t_* \rrbracket$;
- $e\bar{e} \in \llbracket t_0 \cdot t_* \rrbracket$ iff $t_0 \cdot t_* \xrightarrow{e} t''; \sigma$ and $\bar{e} \in \llbracket \sigma t' \rrbracket$ iff $t_0 \xrightarrow{e} t'_0; \sigma$ derivable, $t'' = t'_0 \cdot t_*$ and $\bar{e} \in \llbracket \sigma t'_0 \cdot \sigma t_* \rrbracket$ or $t_0 \not\xrightarrow{e} t'_0; \sigma$, $E(t_0)$ and $t \xrightarrow{e} t'; \sigma$ derivable, $t'' = t' \cdot t_*$ and $\bar{e} \in \llbracket \sigma t' \cdot \sigma t_* \rrbracket$ iff $t_0 \cdot t_* \xrightarrow{e} t'_0 \cdot t_*; \sigma$ and $\bar{e} \in \llbracket \sigma t'_0 \cdot \sigma t_* \rrbracket$ or $t_0 \cdot t_* \xrightarrow{e} t' \cdot t_*; \sigma$ and $\bar{e} \in \llbracket \sigma t' \cdot \sigma t_* \rrbracket$ (by coinductive hypothesis and def. of substitution application) iff $e\bar{e} \in \llbracket t_0 \cdot t_* \rrbracket$

Therefore we can conclude $\llbracket t_0 \cdot t_* \rrbracket = \llbracket t_0 \cdot t_* \rrbracket$. $\square$

By (A.1) and the laws $t \equiv \epsilon \cdot t$ and $t+ \equiv t \cdot t_*$, proved in Appendix A.2, we deduce

$$\begin{aligned} t_* &\equiv \epsilon \cdot t_* \equiv \epsilon \cdot t_* \equiv t_* \\ t+ &\equiv t \cdot t_* \equiv t \cdot t_* \end{aligned}$$

Conditional filter.

$$\theta \gg t_1 : t_2 \equiv ((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2)$$

Proof. By rules (e-filter), (e-*), (e-al) we have that if $E(t_1)$ and $E(t_2)$ are derivable, then $E(\theta \gg t_1 : t_2)$ and $E(((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2))$ are derivable, and the opposite implication holds as well, since there are no other applicable rules.

If $\text{match}(e, \theta) = \sigma$ and $t_1 \xrightarrow{e} t'_1; \sigma'$ is derivable, then by rules (filter-t), (shuffle-l), (and), (*) and (pre) $\theta \gg t_1 : t_2 \xrightarrow{e} \theta \gg t'_1 : t_2; \sigma''$ and $((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2) \xrightarrow{e} ((\epsilon \cdot (\theta : \epsilon) *) \wedge t'_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2); \sigma''$ are derivable with $\sigma'' = \sigma \cup \sigma'$.

If $\text{match}(e, \theta)$ is undefined and $t_2 \xrightarrow{e} t'_2; \sigma'$ is derivable, then by the assumption on match , $\text{match}(e, \bar{\theta}) = \emptyset$ and by rules (filter-f), (shuffle-r), (n-and-l), (n-*), (n-pre), (and), (*) and (pre) $\theta \gg t_1 : t_2 \xrightarrow{e} \theta \gg t_1 : t'_2; \sigma'$ and $((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2) \xrightarrow{e} ((\theta : \epsilon) * \wedge t_1) \mid ((\epsilon \cdot (\bar{\theta} : \epsilon) *) \wedge t'_2); \sigma'$.

If $\text{match}(e, \theta) = \sigma$ and $\theta \gg t_1 : t_2 \xrightarrow{e} t'; \sigma''$ is derivable, then only rule (filter-t) is applicable, therefore $t_1 \xrightarrow{e} t'_1; \sigma'$ is derivable, $t' = \theta \gg t'_1 : t_2$ and $\sigma'' = \sigma \cup \sigma'$.

If $\text{match}(e, \theta) = \sigma$ and $((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2) \xrightarrow{e} t'; \sigma''$ is derivable, then by the assumption on match , $\text{match}(e, \bar{\theta})$ is undefined, therefore no transition is derivable for $\bar{\theta} : \epsilon$ and, hence, for $((\bar{\theta} : \epsilon) * \wedge t_2)$, since there is only one rule for the $*$ and $\wedge$ operators; this means that rule (shuffle-l) has been necessarily applied, and that there exists a transition derivable for $((\theta : \epsilon) * \wedge t_1)$ and, by rule (and) $t_1 \xrightarrow{e} t'_1; \sigma'$ is derivable and by rules (shuffle-l), (and), (*) and (pre) $t' = ((\epsilon \cdot (\theta : \epsilon) *) \wedge t'_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2)$ and $\sigma'' = \sigma \cup \sigma'$.

If $\text{match}(e, \theta)$ is undefined and $\theta \gg t_1 : t_2 \xrightarrow{e} t'; \sigma'$ is derivable, then only rule (filter-f) is applicable, therefore $t_2 \xrightarrow{e} t'_2; \sigma''$ is derivable, $t' = \theta \gg t_1 : t'_2$ and $\sigma'' = \sigma'$.

If $\text{match}(e, \theta)$ is undefined and $((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2) \xrightarrow{e} t'; \sigma'$ is derivable, then there is no transition derivable for $\theta : \epsilon$ and, hence, for $((\theta : \epsilon) * \wedge t_1)$, since there is only one rule for the $*$ and $\wedge$ operators; this means that rule (shuffle-r) has been necessarily applied, and that there exists a transition derivable for $((\bar{\theta} : \epsilon) * \wedge t_2)$ and by rule (and) $t_2 \xrightarrow{e} t'_2; \sigma''$ is derivable; by the assumption on match , $\text{match}(e, \bar{\theta}) = \emptyset$, therefore by rules (shuffle-r), (and), $(*)$ and (pre) $t' = ((\theta : \epsilon) * \wedge t_1) \mid ((\epsilon \cdot (\bar{\theta} : \epsilon) *) \wedge t'_2)$ and $\sigma'' = \sigma'$.

From the facts above and Definition 3.1 we have:

- $\epsilon \in [\![\theta \gg t_1 : t_2]\!]$ iff $E(\theta \gg t_1 : t_2)$ is derivable iff $E(t_1)$ and $E(t_2)$ are derivable iff $E(((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2))$ is derivable iff $\epsilon \in [\![((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2)]\!]$;
- for all e s.t. $\text{match}(e, \theta) = \sigma$, $e\bar{e} \in [\![\theta \gg t_1 : t_2]\!]$ iff $\theta \gg t_1 : t_2 \xrightarrow{e} t'; \sigma'$ is derivable and $\bar{e} \in [\![\sigma'' t']\!]$ iff $t_1 \xrightarrow{e} t'_1; \sigma'$ is derivable, $t' = \theta \gg t'_1 : t_2$, $\sigma'' = \sigma \cup \sigma'$ and $\bar{e} \in [\![\sigma'' \theta \gg \sigma'' t'_1 : \sigma'' t_2]\!]$ iff $t_1 \xrightarrow{e} t'_1; \sigma'$ is derivable, $\sigma'' = \sigma \cup \sigma'$ and $\bar{e} \in [\![\sigma''((\theta : \epsilon) * \wedge t'_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2))]\!]$ (by coinductive hypothesis and def. of substitution application) iff $((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2) \xrightarrow{e} ((\epsilon \cdot (\theta : \epsilon) *) \wedge t'_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2); \sigma''$ is derivable, $\sigma'' = \sigma \cup \sigma'$ and $\bar{e} \in [\![\sigma''((\theta : \epsilon) * \wedge t'_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2))]\!]$ iff $((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2) \xrightarrow{e} ((\epsilon \cdot (\theta : \epsilon) *) \wedge t'_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2); \sigma''$ is derivable, $\sigma'' = \sigma \cup \sigma'$ and $\bar{e} \in [\![\sigma''(((\epsilon \cdot (\theta : \epsilon) *) \wedge t'_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2)))]\!]$ (since $E(\epsilon \cdot t)$ and $\epsilon \cdot t \xrightarrow{e} t'; \sigma$ are derivable iff $E(t)$ and $t \xrightarrow{e} t'; \sigma$ are derivable, see the proof of $\epsilon \cdot t \equiv t$ in Appendix A.2) iff $e\bar{e} \in [\![((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2)]\!]$;
- for all e s.t. $\text{match}(e, \theta)$ is undefined, by the assumption on match , $\text{match}(e, \bar{\theta}) = \emptyset$, therefore $e\bar{e} \in [\![\theta \gg t_1 : t_2]\!]$ iff $\theta \gg t_1 : t_2 \xrightarrow{e} t'; \sigma'$ is derivable and $\bar{e} \in [\![\sigma' t']\!]$ iff $t_2 \xrightarrow{e} t'_2; \sigma'$ is derivable, $t' = \theta \gg t_1 : t'_2$ and $\bar{e} \in [\![\sigma' \theta \gg \sigma' t_1 : \sigma' t'_2]\!]$ iff $t_2 \xrightarrow{e} t'_2; \sigma'$ is derivable and $\bar{e} \in [\![\sigma'((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t'_2))]\!]$ (by coinductive hypothesis and def. of substitution application) iff $((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2) \xrightarrow{e} ((\theta : \epsilon) * \wedge t_1) \mid ((\epsilon \cdot (\bar{\theta} : \epsilon) *) \wedge t'_2); \sigma'$ is derivable and $\bar{e} \in [\![\sigma'((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t'_2))]\!]$ iff $((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2) \xrightarrow{e} ((\theta : \epsilon) * \wedge t_1) \mid ((\epsilon \cdot (\bar{\theta} : \epsilon) *) \wedge t'_2); \sigma'$ is derivable and $\bar{e} \in [\![\sigma'((\theta : \epsilon) * \wedge t_1) \mid ((\epsilon \cdot (\bar{\theta} : \epsilon) *) \wedge t'_2))]\!]$ (since $E(\epsilon \cdot t)$ and $\epsilon \cdot t \xrightarrow{e} t'; \sigma$ are derivable iff $E(t)$ and $t \xrightarrow{e} t'; \sigma$ are derivable, see the proof of $\epsilon \cdot t \equiv t$ in Appendix A.2) iff $e\bar{e} \in [\![((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2)]\!]$.

Therefore we can conclude $\theta \gg t_1 : t_2 \equiv ((\theta : \epsilon) * \wedge t_1) \mid ((\bar{\theta} : \epsilon) * \wedge t_2)$. $\square$

A.2. Laws of the trace calculus

Identity of the shuffle.

$$\epsilon \mid t \equiv t \equiv t \mid \epsilon.$$

Proof. By rule (e- ϵ) and (n- ϵ) $E(\epsilon)$ and $\epsilon \xrightarrow{e}$ are derivable for any event e , and by the rewriting rules, $\epsilon \xrightarrow{e} t; \sigma$ is not derivable for any event e , term t , and substitution σ ; therefore, since rule (e-al) is the only applicable one for shuffle, we deduce that $E(\epsilon \mid t)$ is derivable iff $E(t)$ is derivable iff $E(t \mid \epsilon)$ is derivable.

From the facts above, for term $\epsilon \mid t$ rule (shuffle-l) is not applicable and directly from rule (shuffle-r) we deduce that $\epsilon \mid t \xrightarrow{e} t''; \sigma$ iff $t'' = \epsilon \mid t'$ and $t \xrightarrow{e} t'; \sigma$, while for term $t \mid \epsilon$ rule (shuffle-r) is not applicable and directly from rule (shuffle-l) we deduce that $t \mid \epsilon \xrightarrow{e} t''; \sigma$ iff $t'' = t' \mid \epsilon$ and $t \xrightarrow{e} t'; \sigma$.

Finally, from the facts above and Definition 3.1 we have:

- $\epsilon \in [\![\epsilon \mid t]\!]$ iff $E(\epsilon \mid t)$ is derivable iff $E(t)$ is derivable iff $\epsilon \in [\![t]\!]$;
- similarly as the previous case, $\epsilon \in [\![t \mid \epsilon]\!]$ iff $\epsilon \in [\![t]\!]$;
- $e\bar{e} \in [\![\epsilon \mid t]\!]$ iff $\epsilon \mid t \xrightarrow{e} t''; \sigma$ is derivable and $\bar{e} \in [\![\sigma t'']]\!]$ iff $t'' = \epsilon \mid t'$, $t \xrightarrow{e} t'; \sigma$ is derivable and $\bar{e} \in [\![\epsilon \mid \sigma t']]\!]$ iff $t \xrightarrow{e} t'; \sigma$ is derivable and (by coinductive hypothesis) $\bar{e} \in [\![\sigma t']]\!]$ iff $e\bar{e} \in [\![t]\!]$;
- similarly as the previous case, $e\bar{e} \in [\![t \mid \epsilon]\!]$ iff $e\bar{e} \in [\![t]\!]$.

Therefore we can conclude $\epsilon \mid t \equiv t \equiv t \mid \epsilon$. $\square$

Identity of the intersection.

$$1 \wedge t \equiv t \equiv t \wedge 1.$$

Proof. We recall that 1 is a shorthand for the term t s.t. $t = \epsilon \vee (\tau : t) \vee (\bar{\tau} : t)$, for any fixed event type name⁴⁸ τ .

⁴⁸ We recall that τ is an abbreviation for the ground event type pattern $\tau()$.

By rule (e- ϵ) and (e-or-l) $E(1)$ is derivable, and by rule (e-al) $E(1 \wedge t)$ is derivable iff $E(t)$ is derivable iff $E(t \wedge 1)$ is derivable. By definition of $\bar{\tau}$, we have that for all events e , $\text{match}(e, \tau)$ is defined iff $\text{match}(e, \bar{\tau})$ is undefined; furthermore, since τ denotes a ground event type pattern, by virtue of the assumptions on match , if $\sigma = \text{match}(e, \tau)$, then $\sigma = \emptyset$, and the same implication holds for $\bar{\tau}$. Therefore, by rules (or-l), (or-r), (pre), (n- ϵ), (n-pre) and (n-or) we have that $1 \xrightarrow{e} 1; \emptyset$ is derivable for all events e .

Hence, by rule (and), $1 \wedge t \xrightarrow{e} t''; \sigma$ iff $t'' = 1 \wedge t'$ and $t \xrightarrow{e} t'; \sigma$ and $t \wedge 1 \xrightarrow{e} t''; \sigma$ iff $t'' = t' \wedge 1$ and $t \xrightarrow{e} t'; \sigma$.

Finally, from the facts above and Definition 3.1 we have:

- $\epsilon \in \llbracket 1 \wedge t \rrbracket$ iff $E(1 \wedge t)$ is derivable iff $E(t)$ is derivable iff $\epsilon \in \llbracket t \rrbracket$;
- similarly as the previous case, $\epsilon \in \llbracket t \wedge 1 \rrbracket$ iff $\epsilon \in \llbracket t \rrbracket$;
- $e\bar{e} \in \llbracket 1 \wedge t \rrbracket$ iff $1 \wedge t \xrightarrow{e} t''; \sigma$ is derivable and $\bar{e} \in \llbracket \sigma t'' \rrbracket$ iff $t'' = 1 \wedge t'$, $t \xrightarrow{e} t'; \sigma$ is derivable and $\bar{e} \in \llbracket 1 \wedge \sigma t' \rrbracket$ iff $t \xrightarrow{e} t'; \sigma$ is derivable and (by coinductive hypothesis) $\bar{e} \in \llbracket \sigma t' \rrbracket$ iff $e\bar{e} \in \llbracket t \rrbracket$;
- similarly as the previous case, $e\bar{e} \in \llbracket t \wedge 1 \rrbracket$ iff $e\bar{e} \in \llbracket t \rrbracket$.

Therefore we can conclude $1 \wedge t \equiv t \equiv t \wedge 1$. $\square$

Identity of the concatenation.

$$\epsilon \cdot t \equiv t \equiv t \cdot \epsilon.$$

Proof. By rule (e- ϵ) and (n- ϵ) $E(\epsilon)$ and $\epsilon \xrightarrow{e}$ are derivable for any event e , and by the rewriting rules, $\epsilon \xrightarrow{e} t; \sigma$ is not derivable for any event e , term t , and substitution σ ; therefore, since rule (e-al) is the only applicable one for concatenation we deduce that $E(\epsilon \cdot t)$ is derivable iff $E(t)$ is derivable iff $E(t \cdot \epsilon)$ is derivable.

From the facts above, for term $\epsilon \cdot t$ rule (cat-l) is not applicable and directly from rule (cat-r) we deduce that $\epsilon \cdot t \xrightarrow{e} t'; \sigma$ iff $t \xrightarrow{e} t'; \sigma$, while for term $t \cdot \epsilon$ rule (cat-r) is not applicable and directly from rule (cat-l) we deduce that $t \cdot \epsilon \xrightarrow{e} t''; \sigma$ iff $t'' = t' \cdot \epsilon$ and $t \xrightarrow{e} t'; \sigma$.

Finally, from the facts above and Definition 3.1 we have:

- $\epsilon \in \llbracket \epsilon \cdot t \rrbracket$ iff $E(\epsilon \cdot t)$ is derivable iff $E(t)$ is derivable iff $\epsilon \in \llbracket t \rrbracket$;
- similarly as the previous case, $\epsilon \in \llbracket t \cdot \epsilon \rrbracket$ iff $\epsilon \in \llbracket t \rrbracket$;
- $e\bar{e} \in \llbracket \epsilon \cdot t \rrbracket$ iff $\epsilon \cdot t \xrightarrow{e} t'; \sigma$ is derivable and $\bar{e} \in \llbracket \sigma t' \rrbracket$ iff $t \xrightarrow{e} t'; \sigma$ is derivable and $\bar{e} \in \llbracket \sigma t' \rrbracket$ iff $e\bar{e} \in \llbracket t \rrbracket$;
- similarly as the previous case and the proof for $t \mid \epsilon \equiv t$, $e\bar{e} \in \llbracket t \cdot \epsilon \rrbracket$ iff $e\bar{e} \in \llbracket t \rrbracket$.

Therefore we can conclude $\epsilon \cdot t \equiv t \equiv t \cdot \epsilon$. $\square$

Invariance of 1 w.r.t. filter. The following law holds for any possible ground event type pattern θ :

$$\theta \gg 1 \equiv 1.$$

Proof. We recall that $\theta \gg 1$ is an abbreviation for $\theta \gg 1 : 1$. We have already shown in the proof of the identity law for intersection that $E(1)$ and $1 \xrightarrow{e} 1; \emptyset$ are derivable for all events e ; therefore, by rule (e-filter), $E(\theta \gg 1)$ is derivable. Furthermore, because by definition 1 contains only ground event type patterns, if $1 \xrightarrow{e} 1; \sigma$ is derivable, then $\sigma = \emptyset$.

Let e be an event s.t. $\sigma = \text{match}(e, \theta)$, then for the term $\theta \gg 1$ the only applicable rule is (filter-t) and the only rewriting step that can be derived is $\theta \gg 1 \xrightarrow{e} \theta \gg 1; \sigma$; furthermore, because θ is ground and $\sigma = \emptyset \cup \emptyset$ iff $\sigma = \emptyset$, we deduce that $\theta \gg 1 \xrightarrow{e} \theta \gg 1; \sigma$ is derivable only for $\sigma = \emptyset$. On the other hand, if $\text{match}(e, \theta)$ is undefined, then for the term $\theta \gg 1$ the only applicable rule is (filter-f) and the only rewriting step that can be derived is $\theta \gg 1 \xrightarrow{e} \theta \gg 1; \emptyset$; therefore, for all events e , $\theta \gg 1 \xrightarrow{e} t'; \sigma$ iff $t' = \theta \gg 1$ and $\sigma = \emptyset$.

Finally, from the facts above and Definition 3.1 we have:

- $\epsilon \in \llbracket \theta \gg 1 \rrbracket$ and $\epsilon \in \llbracket 1 \rrbracket$;
- $e\bar{e} \in \llbracket \theta \gg 1 \rrbracket$ iff $\theta \gg 1 \xrightarrow{e} t'; \sigma$ is derivable and $\bar{e} \in \llbracket \sigma t' \rrbracket$ iff $t' = \theta \gg 1$, $\sigma = \emptyset$ and $\bar{e} \in \llbracket \theta \gg 1 \rrbracket$ iff (by coinductive hypothesis) $\bar{e} \in \llbracket 1 \rrbracket$ and $1 \xrightarrow{e} 1; \emptyset$ is derivable iff $e\bar{e} \in \llbracket 1 \rrbracket$;

Therefore we can conclude $\theta \gg 1 \equiv 1$. $\square$

*Relationship between the * and + operators.*

$$t + \equiv t \cdot t *.$$

Proof. By the only applicable rules (e-+) and (e-al) we have that both $E(t+)$ and $E(t \cdot t*)$ are derivable iff $E(t)$ is derivable.

By the only applicable rule (+) we have that $t+ \xrightarrow{e} t''; \sigma$ iff $t \xrightarrow{e} t'; \sigma$ is derivable and $t'' = t' \cdot t*$.

If $t \xrightarrow{e} t'; \sigma$ is derivable, then by rule (cat-l) $t \cdot t* \xrightarrow{e} t' \cdot t*; \sigma$ is derivable; by the unique rule (*), $t* \xrightarrow{e} t''; \sigma$ is derivable iff $t \xrightarrow{e} t'; \sigma$ is derivable, therefore only rule (cat-l) can be applied for $t \cdot t*$ and, hence, if $t \cdot t* \xrightarrow{e} t''; \sigma$ is derivable, then $t \xrightarrow{e} t'; \sigma$ is derivable and $t'' = t' \cdot t*$.

From the facts above and Definition 3.1 we have:

- $\epsilon \in \llbracket t+ \rrbracket$ iff $E(t+)$ is derivable iff $E(t)$ is derivable iff $E(t \cdot t*)$ is derivable iff $\epsilon \in \llbracket t \cdot t* \rrbracket$;
- $e\bar{e} \in \llbracket t+ \rrbracket$ iff $t+ \xrightarrow{e} t''; \sigma$ is derivable and $\bar{e} \in \llbracket \sigma t'' \rrbracket$ iff $t \xrightarrow{e} t'; \sigma$ is derivable, $t'' = t' \cdot t*$ and $\bar{e} \in \llbracket \sigma t'' \rrbracket$ iff $t \cdot t* \xrightarrow{e} t''; \sigma$ is derivable, $t'' = t' \cdot t*$, $\bar{e} \in \llbracket \sigma t'' \rrbracket$ iff $e\bar{e} \in \llbracket t \cdot t* \rrbracket$.

Therefore we can conclude $t+ \equiv t \cdot t*$. $\square$

A.3. Deterministic semantics

We formally prove that the semantics of the trace calculus presented in Section 3 is deterministic: if both $t \xrightarrow{e} t_1; \sigma_1$ and $t \xrightarrow{e} t_2; \sigma_2$ are derivable, then $t_1 = t_2$ and $\sigma_1 = \sigma_2$.

The proof works for the specific instantiation where the relation *match* is defined as in Section 4, and happens to be a partial function (see the proof below): if both $\text{match}(e, \theta) = \sigma_1$ and $\text{match}(e, \theta) = \sigma_2$ are derivable, then $\sigma_1 = \sigma_2$; anyway, the result proved here is more general, since it shows that whenever *match* is deterministic (that is, *match* is a partial function), the induced semantics of the trace calculus is deterministic as well.

Lemma A.1. *If $E(t)$ is derivable, then $NE(t)$ is not derivable.*

Proof. The proof proceeds by induction on the rules defining $E(t)$ and by case analysis on the shape of t ; we only show the details for the only base case $t = \epsilon$ and for $t = t_1 \vee t_2$.

- $t = \epsilon$: $E(t)$ is derivable by axiom (e- ϵ), but $NE(t)$ is not derivable because no conclusion in the rules for $NE(t)$ matches ϵ .
- $t = t_1 \vee t_2$: if $E(t)$ is derivable, then the derivation can only be obtained by first applying rule either (e-or-l) or (e-or-r), that is, either $E(t_1)$ or $E(t_2)$ must be derivable, therefore by induction, either $NE(t_1)$ or $NE(t_2)$ cannot be derived; therefore $NE(t_1 \vee t_2)$ is derivable, because (ne-or) is the only rule for $NE(t)$ whose conclusion matches $t_1 \vee t_2$, and it is applicable only if both $NE(t_1)$ and $NE(t_2)$ are derivable. $\square$

Remark: the lemma above does not prevent that there exists t s.t. both $E(t)$ and $NE(t)$ are not derivable; indeed, this happens, for instance, for $t = \text{if } (x) t_1 \text{ else } t_2$, simply because neither $\text{ev}(x) = \text{true}$, nor $\text{ev}(x) = \text{false}$ is derivable.

The following lemmas prove that function *match* as defined in Section 4 is deterministic. We recall that the meta-variable $\sigma^\perp$ denotes either a pattern substitution, or *fail*.

Lemma A.2. *If $v:vp \rightsquigarrow \sigma_1^\perp$ and $v:vp \rightsquigarrow \sigma_2^\perp$ are derivable, then $\sigma_1^\perp = \sigma_2^\perp$.*

Proof. The proof proceeds by induction on the rules defining $v:vp \rightsquigarrow \sigma^\perp$ and by case analysis on the shape of vp ; we only show the details for the base cases and for $vp = [vp_1, \dots, vp_n]$.

- $vp = x$ or $vp = w$ or $vp = v$: in such cases only one axiom and no other rules are applicable.
- $vp = [vp_1, \dots, vp_n]$: in this case the first applicable rule in the derivation of $v:vp \rightsquigarrow \sigma_1^\perp$ can be one between (arr), (arrf1), (arrf2) and (arrf3); let us assume that the first applied rule is (arr); hence, $v = [v_1, \dots, v_n]$, $\sigma_1^\perp = \sigma$, for all $i \in \{1, \dots, n\}$ $v_i:vp_i \rightsquigarrow \sigma_i$ is derivable, and $\sigma = \sigma_1 \cup \dots \cup \sigma_n$. We show that the first applied rule for the derivation of $v:vp \rightsquigarrow \sigma_2^\perp$ must be (arr) as well.

Let us assume that the first applied rule is (arrf1), hence for all $i \in \{1, \dots, n\}$ $v_i:vp_i \rightsquigarrow \sigma'_i$ is derivable and, by induction, for all $i \in \{1, \dots, n\}$ $\sigma_i = \sigma'_i$, but then the side condition $\sigma'_1 \cup \dots \cup \sigma'_n$ fails of (arrf1) leads to a contradiction.

Let us assume that the first applied rule is (arrf2), hence there exists $i \in \{1, \dots, n\}$ s.t. $v_i:vp_i \rightsquigarrow \text{fail}$ is derivable, but this immediately leads to a contradiction, because $v_i:vp_i \rightsquigarrow \sigma_i$ is derivable, and by induction, $v_i:vp_i \rightsquigarrow \sigma_1^\perp$ and $v_i:vp_i \rightsquigarrow \sigma_2^\perp$ implies $\sigma_1^\perp = \sigma_2^\perp$.

Finally, rule (arrf3) is not applicable because of its side condition.

Since we have proved that the first applied rule for the derivation of $v:vp \rightsquigarrow \sigma_2^\perp$ is (arr), we know that $\sigma_2^\perp = \sigma'$, for all $i \in \{1, \dots, n\}$ $v_i:vp_i \rightsquigarrow \sigma'_i$ is derivable, and $\sigma' = \sigma'_1 \cup \dots \cup \sigma'_n$; hence, by induction, for all $i \in \{1, \dots, n\}$, $\sigma_i = \sigma'_i$, and, therefore, $\sigma = \sigma'$.

The proof proceeds in a similar way when the first applied rule for the derivation of $v:[vp_1, \dots, vp_n] \rightsquigarrow \sigma_1^\perp$ is (arrf1), (arrf2) or (arrf3). $\square$

Lemma A.3. If $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma_1^\perp$ and $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma_2^\perp$ are both derivable, then $\sigma_1^\perp = \sigma_2^\perp$.

Proof. The proof proceeds by induction on the rules defining $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma^\perp$ and by case analysis on the shape of Θ ; we only show the details for $\Theta = \tau(x_1, \dots, x_n) \text{ match op}$ and $\Theta = \tau(x_1, \dots, x_n) \text{ match etp}_1 | \dots | etp_k$.

- $\Theta = \tau(x_1, \dots, x_n) \text{ match op}$: in this case the first applicable rule in the derivation of $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma_1^\perp$ can be (mtch-op) or (mtch-f1). Let us assume that (mtch-op) is the first applied rule, hence, $e:op' \rightsquigarrow \sigma_1^\perp$ is derivable. Because the side conditions of (mtch-op) and (mtch-f1) are disjoint, (mtch-op) must also be the first applied rule in the derivation for $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma_2^\perp$, therefore $e:op' \rightsquigarrow \sigma_2^\perp$ is derivable, and we can conclude $\sigma_1^\perp = \sigma_2^\perp$ by Lemma A.2.

Let us assume that (mtch-f1) is the first applied rule in the derivation of $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma_1^\perp$. Because the side conditions of (mtch-op) and (mtch-f1) are disjoint, (mtch-f1) must also be the first applied rule in the derivation for $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma_2^\perp$, therefore $\sigma_1^\perp = fail = \sigma_2^\perp$.

- $\Theta = \tau(x_1, \dots, x_n) \text{ match etp}_1 | \dots | etp_k$: in this case the first applicable rule in the derivation of $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma_1^\perp$ can be (mtch-der), (mtch-f1) or (mtch-f3). Let us assume that (mtch-der) is the first applied rule. Because the side conditions of (mtch-der) and (mtch-f1) are disjoint, (mtch-f1) cannot be the first applied rule in the derivation for $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma_2^\perp$. Let us assume (mtch-f3) is the first applied rule; then we would derive both $match_{\bar{\Theta}}(e, \bar{\Theta}, \sigma etp_i) = \sigma'_1$ and $match_{\bar{\Theta}}(e, \bar{\Theta}, \sigma etp_i) = fail$, by contradicting the inductive hypothesis, hence (mtch-der) must be the first applied rule also in the derivation of $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma_2^\perp$. Hence, both $match_{\bar{\Theta}}(e, \bar{\Theta}, \sigma etp_i) = \sigma'_1$ and $match_{\bar{\Theta}}(e, \bar{\Theta}, \sigma etp_i) = \sigma'_2$ are derivable, and by induction we conclude $\sigma_1^\perp = \sigma'_1 = \sigma'_2 = \sigma_2^\perp$.

A similar proof can be employed in the case (mtch-f3) is the first rule applied in the derivation of $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma_1^\perp$.

If (mtch-f1) is the first rule applied in the derivation of $match_{\bar{\Theta}}(e, \Theta, etp) = \sigma_1^\perp$, then the proof proceeds as for the case $\Theta = \tau(x_1, \dots, x_n) \text{ match op}$. $\square$

Lemma A.4. If $match_{\bar{\Theta}}(e, \bar{\Theta}, etp) = \sigma_1^\perp$ and $match_{\bar{\Theta}}(e, \bar{\Theta}, etp) = \sigma_2^\perp$ are both derivable, then $\sigma_1^\perp = \sigma_2^\perp$.

Proof. The proof proceeds by induction on the rules defining $match_{\bar{\Theta}}(e, \bar{\Theta}, etp) = \sigma^\perp$, and exploits Lemma A.3. $\square$

Lemma A.5. If $match_{\bar{\Theta}}(e, etp) = \sigma_1$ and $match_{\bar{\Theta}}(e, etp) = \sigma_2$ are both derivable, then $\sigma_1 = \sigma_2$.

Proof. A corollary of Lemma A.4. $\square$

Lemma A.5 shows that any sequence of RML event type definitions gives raise to a *match* relation⁴⁹ which is deterministic (that is, a partial function).

This is essential to ensure that the whole semantics of the trace calculus is deterministic as well; indeed, what we prove in the remaining part of this section is that any deterministic relation (that is, partial function) *match* induces a deterministic semantics on the trace calculus. For this reason, in all claims below we assume that *match* is a partial function from events and event types into substitutions.

Lemma A.6. If $t_1 \xrightarrow{e} t_2; \sigma$ is derivable for some t_2, σ , then $t_1 \not\xrightarrow{e}$ is not derivable.

Proof. The proof proceeds by induction on the rules defining $t_1 \xrightarrow{e} t_2; \sigma$ and by case analysis on the shape of t_1 ; we only show the details for the only base case $t_1 = \theta : t'$ and for $t_1 = t' \vee t''$ and $t_1 = t' \cdot t''$.

- $t_1 = \theta : t'$: if $t_1 \xrightarrow{e} t_2; \sigma$ is derivable, then axiom (pre) has been necessarily applied, hence, by its side condition, there exists σ s.t. $\sigma = match(e, \theta)$; therefore, the side condition of the unique rule for $\theta : t' \not\xrightarrow{e}$ cannot be applied, because its side condition does not hold, hence $t_1 \not\xrightarrow{e}$ cannot be derived.
- $t_1 = t' \vee t''$: in this case the first applied rule in the derivation can only be (or-l) or (or-r). For the former rule we deduce that $t' \xrightarrow{e} t_2; \sigma$ is derivable, hence, $t' \not\xrightarrow{e}$ cannot be derived by inductive hypothesis. For the latter rule we deduce that $t'' \xrightarrow{e} t_2; \sigma$ is derivable, hence, $t'' \not\xrightarrow{e}$ cannot be derived by inductive hypothesis. Since either $t' \not\xrightarrow{e}$ or $t'' \not\xrightarrow{e}$ are not derivable, the only possible rule (n-or) for $t' \vee t'' \not\xrightarrow{e}$ cannot be applied, and, hence, $t' \vee t'' \not\xrightarrow{e}$ is not derivable.
- $t_1 = t' \cdot t''$: in this case the first applied rule in the derivation can only be (cat-l) or (cat-r). For the former rule we deduce that $t' \xrightarrow{e} t_2; \sigma$ is derivable, hence, $t' \not\xrightarrow{e}$ cannot be derived by inductive hypothesis. Therefore, neither of the only matching rules (n-cat-l) and (n-cat-r) can be applied, and, hence, $t_1 \not\xrightarrow{e}$ is not derivable.

⁴⁹ We recall that while $match_{\bar{\Theta}}$ is a more general relation working on event type patterns, the *match* relation used in the semantics of the trace calculus is only defined for event types, which are a syntactic sub-category of event type patterns.

For the latter rule we deduce that $E(t')$ and $t'' \xrightarrow{e} t_2; \sigma$ are derivable, hence, by Lemma A.1 $NE(t')$ is not derivable, and, by inductive hypothesis, $t'' \not\xrightarrow{e}$ is not derivable, therefore, neither of the only matching rules (n-cat-l) and (n-cat-r) can be applied, and, hence, $t_1 \not\xrightarrow{e}$ is not derivable. $\square$

Remark: as for Lemma A.1, also Lemma A.6 does not prevent that there exists t s.t. $t \xrightarrow{e} t'; \sigma$ and $t \not\xrightarrow{e}$ are not derivable for all t', σ ; again, this happens, for instance, for $t = \text{if } (x) t_1 \text{ else } t_2$, simply because neither $ev(x) = \text{true}$, nor $ev(x) = \text{false}$ is derivable.

Before stating the main theorem, we need to prove that evaluation of data expressions is deterministic, under the implicit assumption that the used unary and binary operations are deterministic.

Lemma A.7. *If $ev(d) = v_1$ and $ev(d) = v_2$ are derivable, then $v_1 = v_2$.*

Proof. The proof proceeds by induction on the rules defining $ev(d)$ and by case analysis on the shape of d and it exploits the assumption that the used unary and binary operations are deterministic. $\square$

The final theorem shows that the semantics of the trace calculus is deterministic.

Theorem A.1. *If $t \xrightarrow{e} t_1; \sigma_1$ and $t \xrightarrow{e} t_2; \sigma_2$ are derivable, then $t_1 = t_2$ and $\sigma_1 = \sigma_2$.*

Proof. The proof proceeds by induction on the sum of the depths of the two derivations for $t \xrightarrow{e} t_1; \sigma_1$ and $t \xrightarrow{e} t_2; \sigma_2$, and by case analysis on the shape of t .

- if $t = \epsilon$, then the claim holds vacuously, since there cannot be reduction steps.
- if $t = \theta : t'$, then the derivations can only be obtained by applying rule (pre), and the claim can be derived directly from the assumption that *match* is a partial function, as stated before Lemma A.6.
- if $t = t' \vee t''$, then the derivations can only be obtained by first applying rule (or-l) or (or-r). Because of the premises $t' \xrightarrow{e} t_1; \sigma_1$ and $t_1 \not\xrightarrow{e}$ of rules (or-l) and (or-r), respectively, by Lemma A.6 we deduce that the same rule has necessarily been applied. In case of rule (or-l), $t' \xrightarrow{e} t_1; \sigma_1$ and $t'' \xrightarrow{e} t_2; \sigma_2$ are derivable, and the sum of the depths of the derivations is strictly less than the sum of the depths of the derivations for $t \xrightarrow{e} t_1; \sigma_1$ and $t \xrightarrow{e} t_2; \sigma_2$; hence, by induction, $t_1 = t_2$ and $\sigma_1 = \sigma_2$. The proof proceeds in the same way when the applied rule is (or-r).
- if $t = t' \wedge t''$, then the proof is similar but simpler than the case $t = t' \vee t''$, because only one rule is applicable; it also exploits the fact that $\sigma_1 = \sigma'_1 \cup \sigma''_1$, $\sigma_2 = \sigma'_2 \cup \sigma''_2$, $\sigma'_1 = \sigma'_2$ and $\sigma''_1 = \sigma''_2$ imply $\sigma_1 = \sigma_2$.
- if $t = t' \mid t''$, then the proof proceeds as for the case $t = t' \vee t''$.
- if $t = t' \cdot t''$, then the proof proceeds as for the case $t = t' \vee t''$.
- if $t = \{\text{let } x; t'\}$, then the derivations can only be obtained by first applying either rule (par-t) or (par-f); since the two rules share the same hypothesis, independently of the applied rule, $t' \xrightarrow{e} t''; \sigma'$ and $t' \xrightarrow{e} t''; \sigma''$ are derivable, and the sum of the depths of the derivations is strictly less than the sum of the depths of the derivations for $t \xrightarrow{e} t_1; \sigma_1$ and $t \xrightarrow{e} t_2; \sigma_2$. Hence, by induction we deduce that $t'' = t'''$ and $\sigma' = \sigma''$; from the equality $\sigma' = \sigma''$ and the two side conditions of rules (par-t) or (par-f), we deduce that both derivations have been built by applying the same rule, and conclude that $t_1 = t_2$ and $\sigma_1 = \sigma_2$.
- if $t = (\langle x_1, \dots, x_n \rangle. t') \langle d_1, \dots, d_n \rangle$, then the derivations can only be obtained by first applying rule (app). Because of Lemma A.7, both rules can only be instantiated with the same substitution σ , hence we deduce that $\sigma t \xrightarrow{e} t_1; \sigma_1$ and $\sigma t \xrightarrow{e} t_2; \sigma_2$ are derivable, and the sum of the depths of the derivations is strictly less than the sum of the depths of the derivations for $t \xrightarrow{e} t_1; \sigma_1$ and $t \xrightarrow{e} t_2; \sigma_2$. Therefore by inductive hypothesis we conclude that $t_1 = t_2$ and $\sigma_1 = \sigma_2$.
- if $t = \text{if } (d) t' \text{ else } t''$, then the derivations can only be obtained by first applying either rule (cond-t) or (cond-f); by Lemma A.7 we deduce that the same rule has necessarily been applied.

For rule (cond-t) we deduce that $t' \xrightarrow{e} t_1; \sigma_1$ and $t' \xrightarrow{e} t_2; \sigma_2$ are derivable, and the sum of the depths of the derivations is strictly less than the sum of the depths of the derivations for $t \xrightarrow{e} t_1; \sigma_1$ and $t \xrightarrow{e} t_2; \sigma_2$. Therefore by inductive hypothesis we conclude that $t_1 = t_2$ and $\sigma_1 = \sigma_2$.

The proof for rule (cond-f) proceeds in a similar way. $\square$

References

- [1] M. Leucker, C. Schallhart, A brief account of runtime verification, *J. Log. Algebraic Program.* 78 (2009) 293–303.
- [2] A. Bauer, M. Leucker, C. Schallhart, The good, the bad, and the ugly, but how ugly is ugly?, in: O. Sokolsky, S. Tasiran (Eds.), *Runtime Verification*, Springer, Berlin Heidelberg, Berlin, Heidelberg, 2007, pp. 126–138.
- [3] W. Ahrendt, J.M. Chimento, G.J. Pace, G. Schneider, Verifying data- and control-oriented properties combining static and runtime verification: theory and tools, *Form. Methods Syst. Des.* 51 (2017) 200–265.

- [4] J.C. Davis, C.A. Coghlan, F. Servant, D. Lee, The impact of regular expression denial of service (ReDoS) in practice: an empirical study at the ecosystem scale, in: *Proceedings of the 2018 ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2018, Lake Buena Vista, FL, USA, November 04–09, 2018*, 2018, pp. 246–256.
- [5] D. Ancona, S. Drossopoulou, V. Mascardi, Automatic generation of self-monitoring mass from multiparty global session types in Jason, in: *Declarative Agent Languages and Technologies X – 10th International Workshop, DALT 2012, Valencia, Spain, June 4, 2012*, 2012, pp. 76–95, Revised Selected Papers.
- [6] G. Castagna, M. Dezani-Ciancaglini, L. Padovani, On global types and multi-party session, *Log. Methods Comput. Sci.* 8 (2012).
- [7] D. Ancona, V. Bono, M. Bravetti, J. Campos, G. Castagna, P. Deniérou, S.J. Gay, N. Gesbert, E. Giachino, R. Hu, E.B. Johnsen, F. Martins, V. Mascardi, F. Montesi, R. Neykova, N. Ng, L. Padovani, V.T. Vasconcelos, N. Yoshida, Behavioral types in programming languages, *Found. Trends Program. Lang.* 3 (2016) 95–230.
- [8] D. Ancona, A. Ferrando, V. Mascardi, Parametric runtime verification of multiagent systems, in: *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems, AAMAS 2017, São Paulo, Brazil, May 8–12, 2017*, 2017, pp. 1457–1459.
- [9] L. Franceschini, RML: Runtime Monitoring Language, Ph.D. thesis, DIBRIS – University of Genova, March 2020, <http://hdl.handle.net/11567/1001856>.
- [10] F.L. Bellifemine, G. Caire, D. Greenwood, *Developing Multi-Agent Systems with JADE*, John Wiley & Sons, 2007.
- [11] Y. Falcone, K. Havelund, G. Reger, A tutorial on runtime verification, in: *Engineering Dependable Software Systems*, 2013, pp. 141–175.
- [12] P.-M. Deniérou, N. Yoshida, Multiparty session types meet communicating automata, in: *ESOP'12 (Part of ETAPS 2012)*, in: LNCS, Springer, 2012, pp. 194–213.
- [13] L. Aceto, A. Achilleos, A. Francalanza, A. Ingólfssdóttir, K. Lehtinen, Adventures in monitorability: from branching to linear time and back again, *Proc. ACM Program. Lang.* 3 (2019) 52.
- [14] B. Courcelle, Fundamental properties of infinite trees, *Theor. Comput. Sci.* 25 (1983) 95–169.
- [15] A. Frisch, G. Castagna, V. Benzaken, Semantic subtyping: dealing set-theoretically with function, union, intersection, and negation types, *J. ACM* 55 (2008).
- [16] D. Ancona, A. Corradi, Sound and complete subtyping between coinductive types for object-oriented languages, in: *ECOOP 2014*, 2014, pp. 282–307.
- [17] D. Ancona, A. Corradi, Semantic subtyping for imperative object-oriented languages, in: *OOPSLA 2016*, 2016, pp. 568–587.
- [18] X. Leroy, H. Grall, Coinductive big-step operational semantics, *Inf. Comput.* 207 (2009) 284–304.
- [19] V. Kahlon, F. Ivančić, A. Gupta, Reasoning about threads communicating via locks, in: *Proceedings of the 17th International Conference on Computer Aided Verification, CAV'05*, Springer-Verlag, 2005, pp. 505–518.
- [20] M.F. Atig, A. Bouajjani, K.N. Kumar, P. Saivasan, Verification of asynchronous programs with nested locks, in: *FSTTCS 2017*, 2017, 11.
- [21] L. Clemente, F. Herbretreau, G. Sutre, Decidable topologies for communicating automata with FIFO and bag channels, in: *CONCUR 2014*, 2014, pp. 281–296.
- [22] L. Akroun, G. Salaün, Automated verification of automata communicating via FIFO and bag buffers, *Form. Methods Syst. Des.* 52 (2018) 260–276.
- [23] R. Alur, K. Mamouras, C. Stanford, Modular quantitative monitoring, *Proc. ACM Program. Lang.* 3 (2019) 50.
- [24] Y. Bar-Hillel, M. Perles, E. Shamir, On Formal Properties of Simple Phrase Structure Grammars, *Zeitschrift für Phonetik, Sprachwissenschaft und Kommunikationsforschung*, vol. 14, 1961, pp. 143–172; Reprinted in Y. Bar-Hillel, *Language and Information: Selected Essays on Their Theory and Application*, Addison-Wesley, 1964, pp. 116–150.
- [25] J.E. Hopcroft, J.D. Ullman, *Introduction to Automata Theory, Languages and Computation*, Addison-Wesley, 1979.
- [26] T.J. Parr, R.W. Quong, ANTLR: a predicated-LL(k) parser generator, *Softw. Pract. Exp.* 25 (1995) 789–810.
- [27] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- [28] L. Simon, A. Mallya, A. Bansal, G. Gupta, Coinductive logic programming, in: *Logic Programming, 22nd International Conference, ICLP 2006, Proceedings, Seattle, WA, USA, August 17–20, 2006*, 2006, pp. 330–345.
- [29] K. Sen, S. Kalasapur, T. Brutch, S. Gibbs, Jalangi: a selective record-replay and dynamic analysis framework for javascript, in: *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering, ACM, 2013*, pp. 488–498.
- [30] Y. Falcone, S. Krstic, G. Reger, D. Traytel, A taxonomy for classifying runtime verification tools, in: *Runtime Verification – 18th International Conference, Proceedings, RV 2018, Limassol, Cyprus, November 10–13, 2018*, 2018, pp. 241–262.
- [31] G. Reger, K. Havelund, What is a trace? A runtime verification perspective, in: T. Margaria, B. Steffen (Eds.), *Leveraging Applications of Formal Methods, Verification and Validation: Discussion, Dissemination, Applications – 7th International Symposium, ISOla 2016, Proceedings, Part II, Imperial, Corfu, Greece, October 10–14, 2016*, in: *Lecture Notes in Computer Science*, vol. 9953, 2016, pp. 339–355.
- [32] N. Delgado, A.Q. Gates, S. Roach, A taxonomy and catalog of runtime software-fault monitoring tools, *IEEE Trans. Softw. Eng.* 30 (2004) 859–872.
- [33] K. Havelund, A. Goldberg, Verify your runs, in: *Verified Software: Theories, Tools, Experiments, First IFIP TC 2/WG 2.3 Conference, VSTTE 2005, Revised Selected Papers and Discussions, Zurich, Switzerland, October 10–13, 2005*, 2005, pp. 374–383.
- [34] O. Sokolsky, K. Havelund, I. Lee, Introduction to the special section on runtime verification, *STTT* 14 (2012) 243–247.
- [35] E. Bartocci, Y. Falcone, A. Francalanza, G. Reger, Introduction to runtime verification, in: *Lectures on Runtime Verification – Introductory and Advanced Topics*, 2018, pp. 1–33.
- [36] A.N. Prior, Time and Modality, John Locke Lecture, 1957.
- [37] A. Pnueli, The temporal logic of programs, in: *18th Annual Symposium on Foundations of Computer Science, 1977, IEEE, 1977*, pp. 46–57.
- [38] A. Bauer, M. Leucker, C. Schallhart, Runtime verification for LTL and TLTL, *ACM Trans. Softw. Eng. Methodol.* 20 (2011) 14.
- [39] A. Pnueli, L.D. Zuck, In and out of temporal logic, in: *Proceedings of the Eighth Annual Symposium on Logic in Computer, Science (LICS'93)*, Montreal, Canada, June 19–23, 1993, 1993, pp. 124–135.
- [40] D. Ancona, A. Ferrando, V. Mascardi, Comparing trace expressions and linear temporal logic for runtime verification, in: *Theory and Practice of Formal Methods – Essays Dedicated to Frank de Boer on the Occasion of His 60th Birthday*, 2016, pp. 47–64.
- [41] A. Pnueli, A. Zaks, PSL model checking and run-time verification via testers, in: *FM 2006: Formal Methods, 14th International Symposium on Formal Methods, Proceedings, Hamilton, Canada, August 21–27, 2006*, 2006, pp. 573–586.
- [42] A. Cau, H. Zedan, Refining interval temporal logic specifications, in: *Transformation-Based Reactive Systems Development, 4th International AMAST Workshop on Real-Time Systems and Concurrent and Distributed Software, ARTS'97, Proceedings, Palma, Mallorca, Spain, May 21–23, 1997*, 1997, pp. 79–94.
- [43] P. Thati, G. Rosu, Monitoring algorithms for metric temporal logic specifications, *Electron. Notes Theor. Comput. Sci.* 113 (2005) 145–162.
- [44] R. Alur, K. Etessami, P. Madhusudan, A temporal logic of nested calls and returns, in: *Tools and Algorithms for the Construction and Analysis of Systems, 10th International Conference, TACAS 2004, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2004, Proceedings, Barcelona, Spain, March 29 – April 2, 2004*, March 2004, pp. 467–481.
- [45] B. Bollig, N. Decker, M. Leucker, Frequency linear-time temporal logic, in: *Sixth International Symposium on Theoretical Aspects of Software Engineering, TASE 2012, 4–6 July 2012, Beijing, China, 2012*, pp. 85–92.
- [46] K.G. Larsen, Proof systems for satisfiability in Hennessy-Milner logic with recursion, *Theor. Comput. Sci.* 72 (1990) 265–288.
- [47] D. Kozen, Results on the propositional mu-calculus, *Theor. Comput. Sci.* 27 (1983) 333–354.
- [48] C.A.R. Hoare, An axiomatic basis for computer programming, *Commun. ACM* 12 (1969) 576–580.

- [49] B. Meyer, Applying “design by contract”, *IEEE Comput.* 25 (1992) 40–51.
- [50] G.T. Leavens, A.L. Baker, C. Ruby, Preliminary design of JML: a behavioral interface specification language for Java, *SIGSOFT Softw. Eng. Notes* 31 (2006) 1–38.
- [51] D.R. Cok, OpenJML: JML for Java 7 by extending OpenJDK, in: *NASA Formal Methods – Third International Symposium, NFM 2011, Proceedings*, Pasadena, CA, USA, April 18–20, 2011, 2011, pp. 472–479.
- [52] D. Ancona, A. Ferrando, L. Franceschini, V. Mascardi, Parametric trace expressions for runtime verification of Java-like programs, in: *Proceedings of the 19th Workshop on Formal Techniques for Java-Like Programs, FTEJP’17*, ACM, New York, NY, USA, 2017, 10.
- [53] D. Ancona, F. Dagnino, L. Franceschini, A formalism for specification of Java API interfaces, in: *Companion Proceedings for the ISSTA/ECOOP 2018 Workshops*, ISSTA 2018, Amsterdam, Netherlands, July 16–21, 2018, 2018, pp. 24–26.
- [54] F.S. de Boer, S. de Gouw, Combining monitoring with run-time assertion checking, in: *SFM 2014*, 2014, pp. 217–262.
- [55] P. Klint, T. van der Storm, J.J. Vinju, RASCAL: a domain specific language for source code analysis and manipulation, in: *Ninth IEEE International Working Conference on Source Code Analysis and Manipulation, SCAM 2009*, Edmonton, Alberta, Canada, September 20–21, 2009, 2009, pp. 168–177.
- [56] C. Colombo, G.J. Pace, G. Schneider, Dynamic event-based runtime monitoring of real-time and contextual properties, in: *Formal Methods for Industrial Critical Systems, 13th International Workshop, FMICS 2008, L’Aquila, Italy, September 15–16, 2008*, 2008, pp. 135–149, Revised Selected Papers.
- [57] C. Colombo, G.J. Pace, G. Schneider, LARVA – safer monitoring of real-time Java programs, in: *SEFM 2009*, 2009, pp. 33–37.
- [58] P.K. Pandya, Specifying and Deciding Quantified Discrete-time Duration Calculus Formulae using DCVALID, Technical Report, 2000.
- [59] N. Halbwachs, F. Lagnier, C. Ratel, Programming and verifying real-time systems by means of the synchronous data-flow language LUSTRE, *IEEE Trans. Softw. Eng.* 18 (1992) 785–793.
- [60] J. Hoenicke, Combination of processes, data, and time, Ph.D. thesis, Carl von Ossietzky University of Oldenburg, 2006.
- [61] F. Chen, G. Rosu, Parametric trace slicing and monitoring, in: *Tools and Algorithms for the Construction and Analysis of Systems, 15th International Conference, TACAS 2009, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2009, Proceedings*, York, UK, March 22–29, 2009, 2009, pp. 246–261.
- [62] H. Barringer, Y. Falcone, K. Havelund, G. Reger, D.E. Rydeheard, Quantified event automata: towards expressive and efficient runtime monitors, in: *FM 2012: Formal Methods – 18th International Symposium, Proceedings*, Paris, France, August 27–31, 2012, 2012, pp. 68–84.
- [63] G. Reger, H.C. Cruz, D.E. Rydeheard, MarQ: monitoring at runtime with QEA, in: *Tools and Algorithms for the Construction and Analysis of Systems – 21st International Conference, TACAS 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, Proceedings*, London, UK, April 11–18, 2015, 2015, pp. 596–610.
- [64] G. Kiczales, E. Hilsdale, J. Hugunin, M. Kersten, J. Palm, W.G. Griswold, An overview of AspectJ, in: *ECOOP 2001 – Object-Oriented Programming, 15th European Conference, Proceedings*, Budapest, Hungary, June 18–22, 2001, 2001, pp. 327–353.
- [65] G. Kiczales, J. Lamping, A. Mendhekar, C. Maeda, C.V. Lopes, J. Loingtier, J. Irwin, Aspect-oriented programming, in: *ECOOP’97 – Object-Oriented Programming, 11th European Conference, Proceedings*, Jyväskylä, Finland, June 9–13, 1997, 1997, pp. 220–242.
- [66] F. Chen, G. Rosu, Mop: an efficient and generic runtime verification framework, in: *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2007*, October 21–25, 2007, Montreal, Quebec, Canada, 2007, pp. 569–588.
- [67] F. Chen, G. Rosu, Java-MOP: A Monitoring Oriented Programming Environment for Java, *TACAS*, vol. 3440, Springer, 2005, pp. 546–550.
- [68] A. Bauer, M. Leucker, C. Schallhart, Model-based runtime analysis of distributed reactive systems, in: *17th Australian Software Engineering Conference (ASWEC 2006)*, 8–21 April 2006, Sydney, Australia, 2006, pp. 243–252.
- [69] D. Jin, P.O. Meredith, C. Lee, G. Rosu, JavaMOP: efficient parametric runtime monitoring framework, in: *34th International Conference on Software Engineering, ICSE 2012*, June 2–9, 2012, Zurich, Switzerland, 2012, pp. 1427–1430.
- [70] H. Barringer, K. Havelund, TraceContract: a scala DSL for trace analysis, in: *FM 2011: Formal Methods – 17th International Symposium on Formal Methods, Proceedings*, Limerick, Ireland, June 20–24, 2011, pp. 57–72.
- [71] C. Allan, P. Avgustinov, A.S. Christensen, L.J. Hendren, S. Kuzins, O. Lhoták, O. de Moor, D. Sereni, G. Sittampalam, J. Tibble, Adding trace matching with free variables to aspectj, in: *Proceedings of the 20th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2005*, October 16–20, 2005, San Diego, CA, USA, 2005, pp. 345–364.
- [72] H. Barringer, D.E. Rydeheard, K. Havelund, Rule systems for run-time monitoring: from Eagle to RuleR, *J. Log. Comput.* 20 (2010) 675–706.
- [73] H. Barringer, A. Goldberg, K. Havelund, K. Sen, Rule-based runtime verification, in: *Verification, Model Checking, and Abstract Interpretation, 5th International Conference, VMCAI 2004, Proceedings*, Venice, Italy, January 11–13, 2004, 2004, pp. 44–57.
- [74] C. Colombo, A. Francalanza, R. Mizzi, G.J. Pace, PolyLarva: runtime verification with configurable resource-aware monitoring boundaries, in: G. Eleftherakis, M. Hinchey, M. Holcombe (Eds.), *Software Engineering and Formal Methods – 10th International Conference, SEFM 2012, Proceedings*, Thessaloniki, Greece, October 1–5, 2012, in: *Lecture Notes in Computer Science*, vol. 7504, Springer, 2012, pp. 218–232.
- [75] C. Colombo, A. Francalanza, R. Mizzi, G.J. Pace, Extensible technology-agnostic runtime verification, in: B. Buhnova, L. Happe, J. Kofron (Eds.), *Proceedings 10th International Workshop on Formal Engineering Approaches to Software Components and Architectures, FESCA 2013*, Rome, Italy, March 23, 2013, in: *EPTCS*, vol. 108, 2013, pp. 1–15.
- [76] K. Havelund, Rule-based runtime verification revisited, *STTT* 17 (2015) 143–170.
- [77] R. Mizzi, An extensible and configurable runtime verification framework, in: *Master Thesis Submitted in Partial Fulfillment of the Requirements for the Degree of Master of*, in: *Science in Computer Science and Artificial Intelligence at the University of Malta*, 2012.
- [78] C. Forgy, Rete: a fast algorithm for the many patterns/many objects match problem, *Artif. Intell.* 19 (1982) 17–37.
- [79] M. Martin, B. Livshits, M.S. Lam, Finding application errors and security flaws using PQL: a program query language, in: *OOPSLA 2005*, 2005, pp. 365–383.
- [80] E. Bodden, L.J. Hendren, O. Lhoták, A staged static program analysis to improve the performance of runtime monitoring, in: *ECOOP 2007 – Object-Oriented Programming, 21st European Conference, Proceedings*, Berlin, Germany, July 30 – August 3, 2007, 2007, pp. 525–549.
- [81] J.M. Chimento, W. Ahrendt, G.J. Pace, G. Schneider, StaRVOORs: a tool for combined static and runtime verification of Java, in: *Runtime Verification – 6th International Conference, Proceedings*, RV 2015 Vienna, Austria, September 22–25, 2015, 2015, pp. 297–305.
- [82] W. Ahrendt, B. Beckert, R. Bubel, R. Hähnle, P.H. Schmitt, M. Ulbrich (Eds.), *Deductive Software Verification – The KeY Book – From Theory to Practice*, *Lecture Notes in Computer Science*, vol. 10001, Springer, 2016.
- [83] W. Ahrendt, J.M. Chimento, G.J. Pace, G. Schneider, A specification language for static and runtime verification of data and control properties, in: *FM 2015: Formal Methods – 20th International Symposium, Proceedings*, Oslo, Norway, June 24–26, 2015, 2015, pp. 108–125.
- [84] A. Ferrando, L.A. Dennis, D. Ancona, M. Fisher, V. Mascardi, Verifying and validating autonomous systems: towards an integrated approach, in: C. Colombo, M. Leucker (Eds.), *Runtime Verification – 18th International Conference, RV 2018*, Limassol, Cyprus, November 10–13, 2018, *Proceedings*, in: *Lecture Notes in Computer Science*, vol. 11237, Springer, 2018, pp. 263–281.
- [85] A. Ferrando, L.A. Dennis, D. Ancona, M. Fisher, V. Mascardi, Recognising assumption violations in autonomous systems verification, in: E. André, S. Koenig, M. Dastani, G. Sukthankar (Eds.), *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS 2018*, International Foundation for Autonomous Agents and Multiagent Systems Richland, Stockholm, Sweden, July 10–15, 2018, SC, USA/ACM, 2018, pp. 1933–1935, <http://dl.acm.org/citation.cfm?id=3238028>.

- [86] K. Havelund, T. Pressburger, Model checking JAVA programs using JAVA PathFinder, *Int. J. Softw. Tools Technol. Transf.* 2 (2000) 366–381, <https://doi.org/10.1007/s100090050043>.
- [87] S.D. Brookes, C.A.R. Hoare, A.W. Roscoe, A theory of communicating sequential processes, *J. ACM* 31 (1984) 560–599.
- [88] R. Milner, *A Calculus of Communicating Systems*, Lecture Notes in Computer Science, vol. 92, Springer, 1980.
- [89] U. Engberg, M. Nielsen, A calculus of communicating systems with label passing, *DAIMI Rep. Ser.* 15 (1986).
- [90] M. Brörkens, M. Möller, Dynamic event generation for runtime checking using the JDI, *Electron. Notes Theor. Comput. Sci.* 70 (2002) 21–35.
- [91] B. D'Angelo, S. Sankaranarayanan, C. Sánchez, W. Robinson, B. Finkbeiner, H.B. Sipma, S. Mehrotra, Z. Manna, LOLA: runtime monitoring of synchronous systems, in: 12th International Symposium on Temporal Representation and Reasoning (TIME 2005), 23–25 June 2005, Burlington, Vermont, USA, 2005, pp. 166–174.
- [92] P. Faymonville, B. Finkbeiner, S. Schirmer, H. Torfah, A stream-based specification language for network monitoring, in: *Runtime Verification – 16th International Conference, Proceedings, RV 2016, Madrid, Spain, September 23–30, 2016*, 2016, pp. 152–168.
- [93] L. Convent, S. Hungerecker, M. Leucker, T. Scheffel, M. Schmitz, D. Thoma, TeSSLa: temporal stream-based specification language, *CoRR*, arXiv:1808.10717 [abs], 2018.
- [94] M. Leucker, C. Sánchez, T. Scheffel, M. Schmitz, A. Schramm, TeSSLa: runtime verification of non-synchronized real-time streams, in: *Proceedings of the 33rd Annual ACM Symposium on Applied Computing, SAC 2018, Pau, France, April 09–13, 2018*, 2018, pp. 1925–1933.
- [95] L. Aceto, I. Cassar, A. Francalanza, A. Ingólfsdóttir, On runtime enforcement via suppressions, in: 29th International Conference on Concurrency Theory, CONCUR 2018, September 4–7, 2018, Beijing, China, 2018, 34.
- [96] J.D. Ullman, *Principles of Database and Knowledge-Base Systems*, Computer Science Press, 1989.
- [97] C. Colombo, G.J. Pace, G. Schneider, Dynamic event-based runtime monitoring of real-time and contextual properties, in: *Formal Methods for Industrial Critical Systems, 13th International Workshop, FMICS 2008, L'Aquila, Italy, September 15–16, 2008*, 2008, pp. 135–149, Revised Selected Papers.
- [98] J.O. Blech, Y. Falcone, K. Becker, Towards certified runtime verification, in: T. Aoki, K. Taguchi (Eds.), *Formal Methods and Software Engineering – 14th International Conference on Formal Engineering Methods, ICFEM 2012, Proceeding, Kyoto, Japan, November 12–16, 2012*, in: *Lecture Notes in Computer Science*, vol. 7635, Springer, 2012, pp. 494–509.
- [99] R. Calegari, G. Ciatto, V. Mascardi, A. Omicini, Logic-based technologies for multi-agent systems: a systematic literature review, *Auton. Agents Multi-Agent Syst.* 35 (2021), <https://doi.org/10.1007/s10458-020-09478-3>.
- [100] L.A. Dennis, M. Fisher, M.P. Webster, R.H. Bordini, Model checking agent programming languages, *Autom. Softw. Eng.* 19 (2012) 5–63.
- [101] P. Gammie, R. Der van Meyden, MCK: model checking the logic of knowledge, in: R. Alur, D.A. Peled (Eds.), *International Conference on Computer Aided Verification*, in: *Lecture Notes in Computer Science*, vol. 3114, Springer, Boston, MA, USA, 2004, pp. 479–483.
- [102] A. Lomuscio, F. Raimondi, MCMAS: a model checker for multi-agent systems, in: H. Hermanns, J. Palsberg (Eds.), *Tools and Algorithms for the Construction and Analysis of Systems, TACAS 2006*, in: *Lecture Notes in Computer Science*, vol. 3920, Springer, 2006, pp. 450–454.
- [103] M. Alberti, M. Gavanelli, E. Lamma, P. Mello, P. Torroni, The SCIFF abductive proof-procedure, in: S. Bandini, S. Manzoni (Eds.), *AI*IA 2005: Advances in Artificial Intelligence: 9th Congress of the Italian Association for Artificial Intelligence, Proceedings, Milan, Italy, September 21–32*, in: *Lecture Notes in Artificial Intelligence*, vol. 3673, Springer, Berlin, Heidelberg, 2005, pp. 135–147.
- [104] C. Kroiß, A statistical model checker for situation calculus based multi-agent models, in: *2014 International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS'14), IFAAMAS, Richland, SC, USA, 2014*, pp. 1567–1568, <http://dl.acm.org/citation.cfm?id=2615731.2616065>.
- [105] D. Ancona, L. Franceschini, G. Delzanno, M. Leotta, M. Ribaldo, F. Ricca, Towards runtime monitoring of node.js and its application to the Internet of things, in: D. Pianini, G. Salvaneschi (Eds.), *Proceedings of the First Workshop on Architectures, Languages and Paradigms for IoT, ALP4IoT@iFIM 2017, Turin, Italy, September 18, 2017*, in: *EPTCS*, vol. 264, 2017, pp. 27–42.
- [106] A. Ferrando, R. Cardoso, M. Fisher, D. Ancona, L. Franceschini, V. Mascardi, ROSMonitoring: a runtime verification framework for ROS, in: A. Mohammad, X. Dong, M. Russo (Eds.), *Towards Autonomous Robotic Systems. TAROS 2020*, in: *Lecture Notes in Computer Science*, vol. 12228, Springer, 2020, pp. 387–399.
- [107] R.H. Bordini, J.F. Hübner, M. Wooldridge, *Programming Multi-Agent Systems in AgentSpeak Using Jason*, John Wiley & Sons, 2007.
- [108] V. Mascardi, D. Briola, D. Ancona, On the expressiveness of attribute global types: the formalization of a real multiagent system protocol, in: M. Baldoni, C. Baroglio, G. Boella, R. Micalizio (Eds.), *AI*IA 2013: Advances in Artificial Intelligence – XIIIth International Conference of the Italian Association for Artificial Intelligence, Proceedings, Turin, Italy, December 4–6, 2013*, in: *Lecture Notes in Computer Science*, vol. 8249, Springer, 2013, pp. 300–311.
- [109] D. Briola, V. Mascardi, D. Ancona, Distributed runtime verification of JADE multiagent systems, in: D. Camacho, L. Braubach, S. Venticinque, C. Badica (Eds.), *Intelligent Distributed Computing VIII – Proceedings of the 8th International Symposium on Intelligent Distributed Computing, IDC 2014, Madrid, Spain, September 3–5, 2014*, in: *Studies in Computational Intelligence*, vol. 570, Springer, 2014, pp. 81–91.
- [110] D. Briola, V. Mascardi, D. Ancona, Distributed runtime verification of JADE and Jason multiagent systems with Prolog, in: L. Giordano, V. Gliozzi, G.L. Pozzato (Eds.), *Proceedings of the 29th Italian Conference on Computational Logic, Torino, Italy, June 16–18, 2014*, in: *CEUR Workshop Proceedings*, vol. 1195, 2014, pp. 319–323, <http://ceur-ws.org/Vol-1195/short3.pdf>.
- [111] D. Ancona, D. Briola, A. Ferrando, V. Mascardi, MAS-DRiVe: a practical approach to decentralized runtime verification of agent interaction protocols, in: C. Santoro, F. Messina, M.D. Benedetti (Eds.), *Proceedings of the 17th Workshop “From Objects to Agents” Co-located with 18th European Agent Systems, Summer School (EASSS 2016)*, Catania, Italy, July 29–30, 2016, in: *CEUR Workshop Proceedings*, vol. 1664, 2016, pp. 35–43, <http://ceur-ws.org/Vol-1664/w7.pdf>.
- [112] A. Ferrando, D. Ancona, V. Mascardi, Decentralizing MAS monitoring with DecAMon, in: K. Larson, M. Winikoff, S. Das, E.H. Durfee (Eds.), *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems, AAMAS 2017, São Paulo, Brazil, May 8–12, 2017*, ACM, 2017, pp. 239–248, <http://dl.acm.org/citation.cfm?id=3091164>.
- [113] D. Ancona, D. Briola, A. Ferrando, V. Mascardi, Runtime verification of fail-uncontrolled and ambient intelligence systems: a uniform approach, *Intell. Artif.* 9 (2015) 131–148, <https://doi.org/10.3233/IA-150084>.
- [114] D. Ancona, D. Briola, A. Ferrando, V. Mascardi, Global protocols as first class entities for self-adaptive agents, in: G. Weiss, P. Yolum, R.H. Bordini, E. Elkind (Eds.), *Proceedings of the 2015 International Conference on Autonomous Agents and Multiagent Systems, AAMAS 2015, Istanbul, Turkey, May 4–8, 2015*, ACM, 2015, pp. 1019–1029, <http://dl.acm.org/citation.cfm?id=2773282>.
- [115] A. Ferrando, D. Ancona, V. Mascardi, Monitoring patients with hypoglycemia using self-adaptive protocol-driven agents: a case study, in: M. Baldoni, J.P. Müller, I. Nunes, R. Zalila-Wenkstern (Eds.), *Engineering Multi-Agent Systems – 4th International Workshop, EMAS 2016, Revised, Selected, and Invited Papers, Singapore, Singapore, May 9–10, 2016*, in: *Lecture Notes in Computer Science*, vol. 10093, Springer, 2016, pp. 39–58.